VAULTRA

Self-Storage Facility Rental and Management System

Hướng dẫn bảo vệ dự án

Nghiệp vụ 7 Flow · Kiến trúc và mã nguồn · Kiểm thử · Kịch bản demo · Lời thoại thuyết trình · Ngân hàng 100+ câu hỏi phản biện · Rủi ro và cách xử lý tình huống

Thành viên	Phần chính
Khang	Mở đầu, tổng quan, Flow 1 (Đặt kho), Flow 7 (Hỗ trợ)
Triển	Flow 2 (Check-in & bàn giao), Flow 3 (Quản lý thuê & trả kho)
Phúc	Flow 4 (Chính sách, phí, doanh thu), chiến lược kiểm thử
Trường	Flow 5 (Kho & nhân sự), Flow 6 (Gia hạn & quá hạn), kết luận

Phiên bản 1.0 · Tháng 11/2026 · Căn cứ: tài liệu đặc tả chính thức + mã nguồn repo *vaultra* (nhánh main) + bản triển khai vaultra-844.pages.dev

Mục lục

Chương 1. Tổng quan dự án và phân công 5
1.1 VAULTRA trong một trang 5
1.2 Bảy Flow nối với nhau như thế nào 5
1.3 Bảng phân công 7 Flow 6

Chương 2. Kiến trúc và mã nguồn 8
2.1 Kiến trúc tổng thể 8
2.2 Bảng thành phần 9
2.3 Luồng một request đi qua hệ thống 10
2.4 Xác thực và phân quyền 10
2.5 Database 11
2.6 Thanh toán, lưu tệp, email, realtime 11
2.7 Xử lý lỗi, nhật ký, triển khai 12

Chương 3. Phân tích 7 Flow nghiệp vụ 13
3.1 Flow 1 — Đặt kho (Storage Unit Reservation) 13
3.2 Flow 2 — Check-in và bàn giao (Check-in & Handover) 16
3.3 Flow 3 — Quản lý kho đang thuê và trả kho (Rented Storage Unit Management) 18
3.4 Flow 4 — Chính sách, phí và theo dõi doanh thu (Business Rules, Fee Management, Revenue Monitoring) 21
3.5 Flow 5 — Quản lý kho và nhân sự cơ sở (Facility Storage and Staff Management) 23
3.6 Flow 6 — Gia hạn và xử lý quá hạn (Storage Renewal and Overdue Handling) 25
3.7 Flow 7 — Hỗ trợ và xử lý sự cố (Support Request and Issue Handling) 28

Chương 4. Hướng dẫn kiểm thử toàn hệ thống 31
4.1 Các loại kiểm thử áp dụng cho VAULTRA 31
4.2 Test case cho 7 Flow 32
4.3 Kiểm thử nghiệp vụ chuyên sâu 36
4.4 Kiểm thử API bằng Postman 37
4.5 Kiểm tra database 38
4.6 Kiểm thử tự động có trong repo 39

Chương 5. Kịch bản demo thực tế 40
5.1 Thứ tự trình bày và demo 40
5.2 Kịch bản chi tiết từng Flow 40
5.3 Checklist trước ngày bảo vệ 44

Chương 6. Kịch bản thuyết trình cho 4 thành viên 45
6.1 Khang — Mở đầu, tổng quan, Flow 1 và Flow 7 45
6.2 Triển — Flow 2 và Flow 3 46
6.3 Phúc — Flow 4 và tổng kết kiểm thử 48
6.4 Trường — Flow 5, Flow 6 và kết luận 49
6.5 Câu chuyển tiếp giữa các phần 51

Chương 7. Ngân hàng câu hỏi giảng viên 52
7.1 Tổng quan dự án 52
7.2 Phân tích nghiệp vụ 53
7.3 Câu hỏi về 7 Flow 54
7.4 Database 56
7.5 Backend 57
7.6 Frontend 59
7.7 Authentication và Authorization 60

7.8 API 61
7.9 Bảo mật 62
7.10 Testing 63
7.11 Kiến trúc hệ thống 64
7.12 Công nghệ sử dụng 65
7.13 Xử lý lỗi và ngoại lệ 66
7.14 Câu hỏi phản biện chuyên sâu 67
7.15 Câu hỏi riêng cho từng thành viên 68

Chương 8. Phân tích lỗi và rủi ro 70
8.1 Bảng tổng hợp rủi ro 70
8.2 Chi tiết từng rủi ro 71
8.3 Lỗi đã phát hiện và đã sửa trong quá trình hoàn thiện 75
8.4 Đối chiếu các nhóm lỗi thường bị hỏi 76

Chương 9. Checklist ôn tập 77
9.1 Checklist riêng từng thành viên 77
9.2 Checklist kiến thức chung (cả 4 người) 80
9.3 Kế hoạch ôn tập 7 ngày 80

Chương 10. Xử lý tình huống khi bảo vệ 81

Phụ lục. Tra cứu nhanh 82
A.1 Tài khoản, địa chỉ, lệnh 82
A.2 Thuật ngữ 82
A.3 Báo cáo tình trạng tài liệu 83

Cách dùng tài liệu này

Tài liệu được viết từ ba nguồn có thể kiểm chứng: **bản đặc tả chính thức** (file *Đặc tả hệ thống Self-Storage Facility Rental and Management System*, 22 mục), **mã nguồn** trong repo (apps/api, apps/web, apps/gateway, packages/shared) và **bản triển khai thật** trên Cloudflare. Mọi tên bảng, endpoint, màn hình, tham số chính sách trong tài liệu đều lấy từ mã nguồn; chỗ nào chưa xác minh được đều ghi rõ.

Ký hiệu	Ý nghĩa
Đã kiểm chứng	Đã chạy thật (test tự động hoặc gọi API trên bản deploy) trong quá trình chuẩn bị tài liệu, ghi kèm ngày và kết quả.
Chưa xác minh	Có trong mã nguồn nhưng chưa chạy lại để xác nhận kết quả; khi bảo vệ chỉ nói "đã cài đặt", không nói "đã kiểm thử".
Giới hạn	Hạn chế có thật của hệ thống, cần chủ động nói trước nếu giảng viên hỏi tới.
Not Tested	Trạng thái mặc định của mọi test case trong Chương 4. Nhóm tự điền Actual Result/Status sau khi chạy.

Lộ trình đọc theo vai trò

Thành viên	Đọc kỹ	Đọc lướt (vẫn phải hiểu)
Khang	Ch.1, Ch.2 (mức tổng quan), Flow 1 và Flow 7 ở Ch.3, Ch.5 phần F1/F7, Ch.6 lời thoại của Khang, Ch.10	Các Flow còn lại ở mức "mục đích – ai làm – kết quả"; Ch.7 nhóm câu hỏi tổng quan
Triển	Flow 2, Flow 3, phần thanh toán/hóa đơn ở Ch.2, test case F2/F3, SQL F2/F3	Toàn bộ Ch.7, Ch.8
Phúc	Flow 4, Ch.4 toàn bộ (Phúc là người trả lời câu hỏi kiểm thử), Ch.8	Ch.2, các Flow còn lại
Trường	Flow 5, Flow 6, phần job nền/realtime ở Ch.2, test case F5/F6	Ch.7, Ch.8, Ch.10

> **Nguyên tắc vàng khi bảo vệ**
> Nói đúng những gì hệ thống đang làm. Khi chưa chắc, nói "em xin phép kiểm tra trên hệ thống/mã nguồn" và mở ra xem. Một câu trả lời trung thực kèm bằng chứng luôn tốt hơn một câu trả lời trôi chảy nhưng sai.

CHƯƠNG 1

Tổng quan dự án và phân công

> VAULTRA là gì, giải quyết bài toán nào, nhóm chia 7 Flow cho 4 người ra sao và vì sao chia như vậy.

1.1 VAULTRA trong một trang

VAULTRA là nền tảng cho thuê và vận hành **kho tự quản** (self-storage) nhiều cơ sở. Khách tìm kho đúng kích thước, xem giá và số kho trống, đặt và thanh toán online; cơ sở nhận khách, ký hợp đồng điện tử, bàn giao kho có ảnh và cấp mã PIN; doanh nghiệp quản lý chính sách, giá, công nợ, doanh thu trên cùng một dữ liệu.

Bài toán	Khách cần biết ngay kho nào còn trống và giá bao nhiêu; cơ sở cần biết hôm nay ai đến nhận/trả kho; doanh nghiệp cần chính sách thống nhất (cọc, hủy, gia hạn, quá hạn) và số liệu theo cơ sở; quản trị cần kiểm soát tài khoản và phân quyền.
5 vai trò	**CUSTOMER** (khách thuê) · **FACILITY_STAFF** (nhân viên cơ sở) · **FACILITY_MANAGER** (quản lý cơ sở) · **BUSINESS_OPS_MANAGER** (quản lý vận hành doanh nghiệp, viết tắt BOM) · **SYSTEM_ADMIN** (quản trị hệ thống). Định nghĩa ở *packages/shared/src/permissions.ts*.
7 Flow bắt buộc	F1 Đặt kho · F2 Check-in & bàn giao · F3 Quản lý kho đang thuê & trả kho · F4 Chính sách, phí, doanh thu · F5 Kho & nhân sự cơ sở · F6 Gia hạn & quá hạn · F7 Hỗ trợ & sự cố (đặc tả mục 6).
Mục tiêu cốt lõi	Không bao giờ giao một kho cho hai khách cùng thời gian (bảo đảm ở database); mọi đồng tiền truy được nguồn gốc (hóa đơn → thanh toán → phân bổ → hoàn tiền → sổ cái); phân quyền ở backend theo vai trò × phạm vi cơ sở.
Quy mô mã nguồn	Backend NestJS: 306 endpoint trong 13 controller · Database: 104 bảng, 129 enum (schema.prisma) · Frontend React: 6 khu vực giao diện (công khai, /app, /staff, /manage, /ops, /admin) · 96 mẫu thông báo · 16 job nền.
Bản chạy thật	Giao diện: **https://vaultra-844.pages.dev** · API: **https://vaultra-gateway.zerngulon.workers.dev** (Cloudflare Pages + Worker).

1.2 Bảy Flow nối với nhau như thế nào

Bảy quy trình chạy trên cùng một dữ liệu. F1 tạo đơn đặt thuê, F2 biến đơn thành hợp đồng đang hiệu lực, F3 vận hành hợp đồng đến khi trả kho, F6 gia hạn hoặc xử lý khi khách nợ. F4 cung cấp chính sách và giá cho tất cả; F5 cung cấp kho và nhân lực; F7 chạy song song mọi lúc.



Hình 1.1 — Vòng đời một kho và vị trí của 7 Flow (theo đặc tả mục 6)

1.3 Bảng phân công 7 Flow

Thành viên	Flow phụ trách	Độ khó	Thời lượng	Nội dung trình bày	Nội dung demo
Khang	Mở đầu + Tổng quan + **F1 Đặt kho** + **F7 Hỗ trợ**	Dễ – Trung bình	8 phút	Bài toán, 5 vai trò, kiến trúc mức tổng quan; F1: chọn kho, báo giá, giữ chỗ 15 phút, thanh toán VNPay; F7: tạo ticket, SLA, xử lý, đánh giá	Đặt một kho bằng tài khoản khách và thanh toán thẻ test; tạo ticket, nhân viên trả lời, khách đánh giá
Triển	**F2 Check-in & bàn giao** + **F3 Quản lý thuê & trả kho**	Khó	9 phút	Wizard 4 bước, điều kiện cứng trước khi cấp PIN, ký OTP + PDF có SHA-256; hóa đơn định kỳ, phụ phí, trả kho, công thức quyết toán cọc	Nhận khách vừa đặt ở F1; ký hợp đồng; cấp PIN; khách xem PIN, mở/đóng kho; gửi yêu cầu trả kho và xem ước tính
Phúc	**F4 Chính sách, phí, doanh thu** + chiến lược kiểm thử	Khó	7 phút	Chính sách có phiên bản (DRAFT → SCHEDULED → ACTIVE), xem trước tác động, khung giá, khuyến mãi, miễn giảm, đối soát, báo cáo; tổng kết kiểm thử	Tạo phiên bản chính sách, xem tác động, xuất bản; chạy đối soát; xem dashboard so sánh cơ sở
Trường	**F5 Kho & nhân sự** + **F6 Gia hạn & quá hạn** + kết luận	Khó	8 phút	Sơ đồ kho, máy trạng thái kho, thuật toán phân bổ, ca/công việc/bảo trì; gia hạn có OTP, chuỗi quá hạn GRACE → LATE → LOCKED_OUT → FINAL_NOTICE → DEFAULT	Đổi trạng thái kho trên sơ đồ, giao việc; gia hạn hợp đồng ở F2; mở màn hình công nợ quá hạn

Lý do phân chia

- **Khang** nhận các phần trực quan, kể được bằng ví dụ đời thường: "tôi cần chỗ để đồ, tôi đặt online" (F1) và "kho tôi bị dột, tôi báo hỗ trợ" (F7). Cả hai Flow demo hoàn toàn trên giao diện, ít thuật toán. Khang vẫn có phần có chiều sâu: giữ chỗ 15 phút, chống đặt trùng khi 100 người cùng đặt (có test tự động làm bằng chứng), SLA theo giờ làm việc. Khang mở đầu buổi bảo vệ vì Khang hiểu toàn bộ dự án nhất.
- **Triển** nhận F2 và F3 vì hai Flow này liền mạch (kho vừa nhận ở F2 là kho được quản lý ở F3) và nhiều quy tắc tiền: điều kiện cứng trước khi cấp mã, quyết toán cọc.
- **Phúc** nhận F4 vì F4 là "luật chơi" của cả hệ thống (chính sách có phiên bản, giá, phí) và gắn với kiểm thử: phần lớn unit test của dự án kiểm tra đúng các quy tắc F4 (tính giá, phí trễ, quyết toán). Phúc thêm phần tổng kết kiểm thử để cân thời lượng.
- **Trường** nhận F5 và F6: F5 là nền tảng kho/nhân sự, F6 là chuỗi quá hạn nhiều trạng thái nhất; cả hai dùng job nền và máy trạng thái nên cùng một mạch kiến thức. Trường kết luận buổi bảo vệ.

Đánh giá cân bằng

Tiêu chí	Khang	Triển	Phúc	Trường
Số Flow chính	2 (+ mở đầu)	2	1 (+ kiểm thử)	2 (+ kết luận)
Thời lượng nói	8 phút	9 phút	7 phút	8 phút
Độ khó nghiệp vụ	Dễ – TB	Khó	Khó	Khó
Khối lượng demo	Nhiều thao tác, đơn giản	Nhiều bước, cần chuẩn bị ảnh	Ít thao tác, nhiều giải thích	Trung bình
Số câu hỏi riêng phải luyện	12	15	15	15

> **Cả 4 người đều phải nắm tổng quan 7 Flow**
> Giảng viên có thể hỏi bất kỳ ai về bất kỳ Flow nào. Mục 9.5 có checklist kiến thức chung; mỗi người tự kiểm tra bằng cách nói lại được "mục đích – ai làm – trạng thái đầu/cuối" của cả 7 Flow trong 30 giây mỗi Flow.

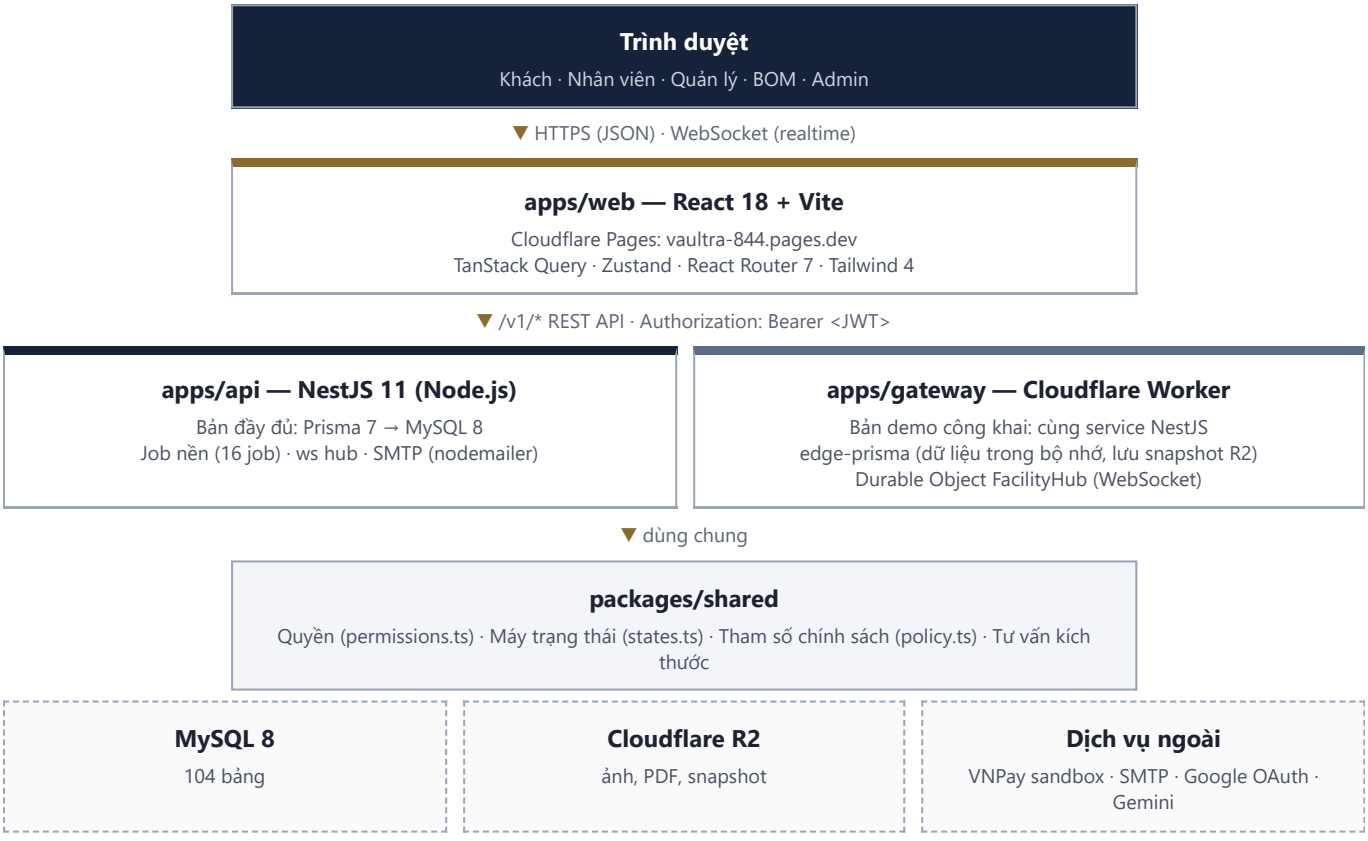
CHƯƠNG 2

Kiến trúc và mã nguồn

> Hệ thống được dựng từ những phần nào, mỗi phần nằm ở đâu trong repo và nói về nó thế nào khi bảo vệ. Chỉ liệt kê công nghệ có thật trong package.json và mã nguồn.

2.1 Kiến trúc tổng thể

Repo là một monorepo pnpm gồm 4 phần. Có **hai cách chạy** backend: bản Node.js đầy đủ với MySQL (dùng khi phát triển và chạy bộ test e2e) và bản Cloudflare Worker dùng cho trang demo công khai. Hai bản dùng **chung một mã nghiệp vụ** (các service NestJS trong apps/api/src); Worker chỉ thay lớp hạ tầng (database, lưu tệp, email) bằng "shim".



Hình 2.1 — Kiến trúc hệ thống VAULTRA

Điểm phải nói thật khi bảo vệ

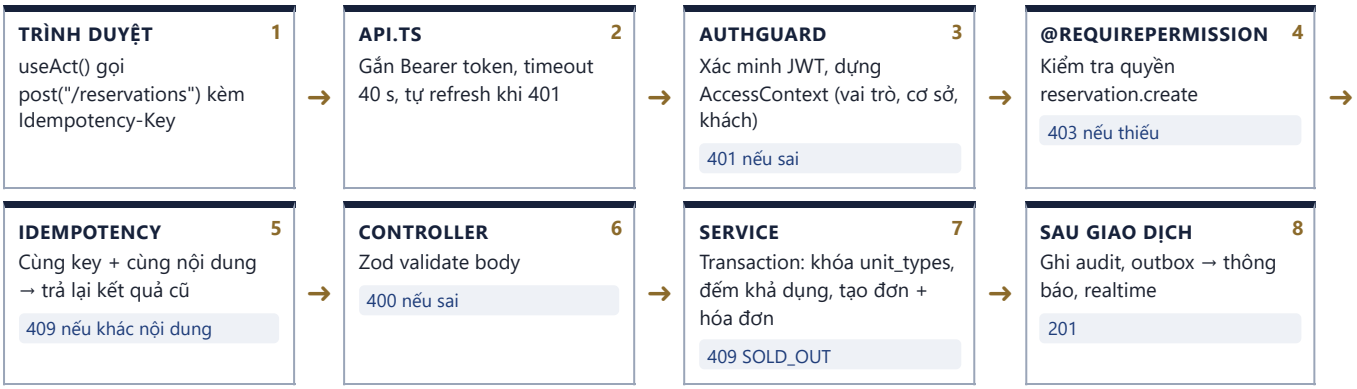
Bản trên Cloudflare Worker **không dùng MySQL**: dữ liệu nằm trong bộ nhớ của Worker và được lưu thành một file JSON trên R2 (*apps/gateway/src/edge-prisma.ts*, khóa *_db/vaultra-diff-v1.json*). Các bảo đảm khóa hàng *FOR UPDATE*, transaction và ràng buộc UNIQUE của MySQL được mô phỏng lại trong edge-prisma (UNIQUE, cột sinh, hoàn tác giao dịch), nhưng bằng chứng mạnh nhất (100 request đồng thời chỉ 1 thành công) đến từ bản Node + MySQL. Lý do: Cloudflare không có MySQL managed; MySQL đầy đủ chạy qua *infra/docker-compose.yml* hoặc *start.bat*. Xem rủi ro ARCH-01 ở Chương 8.

2.2 Bảng thành phần

Thành phần	Công nghệ	Vai trò	File / Class liên quan	Cách giải thích khi bảo vệ
Frontend	React 18, Vite 6, TypeScript, Tailwind 4	Giao diện 6 khu vực theo vai trò	apps/web/src/App.tsx, pages/{public,app,staff,manage,ops,admin}	"Mỗi vai trò có khu vực riêng; menu chỉ hiện chức năng mà vai trò đó có quyền, nhưng quyền thật được kiểm ở backend."
Gọi API & cache	TanStack Query 5	Tải dữ liệu, cache, tải lại sau thao tác	web/src/lib/api.ts, lib/hooks.ts (useGet, useAct)	"Sau mỗi thao tác, các danh sách liên quan tự tải lại; khi có người khác thay đổi, realtime báo để tải lại."
Trạng thái đăng nhập	Zustand	Phiên, vai trò, quyền, cơ sở đang chọn	web/src/lib/auth.ts	"Access token chỉ nằm trong bộ nhớ, không ghi ra ổ đĩa."
Backend	NestJS 11	Toàn bộ nghiệp vụ, 306 endpoint	apps/api/src/*/ *.controller.ts, *.service.ts	"Modular monolith: mỗi nghiệp vụ một module, chung một database để giao dịch tiền nhất quán."
Truy cập dữ liệu	Prisma 7 + MySQL 8	ORM, transaction, khóa hàng	api/prisma/schema.prisma, common/prisma.service.ts	"Schema 104 bảng; các thao tác tiền chạy trong transaction; khóa hàng bằng SELECT … FOR UPDATE."
Kiểm tra dữ liệu vào	Zod	Validate body/query của mọi API	parse(z.object(...)) trong từng controller	"Sai định dạng trả 400 VALIDATION_FAILED kèm danh sách trường lỗi."
Xác thực	jose (JWT EdDSA), Argon2	Đăng nhập, access/refresh token, Google OAuth	auth/auth.service.ts, jwt.service.ts, auth.guard.ts	"Mật khẩu băm Argon2; access token ngắn hạn; refresh token xoay vòng, dùng lại token cũ là thu hồi cả phiên."
Phân quyền	RBAC × phạm vi	Quyền theo vai trò và theo cơ sở	auth/access-context.ts, @RequirePermission, packages/shared/permissions.ts	"Mỗi request có AccessContext; truy vấn tự thêm điều kiện facility_id/customer_id; ngoài phạm vi trả 404."
Tiền tệ	decimal.js	Tính tiền không sai số	common/util.ts (D, vnd), domain/pricing.service.ts	"Không dùng số thực; làm tròn đến đồng ở từng dòng hóa đơn."
Thanh toán	VNPay sandbox (pay 2.1.0)	Thanh toán online, IPN có chữ ký	billing/payment.service.ts, vnpay.client.ts, vnpay.controller.ts	"Trang của VNPay nhập thẻ; tiền chỉ được ghi nhận khi IPN có chữ ký HMAC-SHA512 hợp lệ và đúng số tiền."
Tài liệu PDF	pdfkit + font DejaVu	Hợp đồng, biên bản, hóa đơn, biên nhận	media/pdf.service.ts, contract/contract-template.ts	"Mỗi PDF lưu mã băm SHA-256; sửa một byte là mã băm khác."
Lưu tệp	R2 / thư mục local	Ảnh hiện trạng, PDF, chữ ký	media/media.service.ts, storage.service.ts	"Tải lên bằng URL ký có hạn; kiểm tra magic bytes, kích thước, SHA-256 rồi mới chuyển READY."
Thông báo & email	nodemailer / SMTP Worker	96 mẫu, in-app + email, tệp .ics	notifications/notification.service.ts, templates.ts	"Thông báo ghi vào hàng đợi giao nhận; bộ gửi thử lại có giãn cách; mã OTP gửi ngay."
Realtime	ws / Durable Object	Đẩy thông báo và tín hiệu "dữ liệu đổi"	realtime/realtime.hub.ts, web/src/lib/realtime.ts, gateway FacilityHub	"Tab của người khác tự cập nhật mà không cần F5; vé kết nối có chữ ký, hạn 60 giây."
Sự kiện miền	Transactional outbox	Sự kiện không bị mất khi giao dịch lỗi	common/events.service.ts, bảng outbox_events	"Sự kiện ghi cùng giao dịch; giao dịch hủy thì sự kiện cũng mất, không có thông báo sai."
Job nền	JobsService (16 job)	Hết hạn giữ chỗ, hóa đơn kỳ, quá hạn, gia hạn, SLA, số liệu ngày	jobs/jobs.service.ts	"Chạy định kỳ: bản Node dùng timer, bản Worker dùng cron 5 phút (chạy job đến hạn theo job_runs)."
Nhật ký	audit_logs, login_history	Ai làm gì, lúc nào, lý do	audit/audit.service.ts	"Mọi thao tác nghiệp vụ và bảo mật đều ghi nhật ký; không xóa."

Thành phần	Công nghệ	Vai trò	File / Class liên quan	Cách giải thích khi bảo vệ
Trợ lý AI	Gemini + RAG	Chatbot trả lời theo kho tri thức	assistant/assistant.service.ts, kb.service.ts	"Tài liệu tri thức được chia đoạn; chatbot chỉ trả lời dựa trên các đoạn liên quan."
Triển khai	Cloudflare Pages, Worker, R2, DO	Bản demo công khai	apps/gateway/wrangler.toml, deploy-cloudflare.bat	"Frontend tĩnh trên Pages, API trên Worker, ảnh trên R2."

2.3 Luồng một request đi qua hệ thống



Hình 2.2 — Luồng xử lý POST /v1/reservations

Định dạng lỗi thống nhất ở mọi API: `{ "error": { "code", "message", "details"?, "requestId" } }` (common/errors.ts). Lỗi không lường trước chỉ ghi chi tiết ở server, trả cho người dùng "Lỗi hệ thống" kèm requestId để tra log.

2.4 Xác thực và phân quyền



Hình 2.3 — Luồng xác thực

Lớp kiểm tra	Ở đâu	Kết quả khi vi phạm
Chưa đăng nhập / token sai	auth.guard.ts	401 UNAUTHENTICATED
Thiếu quyền	@RequirePermission(...) trên endpoint	403 FORBIDDEN
Có quyền nhưng bản ghi ngoài phạm vi (khách khác, cơ sở khác)	ctx.assertScope(...) trong service, ctx.scope(...) trong truy vấn	404 NOT_FOUND (không lộ bản ghi tồn tại)
Đúng quyền nhưng sai trạng thái	Máy trạng thái packages/shared/states.ts	409 INVALID_STATUS_TRANSITION
Sửa chồng dữ liệu	Cột version (khóa lạc quan)	409 STALE_RESOURCE

Google OAuth dùng luồng Authorization Code + PKCE; tham số state được ký HMAC và lưu trong cookie 10 phút (auth.service.ts: googleStart, googleCallback). Email đã có tài khoản thường sẽ phải xác nhận liên kết bằng mật khẩu hoặc mã gửi email.

2.5 Database

Schema nằm ở *apps/api/prisma/schema.prisma* (104 model ↔ 104 bảng MySQL, tên bảng dạng snake_case qua @@map). Migration SQL ở *apps/api/prisma/migrations/*. Các quy ước quan trọng:

- Mỗi bảng có **id** BIGINT tăng dần (nội bộ) và **public_id** ULID 26 ký tự (dùng ra ngoài API) để chống đoán ID. API trả public_id dưới tên trường *id*.
- Tiền: DECIMAL(15,2); hóa đơn có ràng buộc CHECK *total = subtotal − discount_total + tax_total*.
- **Chống giao một kho cho hai đơn:** bảng unit_allocations có cột sinh *open_unit_key = IF(status = 'ACTIVE', unit_id, NULL)* và UNIQUE(open_unit_key). Tương tự access_credentials có *active_pin_key* để PIN không trùng trong cùng cơ sở.
- Không xóa dữ liệu tiền, hợp đồng, biên bản, nhật ký; danh mục dùng xóa mềm (deleted_at).
- Bảng lịch sử trạng thái: reservation_status_history, contract_status_history, unit_status_history, ticket_events, task_events.

Nhóm bảng	Bảng tiêu biểu
Tài khoản & phân quyền	users, customers, roles, permissions, role_permissions, user_roles, user_sessions, refresh_tokens, auth_tokens, login_history
Cơ sở & kho	facilities, zones, unit_types, units, unit_status_history, floor_plans, unit_shapes, price_history
Đặt thuê & hợp đồng	quotes, reservations, unit_allocations, appointment_slots, appointments, contracts, contract_terms, contract_documents, contract_signatures, checkin_sessions, identity_verifications, inspections, inspection_items, handover_records, access_credentials, access_events
Trả kho, gia hạn, quá hạn	move_out_requests, damage_items, deposit_settlements, renewal_offers, delinquency_events, collection_notes, promise_to_pay, overlocks, termination_requests, abandoned_goods_records
Tiền	policies, policy_versions, price_bands, price_change_requests, fee_types, promotions, promotion_redemptions, billing_schedules, invoices, invoice_items, payments, payment_allocations, payment_webhook_events, refunds, deposits, credit_notes, fee_waivers, customer_ledger_entries, reconciliation_batches, reconciliation_items, idempotency_keys
Vận hành & hỗ trợ	shift_templates, staff_shifts, shift_swap_requests, work_tasks, task_events, work_orders, maintenance_plans, support_tickets, ticket_messages, ticket_events, sla_policies, ticket_ratings
Hệ thống	notification_templates, notifications, notification_deliveries, audit_logs, outbox_events, job_runs, report_exports, daily_facility_metrics, system_settings, kb_articles, kb_chunks

2.6 Thanh toán, lưu tệp, email, realtime

Thanh toán (VNPay)	payment.service.ts: startGateway tạo payment PENDING và URL ký HMAC-SHA512 tới VNPay. VNPay gọi lại **IPN** (GET /payments/vnpay/ipn, nguồn sự thật) và chuyển trình duyệt về **return**. Cả hai đi qua settle(): khóa hàng payment, bỏ qua nếu đã xử lý, so số tiền (thiếu tiền → MISMATCH, không ghi nhận), rồi phân bổ vào hóa đơn theo hạn cũ nhất trước. Tiền mặt/chuyển khoản tại quầy: POST /payments/offline.
Lưu tệp	POST /media/upload-url → PUT tệp lên URL ký → POST /media/:id/complete (kiểm tra loại tệp bằng magic bytes, kích thước theo mục đích, SHA-256) → trạng thái READY. Chỉ tệp READY mới được gắn vào biên bản/ticket.
Email	Hai đường: thông báo sự kiện ghi notification_deliveries (QUEUED) và bộ gửi gửi sau; mã OTP/link ký gửi ngay (sendDirect). Lịch hẹn kèm tệp .ics để thêm vào lịch.

Realtime	POST /auth/realtime-ticket cấp vé 60 giây; trình duyệt mở WebSocket /v1/realtime. Sau mỗi thao tác ghi, server đẩy {type:"data.changed"} và các sự kiện riêng (thông báo theo người dùng, task theo cơ sở). Trình duyệt tải lại các dữ liệu đang hiển thị.
Job nền	16 job trong jobs.service.ts: reservation.expire-holds (mỗi phút), billing.recurring, delinquency.run, renewal.windows, renewal.expiries, moveout.auto-finalize, support.sla, policy.activate-scheduled, report.daily-metrics… Bản Node chạy theo timer; bản Worker chạy từ cron mỗi 5 phút, mỗi lượt chỉ chạy job đã đến hạn theo bảng job_runs (job lỗi được chạy lại lượt sau). Chạy tay qua POST /internal/jobs/:name (cần khóa nội bộ).

2.7 Xử lý lỗi, nhật ký, triển khai

- **Lỗi nghiệp vụ** ném AppError(code, message, details) → bộ lọc chung đổi thành JSON thống nhất. Danh sách mã: đặc tả mục 7.5 (VALIDATION_FAILED, UNIT_TYPE_SOLD_OUT, CHECKIN_PRECONDITION_FAILED, RENEWAL_BLOCKED_OUTSTANDING_BALANCE, STALE_RESOURCE, IDEMPOTENCY_CONFLICT…).
- **Frontend**: lỗi API hiện toast; màn hình tải dữ liệu lỗi hiện "Không tải được dữ liệu" + nút Thử lại; lỗi render được ErrorBoundary chặn để không trắng cả trang; request quá 40 giây báo "Máy chủ phản hồi quá chậm".
- **Nhật ký**: audit_logs (nghiệp vụ, bảo mật), login_history, ticket_events, các bảng *_status_history; log server có requestId.
- **Triển khai demo**: Worker bằng *wrangler deploy* (apps/gateway/wrangler.toml), giao diện bằng *wrangler pages deploy apps/web/dist*; bí mật (khóa JWT, khóa mã hóa trường, SMTP, VNPay) đặt bằng *wrangler secret put*. Bản đầy đủ: *start.bat* hoặc *infra/docker-compose.yml* với MySQL.

CHƯƠNG 3

Phân tích 7 Flow nghiệp vụ

> Mỗi Flow trình bày theo 20 mục yêu cầu: mục đích, actor, điều kiện, luồng chính/thay thế/ngoại lệ, trạng thái, bảng, API, màn hình, nơi xử lý logic, quy tắc, lỗi, cách kiểm thử, cách demo và câu hỏi. Nội dung đối chiếu giữa đặc tả mục 6 và mã nguồn.

Cách đọc trạng thái triển khai

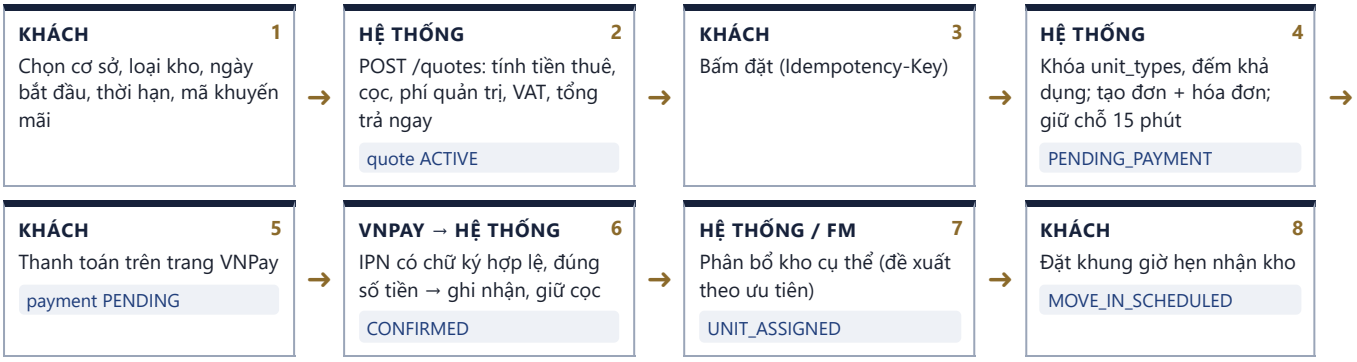
Dòng "Trạng thái triển khai" ở cuối mỗi bảng cho biết cái gì đã chạy thử và ra kết quả, cái gì mới chỉ có trong mã nguồn. Đừng nói "đã kiểm thử" cho phần ghi "Chưa xác minh".

3.1 Flow 1 — Đặt kho (Storage Unit Reservation)

Người trình bày chính: **Khang** · Độ khó: Trung bình · Màn hình chính: /units, /book/:unitTypeId, /app/reservations

1. Mục đích nghiệp vụ	Đưa khách từ lúc tìm kho đến khi có đơn đặt thuê đã thanh toán, đã có kho cụ thể và lịch hẹn nhận kho. Kết thúc khi đơn ở MOVE_IN_SCHEDULED (chuyển sang F2) hoặc bị hủy/hết hạn.
2. Vấn đề thực tế	Khách không muốn đến quầy chỉ để hỏi giá và còn kho không. Doanh nghiệp không được để hai khách cùng giữ một kho, và không được để khách "giữ chỗ ảo" mà không trả tiền.
3. Actor	Khách vãng lai (xem, báo giá) · CUSTOMER (đặt, trả tiền, đặt lịch, hủy) · FACILITY_STAFF (đặt hộ tại quầy) · FACILITY_MANAGER (phân bổ kho, hủy ngoại lệ) · Hệ thống (giữ chỗ, hết hạn, phân bổ, thông báo).
4–5. Điều kiện bắt đầu / tiên quyết	Cơ sở đang nhận đặt (accepts_bookings) · loại kho is_bookable · khách đăng nhập, email đã xác minh · ngày bắt đầu trong [hôm nay, hôm nay + 60 ngày] · thời hạn thuộc {1, 3, 6, 12} tháng · khách có không quá 3 đơn đang chờ thanh toán.

Sơ đồ quy trình (luồng chính)



Hình 3.1 — Luồng chính F1

Bước 4 là điểm quan trọng nhất: kiểm tra còn kho và tạo đơn nằm trong **một giao dịch**, có khóa hàng trên loại kho, nên hai khách bấm cùng lúc vào kho cuối cùng thì chỉ một người thành công, người còn lại nhận UNIT_TYPE_SOLD_OUT. Tiền chỉ được ghi nhận ở bước 6 (IPN từ VNPay), không phải ở trang khách quay về.



Nhánh khác: PENDING_PAYMENT → EXPIRED (hết 15 phút) · mọi trạng thái trước CHECKED_IN → CANCELLED · CONFIRMED/UNIT_ASSIGNED/MOVE_IN_SCHEDULED → NO_SHOW

Máy trạng thái reservations.status (packages/shared/src/states.ts)

8. Luồng thay thế	• Nhân viên đặt hộ tại quầy (POST /customers tạo hồ sơ khách, rồi đặt) và ghi nhận tiền mặt/chuyển khoản (POST /payments/offline). • Khách đặt lịch hẹn ngay sau khi thanh toán, trước khi có kho cụ thể: đơn vẫn CONFIRMED, chuyển MOVE_IN_SCHEDULED khi phân bổ xong. • Thanh toán thất bại: khách bấm thanh toán lại (POST /payments/:id/retry hoặc /reservations/:id/pay) đến khi hết giờ giữ chỗ. • Khách hủy: GET /reservations/:id/cancellation-preview cho xem trước số tiền hoàn theo bậc hủy, rồi POST /reservations/:id/cancel.
9. Ngoại lệ	• Hết kho (UNIT_TYPE_SOLD_OUT) kèm gợi ý loại kho/cơ sở khác. • Quá 3 đơn chờ thanh toán: từ chối tạo đơn mới. • IPN sai chữ ký → INVALID_SIGNATURE, không ghi nhận gì; IPN thiếu tiền → không xác nhận đơn, payment bị đánh MISMATCH. • Quote quá hạn 15 phút → phải báo giá lại. • Khung giờ hẹn đã đầy (capacity) hoặc đã qua trong ngày → không chọn được.
10. Điều kiện hoàn thành	Đơn MOVE_IN_SCHEDULED, kho RESERVED, cọc HELD, hóa đơn RESERVATION PAID, có lịch hẹn BOOKED, hợp đồng nháp PENDING_SIGNATURE.
11. Dữ liệu trước → sau	quote: — → ACTIVE → USED · reservation: — → PENDING_PAYMENT → CONFIRMED → UNIT_ASSIGNED → MOVE_IN_SCHEDULED · invoice: — → ISSUED → PAID · payment: — → PENDING → SUCCEEDED · deposit: — → HELD · unit: AVAILABLE → RESERVED · unit_allocation: — → ACTIVE · appointment: — → BOOKED.
12. Bảng database	quotes, reservations, reservation_status_history, invoices, invoice_items, payments, payment_allocations, payment_webhook_events, deposits, unit_allocations, appointment_slots, appointments, contracts (nháp), promotion_redemptions, idempotency_keys, customer_ledger_entries, notifications, audit_logs.
13. API / backend	GET /catalog/facilities · GET /catalog/unit-types/:id/availability · POST /quotes · POST /promotions/validate · POST /reservations · POST /reservations/:id/pay · GET /payments/vnpay/ipn · GET /payments/vnpay/return · GET /facilities/:id/appointment-slots · POST /appointments · GET /reservations/:id/cancellation-preview · POST /reservations/:id/cancel · GET /reservations/:id/allocation-suggestions · POST /reservations/:id/allocate.
14. Màn hình frontend	Công khai: /locations, /units, /size-guide (tư vấn kích thước 3D), /book/:unitTypeId (wizard 6 bước), /payment/result. Khách: /app/reservations, /app/reservations/:id. Quản lý: /manage/reservations (phân bổ).
15. Business logic nằm ở	reservation/reservation.service.ts (tạo đơn, khả dụng, hủy, hết hạn giữ chỗ, đặt lịch) · domain/pricing.service.ts (báo giá) · billing/payment.service.ts (startGateway, vnpayCallback, settle) · catalog/catalog.controller.ts (catalog, quotes).

16. Quy tắc nghiệp vụ quan trọng	1. Giữ chỗ hold_minutes = 15; job reservation.expire-holds chuyển đơn quá hạn sang EXPIRED và hóa đơn sang VOID. 2. Khả dụng = số kho bán được − số đơn/hợp đồng còn hiệu lực cùng loại có khoảng ngày giao nhau (tính cả đơn PENDING_PAYMENT chưa hết giờ giữ) — đặc tả 7.3. 3. Số tiền trả khi đặt theo reservation_payment_mode = DEPOSIT_AND_FIRST_PERIOD: cọc (1 tháng tiền thuê) + kỳ đầu + phí quản trị. 4. Giá khóa theo quote; khuyến mãi kiểm tra hiệu lực, phạm vi, số lượt, giới hạn mỗi khách. 5. Phân bổ chỉ chọn kho cùng loại, cùng cơ sở, AVAILABLE, không có allocation mở; UNIQUE(open_unit_key) chặn ở database. 6. Bậc hủy: ≥ 7 ngày hoàn 100%; 2–6 ngày hoàn 50% cọc và 100% tiền thuê; < 48 giờ mất cọc, hoàn 100% tiền thuê; phí quản trị không hoàn.
17. Lỗi có thể xảy ra	• Khách bấm "Đặt" hai lần do mạng chậm → nếu không có Idempotency-Key sẽ tạo hai đơn (đã chặn bằng interceptor). • Trang return của VNPay bị khách sửa URL → không ảnh hưởng vì chữ ký được kiểm tra lại. • Trên bản Worker, job hết hạn giữ chỗ chạy theo cron 5 phút: đơn quá 15 phút có thể còn hiển thị PENDING_PAYMENT thêm vài phút (không còn giữ kho vì khả dụng dựa vào hold_expires_at).
18. Cách kiểm thử	Test tự động: apps/api/test/e2e/f1-reservation.ts (11 kiểm tra), nfr03-concurrency.ts (100 request đồng thời), vnpay.ts (13 kiểm tra chữ ký/số tiền). Test case thủ công: TC-F1-01 → TC-F1-12 ở mục 4.2.
19. Cách demo	Khang, tài khoản customer@vaultra.test, mục 5.2 bước F1.
Trạng thái triển khai	**Đã kiểm chứng** 08/11/2026 trên Worker cục bộ: nfr03 5/5 đạt (100 request → 1 thành công, 99 UNIT_TYPE_SOLD_OUT); f1-reservation 10/12 (2 kiểm tra idempotency lỗi do dữ liệu cục bộ hết chỗ, chưa xác minh lại). **Chưa xác minh** lại trên MySQL sau các thay đổi gần nhất.

Ví dụ thực tế và kịch bản thao tác

Chị Lan cần chỗ để đồ khi chuyển nhà. Chị chọn cơ sở Vaultra Cầu Giấy, loại "Kho nhỏ 2.5m²", bắt đầu ngày mai, thuê 3 tháng. Hệ thống báo: tiền thuê kỳ đầu, cọc 1 tháng, phí quản trị 100.000 ₫, tổng trả ngay. Chị bấm đặt, có 15 phút để thanh toán. Thanh toán bằng thẻ NCB test xong, đơn chuyển "Đã xác nhận", hệ thống gán kho A-012 và chị chọn khung 09:00–12:00 để đến nhận.

#	Thao tác	Kết quả mong đợi
1	Mở /units, chọn cơ sở và loại kho, bấm "Đặt kho"	Hiện dải "Lịch trống 8 tuần" và bảng giá từng dòng
2	Chọn ngày bắt đầu, thời hạn 3 tháng, (tùy chọn) nhập mã khuyến mãi	Tổng tiền cập nhật, có dòng giảm giá nếu mã hợp lệ
3	Chọn ngày và khung giờ hẹn, đồng ý điều khoản, bấm "Thanh toán trực tuyến & giữ chỗ"	Chuyển sang trang VNPay sandbox
4	Nhập thẻ NCB 9704198526191432198, NGUYEN VAN A, 07/15, OTP 123456	Quay về /payment/result, đơn "Đã xác nhận"
5	Mở /app/reservations/:id	Thấy mã đơn RSV-..., hóa đơn PAID, mã kho (nếu đã phân bổ), lịch hẹn

20. Câu hỏi giảng viên có thể đặt ra

- Hai người cùng đặt kho cuối cùng thì sao? (Khóa hàng + UNIQUE, bằng chứng nfr03.)
- Vì sao không ghi nhận tiền ở trang return? (Return do trình duyệt gửi, có thể giả; IPN là server-to-server có chữ ký.)
- Khách đặt rồi không trả tiền thì kho bị giữ bao lâu? (15 phút, hold_minutes.)
- Vì sao khách đặt theo loại kho chứ không chọn đúng kho? (Đề bài: kho cụ thể phân bổ sau; giữ linh hoạt vận hành.)

- Bấm đặt hai lần có ra hai đơn không? (Idempotency-Key.)

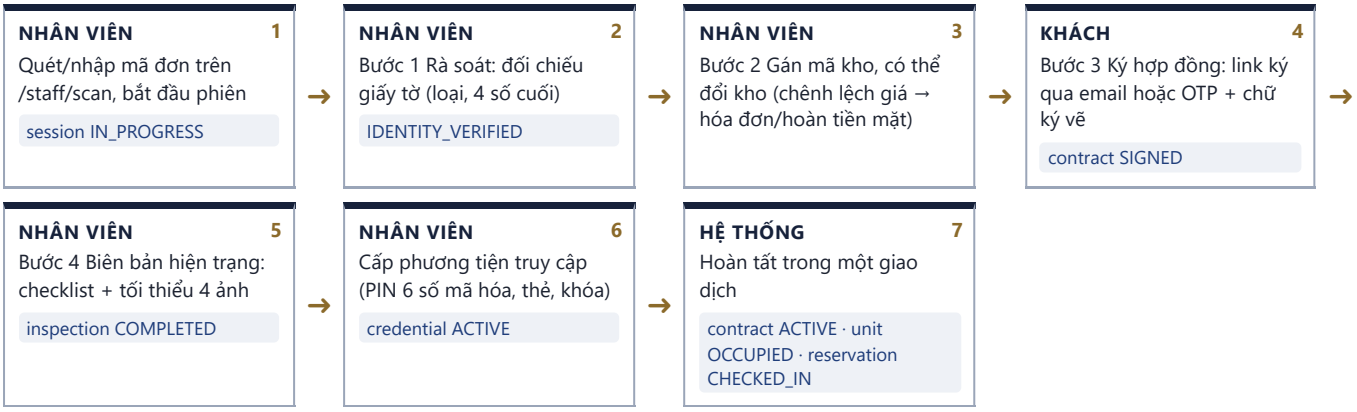
> **Điểm cần lưu ý khi bảo vệ**
> Nhấn mạnh hai bảo đảm: **không bán trùng** (giao dịch + khóa + UNIQUE, có test 100 request) và **tiền chỉ ghi nhận khi có chữ ký hợp lệ**. Hết hạn giữ chỗ do job reservation.expire-holds xử lý (bản demo chạy qua cron 5 phút, nên đơn có thể chuyển EXPIRED chậm tối đa khoảng 5 phút).

3.2 Flow 2 — Check-in và bàn giao (Check-in & Handover)

Người trình bày chính: **Triển** · Độ khó: Khó · Màn hình chính: /staff/scan, /staff/checkin/:sid

1. Mục đích nghiệp vụ	Biến đơn đặt thuê thành hợp đồng đang hiệu lực: xác minh khách, thu đủ tiền đến hạn, ký hợp đồng, kiểm tra hiện trạng có ảnh, cấp phương tiện truy cập, ký biên bản. Kết thúc khi kho OCCUPIED và hợp đồng ACTIVE.
2. Vấn đề thực tế	Bàn giao giấy tờ dễ thất lạc, không có ảnh hiện trạng thì khi trả kho không chứng minh được hư hại; cấp mã trước khi ký hợp đồng hoặc chưa thu tiền là rủi ro pháp lý và tài chính.
3. Actor	FACILITY_STAFF (thực hiện wizard) · CUSTOMER (ký hợp đồng, nhận PIN) · FACILITY_MANAGER (đổi kho, xử lý ngoại lệ) · Hệ thống.
4–5. Điều kiện bắt đầu / tiên quyết	Đơn MOVE_IN_SCHEDULED hoặc UNIT_ASSIGNED; kho RESERVED gắn allocation của đơn; ngày hiện tại trong cửa sổ nhận kho (start_date đến start_date + 7 ngày); nhân viên thuộc cơ sở của đơn.

Sơ đồ quy trình (luồng chính)



Hình 3.2 — Luồng chính F2

Backend có "cổng cứng": không cấp mã truy cập khi hợp đồng chưa SIGNED hoặc còn hóa đơn nhận kho chưa trả; API trả 409 CHECKIN_PRECONDITION_FAILED kèm danh sách điều kiện còn thiếu (ví dụ ["CONTRACT_NOT_SIGNED"]). Giao diện chỉ hiện nút, quyết định nằm ở backend.



Nhánh: ACTIVE → EXPIRED / TERMINATED; PENDING_SIGNATURE/SIGNED → CANCELLED; MOVE_OUT_SCHEDULED → ACTIVE khi khách hủy trả kho

Máy trạng thái contracts.status

8. Luồng thay thế	• Khách đã ký online trước khi đến: bước 3 tự hoàn tất. • Kho thực tế không đạt (POST /checkin-sessions/:id/unit-unfit): kho → MAINTENANCE, đổi sang kho cùng loại (swap-unit). • Đổi sang kho rẻ hơn: hệ thống tạo yêu cầu hoàn tiền mặt phần chênh; đắt hơn: hóa đơn chênh lệch phải trả trước khi tiếp tục. • Nhân viên ghi nhận tiền mặt ngay trong phiên (POST /checkin-sessions/:id/record-payment).
9. Ngoại lệ	• Hủy phiên (abort) với lý do: giấy tờ không khớp, chưa thanh toán, khách từ chối hợp đồng, kho không đạt, khách rời đi → mọi thay đổi chưa hoàn tất không được áp dụng. • Ảnh chưa tải xong (media chưa READY) → lưu biên bản báo "Tệp chưa tải lên xong". • Khách đến ngoài cửa sổ nhận kho → không bắt đầu được phiên.
10. Điều kiện hoàn thành	Hợp đồng ACTIVE, kho OCCUPIED, biên bản bàn giao có đủ ảnh, ít nhất một credential ACTIVE, lịch hóa đơn kỳ sau (billing_schedules) đã tạo.
11. Dữ liệu trước → sau	checkin_session: — → IN_PROGRESS → COMPLETED · contract: PENDING_SIGNATURE → SIGNED → ACTIVE · reservation: MOVE_IN_SCHEDULED → CHECKED_IN · unit: RESERVED → OCCUPIED · appointment: BOOKED → COMPLETED · access_credential: — → ACTIVE · handover_record, inspection: tạo mới.
12. Bảng database	checkin_sessions, identity_verifications, contracts, contract_documents, contract_signatures, inspections, inspection_items, media_files, access_credentials, handover_records, unit_status_history, unit_allocations, billing_schedules, work_tasks, audit_logs.
13. API / backend	GET /ops/lookup · POST /checkin-sessions · GET /checkin-sessions/:id · POST …/verify-identity · …/record-payment · …/assign-unit · …/swap-unit · …/send-contract-email · …/sign-otp · …/sign · …/inspection · …/issue-access · …/complete · …/abort · …/unit-unfit · POST /contracts/:id/sign-by-email (khách ký qua link) · GET /contracts/:id/documents.
14. Màn hình frontend	/staff/scan, /staff/checkin/:sid (wizard 4 bước: Rà soát thông tin · Gán mã kho & Đổi kho · Ký hợp đồng (Email) · Bàn giao & Cấp mã PIN), /sign-contract (trang ký của khách), /app/contracts/:id.
15. Business logic nằm ở	checkin/checkin.service.ts (phiên, điều kiện, hoàn tất) · contract/contract.service.ts + contract-template.ts (sinh hợp đồng, PDF, ký) · access/access.service.ts (PIN AES-256-GCM, hash tra cứu) · media/media.service.ts (ảnh).
16. Quy tắc nghiệp vụ quan trọng	1. Không cấp credential khi hợp đồng chưa SIGNED hoặc còn hóa đơn nhận kho chưa PAID (409 CHECKIN_PRECONDITION_FAILED). 2. Tối thiểu 4 ảnh hiện trạng (min_handover_photos); ảnh lưu SHA-256, người chụp, thời điểm. 3. PIN 6 số, mã hóa AES-256-GCM (pin_enc), không gửi qua email, chỉ hiển thị trong ứng dụng; duy nhất trong các PIN đang hoạt động của cơ sở (active_pin_key UNIQUE). 4. PDF hợp đồng sinh từ mẫu + snapshot giá/chính sách, lưu SHA-256 trong contract_documents. 5. Hoàn tất chạy trong một giao dịch: thất bại giữa chừng thì không có trạng thái nửa vời.
17. Lỗi có thể xảy ra	• Email ký hợp đồng không đến (SMTP lỗi): xem notification_deliveries.status/last_error; nhân viên dùng phương án OTP tại quầy. • Khách đã ký nhưng màn hình nhân viên chưa cập nhật: realtime đẩy "dữ liệu đổi"; nếu mất kết nối WebSocket thì bấm làm mới.
18. Cách kiểm thử	Test tự động: apps/api/test/e2e/f2-checkin.ts (24 kiểm tra). Test case thủ công TC-F2-01 → TC-F2-11.
19. Cách demo	Triển, tài khoản staff.hn@vaultra.test (nhân viên Cầu Giấy) + khách từ F1, mục 5.2 bước F2.
Trạng thái triển khai	**Đã kiểm chứng một phần** 08/11/2026 trên Worker cục bộ: f2-checkin 17/24 đạt; 7 kiểm tra lỗi vì test đọc OTP từ hộp thư giả của bản Node trong khi Worker gửi email thật, các bước sau là hệ quả. Luồng ký qua email đã được người dùng xác nhận nhận được thư trên bản deploy.

Ví dụ thực tế và kịch bản thao tác

Chị Lan đến cơ sở. Nhân viên nhập mã RSV-2026-000105, kiểm tra CCCD (4 số cuối khớp), gán kho A-003. Hệ thống gửi link ký hợp đồng vào email chị; chị ký trên điện thoại. Nhân viên chụp 4 ảnh cửa, sàn, tường, trần, rồi cấp PIN. Chị mở ứng dụng, tab Truy cập hiện mã PIN 6 số.

#	Thao tác	Kết quả mong đợi
1	Nhân viên mở /staff/scan, nhập mã đơn, bấm Bắt đầu check-in	Mở wizard bước 1
2	Bước 1: chọn loại giấy tờ, nhập 4 số cuối, xác nhận	Bước 1 hoàn tất
3	Bước 2: chọn mã kho (giữ nguyên hoặc đổi)	Kho được gán cho phiên
4	Bước 3: gửi link ký; khách mở email, ký	Bước 3 chuyển "Đã ký"; xem được PDF hợp đồng
5	Bước 4: điền checklist, tải ≥ 4 ảnh, lưu biên bản; cấp PIN; Hoàn tất	Hợp đồng ACTIVE; khách thấy PIN ở /app/contracts/:id → Truy cập

20. Câu hỏi giảng viên có thể đặt ra

- Nếu nhân viên cố cấp PIN trước khi khách ký thì sao? (409 CHECKIN_PRECONDITION_FAILED.)
- Làm sao chứng minh hợp đồng không bị sửa sau khi ký? (SHA-256 của PDF lưu cùng chữ ký.)
- PIN lưu thế nào? Admin xem được không? (Mã hóa AES-256-GCM; chỉ khách và nhân viên cơ sở có quyền mới giải mã, mỗi lần xem ghi audit.)
- Mất mạng giữa chừng thì sao? (Phiên giữ trạng thái từng bước; hoàn tất là một giao dịch.)

> **Điểm cần lưu ý khi bảo vệ**
> Nhấn mạnh "cổng cứng" ở backend và tính toàn vẹn của hợp đồng (PDF + SHA-256 + chữ ký + IP). Chuẩn bị sẵn 4 ảnh trên máy demo; nếu email chậm, dùng phương án ký OTP.

3.3 Flow 3 — Quản lý kho đang thuê và trả kho (Rented Storage Unit Management)

Người trình bày chính: **Triển** · Độ khó: Khó · Màn hình chính: /app/contracts/:id, /staff/moveouts, /manage/moveouts

1. Mục đích nghiệp vụ	Quản lý mỗi kho từ khi OCCUPIED đến khi trả kho và quyết toán xong: hóa đơn định kỳ, thanh toán, phụ phí, thay đổi truy cập, yêu cầu trả kho, kiểm tra khi trả, quyết toán cọc.
2. Vấn đề thực tế	Khách thuê nhiều kho cần một nơi xem hết; cơ sở cần thu đúng hạn, ghi phụ phí có căn cứ; khi trả kho phải tính tiền cọc minh bạch, có ảnh trước/sau.
3. Actor	CUSTOMER · FACILITY_STAFF (phụ phí theo danh mục, kiểm tra, thu hồi credential) · FACILITY_MANAGER (duyệt hư hại, duyệt hoàn cọc trong hạn mức) · Hệ thống (job hóa đơn kỳ, nhắc hạn, tự chốt quyết toán).
4–5. Điều kiện bắt đầu / tiên quyết	Hợp đồng ACTIVE, kho OCCUPIED. Yêu cầu trả kho: báo trước tối thiểu 7 ngày (move_out_notice_days) với khách.

Sơ đồ quy trình (luồng chính)



Hình 3.3 — Luồng chính F3

Công thức quyết toán (moveout.service.ts, hàm estimate): **net = cọc đang giữ − hóa đơn còn nợ − phí hư hại − phí vệ sinh − phí chấm dứt sớm + hoàn tiền trả trước**. Dương → tạo hoàn tiền chờ duyệt; âm → hóa đơn SETTLEMENT cho phần thiếu. Khách có 3 ngày (dispute_days) để khiếu nại trước khi chốt.

`SCHEDULED` → `INSPECTED` → `COMPLETED`

move_out_requests.status; có thể CANCELLED trước ngày trả. Quyết toán: DRAFT → (DISPUTED) → FINALIZED

Trạng thái yêu cầu trả kho

8. Luồng thay thế	• Nhân viên tạo yêu cầu trả kho hộ (không bị ràng buộc báo trước 7 ngày). • Khách hủy yêu cầu trước ngày trả: hợp đồng về ACTIVE, lịch hẹn và việc kiểm tra bị hủy. • Khách không đồng ý biên bản: quyết toán DISPUTED, khiếu nại trong 3 ngày. • Phụ phí phát sinh (POST /contracts/:id/charges) chỉ chọn từ danh mục fee_types do BOM quản lý.
9. Ngoại lệ	• Phí hư hại vượt ngưỡng damage_approval_threshold (1.000.000 ₫) cần FM duyệt (POST /damage-items/:id/decide). • Kho chỉ về AVAILABLE khi không còn credential ACTIVE, đã có biên bản trả kho và nhân viên xác nhận vệ sinh (final-check). • Hủy yêu cầu khi đã đến ngày trả → bị từ chối.
10. Điều kiện hoàn thành	Hóa đơn kỳ được thanh toán; khi trả kho: hợp đồng ENDED → CLOSED (khi số dư = 0 và cọc đã quyết toán), kho AVAILABLE hoặc MAINTENANCE.
11. Dữ liệu trước → sau	move_out_request: — → SCHEDULED → INSPECTED → COMPLETED · contract: ACTIVE → MOVE_OUT_SCHEDULED → ENDED → CLOSED · access_credential: ACTIVE → REVOKED · unit: OCCUPIED → PENDING_INSPECTION → AVAILABLE · deposit: HELD → APPLIED/REFUNDED · deposit_settlement: — → DRAFT → FINALIZED · refund: — → PENDING_APPROVAL → …
12. Bảng database	billing_schedules, invoices, invoice_items, payments, payment_allocations, customer_ledger_entries, access_credentials, access_events, inspections, damage_items, move_out_requests, handover_records, deposit_settlements, refunds, unit_status_history.
13. API / backend	GET /contracts, /contracts/:id · GET /invoices · POST /invoices/pay · POST /contracts/:id/charges · GET /contracts/:id/access · POST /contracts/:id/access/pin/rotate · …/remote-unlock · …/remote-lock · POST /access-credentials/:id/report-lost · GET /contracts/:id/move-out-estimate · POST /contracts/:id/move-out · POST /move-out-requests/:id/cancel · POST /move-out-requests/:id/inspection · POST /move-out-requests/:id/complete · GET /contracts/:id/settlement · POST /contracts/:id/settlement/dispute · POST /settlements/:id/finalize · POST /units/:id/final-check.

14. Màn hình frontend	Khách: /app/contracts/:id (tab Tổng quan, Truy cập, Hóa đơn, Tài liệu, Gia hạn, Trả kho), /app/billing. Nhân viên: /staff/moveouts, /staff/moveout/:id. Quản lý: /manage/moveouts, /manage/billing, /manage/approvals.
15. Business logic nằm ở	moveout/moveout.service.ts (estimate, request, inspect, complete, finalize, finalUnitCheck, autoFinalizeJob) · billing/billing.service.ts (hóa đơn kỳ, phụ phí) · billing/payment.service.ts (phân bổ, hoàn tiền) · access/access.service.ts.
16. Quy tắc nghiệp vụ quan trọng	1. Hóa đơn kỳ sinh trước invoice_lead_days = 7 ngày; due_date = ngày bắt đầu kỳ. 2. Một payment có thể trả nhiều hóa đơn: phân bổ hạn cũ nhất trước, phần thừa thành số dư có (credit). 3. Phí hư hại phải có ảnh bằng chứng và gắn mục checklist. 4. Trả sớm: early_termination_refund = NONE (mặc định) hoặc UNUSED_FULL_MONTHS. 5. Không xóa/sửa dòng hóa đơn đã phát hành; điều chỉnh bằng credit note.
17. Lỗi có thể xảy ra	• Tự chốt quyết toán sau 3 ngày (moveout.auto-finalize) chạy theo giờ thật, không tua được trên bản demo (RISK-10): khi demo dùng nút chốt thủ công. • Khách mở/đóng kho từ xa: hiện là mô phỏng (ghi access_events, chưa nối khóa thật).
18. Cách kiểm thử	Test tự động: apps/api/test/e2e/f3-moveout.ts (15 kiểm tra); unit test "move-out settlement calculation" trong test/unit/shared.test.ts. Test case thủ công TC-F3-01 → TC-F3-11.
19. Cách demo	Triển, khách từ F1/F2 (hoặc customer@vaultra.test với hợp đồng CT-2026-000018 đang hiệu lực), mục 5.2 bước F3.
Trạng thái triển khai	**Đã kiểm chứng** 08/11/2026 trên Worker cục bộ: f3-moveout 15/15 đạt; unit test quyết toán đạt. Mở/đóng kho từ xa đã gọi thử trên bản deploy (OPEN → CLOSED).

Ví dụ thực tế và kịch bản thao tác

Sau 3 tháng, chị Lan báo trả kho ngày 20. Ước tính: cọc 690.000 ₫, còn nợ 50.000 ₫ phụ phí, không hư hại → dự kiến hoàn 640.000 ₫. Ngày 20, nhân viên chụp ảnh so với lúc nhận, không thấy hư hại, thu hồi PIN; chị ký biên bản. Sau 3 ngày không khiếu nại, quyết toán được chốt và tạo yêu cầu hoàn tiền.

#	Thao tác	Kết quả mong đợi
1	Khách mở /app/contracts/:id → tab Trả kho, chọn ngày	Hiện ước tính quyết toán (cọc, nợ, hoàn dự kiến)
2	Gửi yêu cầu	Hợp đồng "Sắp trả kho"; có lịch kiểm tra
3	Nhân viên /staff/moveouts → mở yêu cầu, kiểm tra, (tùy chọn) ghi hư hại kèm ảnh	Yêu cầu INSPECTED
4	Chốt kiểm tra, khách ký đồng ý	PIN bị thu hồi; kho "Chờ kiểm tra"; quyết toán DRAFT
5	Quản lý chốt quyết toán; nhân viên kiểm tra cuối	Kho về "Trống"; hoàn tiền chờ duyệt

20. Câu hỏi giảng viên có thể đặt ra

- Công thức quyết toán cọc là gì? Ví dụ số?
- Vì sao không tự trừ phí hư hại ngay? (Có 3 ngày khiếu nại; phí lớn cần FM duyệt.)
- Kho về "Trống" khi nào? (Ba điều kiện backend kiểm tra.)
- Khách trả một phần hóa đơn thì sao? (PARTIALLY_PAID; vẫn tính vào chuỗi quá hạn F6.)

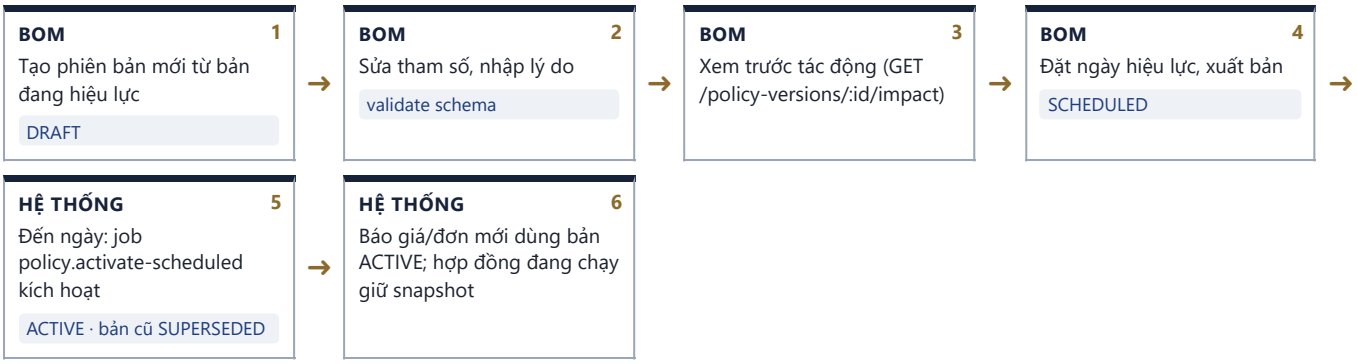
> **Điểm cần lưu ý khi bảo vệ**
> Chuẩn bị sẵn một ví dụ số cho công thức quyết toán. Nói rõ hoàn tiền thực hiện bằng chuyển khoản/tiền mặt do nhân viên ghi nhận (mark-manual), vì cổng thanh toán không tự hoàn.

3.4 Flow 4 — Chính sách, phí và theo dõi doanh thu (Business Rules, Fee Management, Revenue Monitoring)

Người trình bày chính: **Phúc** · Độ khó: Khó · Màn hình chính: /ops/policies, /ops/pricing, /ops/promotions, /ops/finance, /ops/reports

1. Mục đích nghiệp vụ	BOM thiết lập và thay đổi quy tắc kinh doanh (cọc, hủy, gia hạn, trả kho, quá hạn), khung giá, danh mục phí, khuyến mãi, miễn giảm; theo dõi doanh thu và hiệu quả từng cơ sở. Mọi quy tắc là dữ liệu có phiên bản, không viết cứng trong code.
2. Vấn đề thực tế	Nếu sửa chính sách trực tiếp, hợp đồng cũ bị đổi điều khoản giữa chừng; nếu giá do từng cơ sở tự đặt không giới hạn thì khó quản trị; doanh nghiệp cần biết phát hành bao nhiêu, thu bao nhiêu, nợ bao nhiêu.
3. Actor	BUSINESS_OPS_MANAGER (sở hữu chính sách, giá, duyệt vượt hạn mức) · FACILITY_MANAGER (đặt giá trong khung, đề xuất ngoài khung, duyệt miễn giảm trong hạn mức) · FACILITY_STAFF (đề xuất miễn giảm) · Hệ thống (áp dụng, tổng hợp số liệu).
4–5. Điều kiện bắt đầu / tiên quyết	Có bộ chính sách mặc định được seed (DEFAULT_POLICY_PARAMS trong packages/shared/src/policy.ts).

Sơ đồ quy trình (luồng chính)



Hình 3.4 — Luồng chính F4

Chính sách **không hồi tố**: hợp đồng lưu policy_version_id lúc ký nên đổi chính sách không làm đổi điều khoản hợp đồng cũ. Validate có quan hệ thứ tự: grace_days < lockout_day < final_notice_day < default_day; bậc hủy phải giảm dần (unit test policyParamsSchema).



SCHEDULED có thể hủy trước ngày hiệu lực (POST /policy-versions/:id/cancel); muốn sửa bản ACTIVE thì tạo bản mới

Vòng đời policy_versions

8. Luồng thay thế	• Giá ngoài khung: FM tạo price_change_requests → BOM duyệt/từ chối (POST /price-change-requests/:id/decide). • Miễn giảm: nhân viên/FM đề xuất trên một dòng hóa đơn (POST /fee-waivers) → duyệt theo hạn mức (FM ≤ 1.000.000 ₫, vượt → BOM) → sinh credit note. • Đối soát thanh toán (POST /reconciliations): so sao kê của cổng (VNPay) hoặc sổ thu tại quầy (tiền mặt/chuyển khoản) với hệ thống theo ngày; dòng lệch có nút giải trình.

9. Ngoại lệ	• Tham số không hợp lệ → 400 kèm danh sách trường; trùng hiệu lực cùng phạm vi → POLICY_OVERLAP. • Khuyến mãi đã có lượt dùng chỉ được tạm dừng/kết thúc, không xóa. • VNPay sandbox không cho Worker tra cứu (querydr trả 522): đối soát demo dùng dữ liệu của chính hệ thống làm sao kê (Chương 8, RISK-07).
10. Điều kiện hoàn thành	Phiên bản chuyển ACTIVE đúng ngày và báo giá mới phản ánh tham số mới; báo cáo/dashboard hiển thị đúng số liệu.
11. Dữ liệu trước → sau	policy_version: — → DRAFT → SCHEDULED → ACTIVE; bản cũ → SUPERSEDED · price_change_request: PENDING → APPROVED/REJECTED · fee_waiver: REQUESTED → APPROVED → credit_note · reconciliation_batch: — → MATCHED/HAS_MISMATCH → RESOLVED.
12. Bảng database	policies, policy_versions, price_bands, price_history, price_change_requests, fee_types, promotions, promotion_redemptions, fee_waivers, credit_notes, daily_facility_metrics, report_exports, reconciliation_batches, reconciliation_items, audit_logs.
13. API / backend	GET/POST /policies · POST /policies/:id/versions · PATCH /policy-versions/:id · GET /policy-versions/:id/impact · POST /policy-versions/:id/publish · …/cancel · GET/POST/DELETE /price-bands · GET /price-change-requests · POST /fee-types · GET/POST/PATCH /promotions · POST /fee-waivers · POST /fee-waivers/:id/decide · POST /invoices/:id/credit-notes · GET/POST /reconciliations · GET /dashboard/portfolio · GET /reports/... (report.controller.ts).
14. Màn hình frontend	/ops/policies, /ops/pricing, /ops/promotions, /ops/approvals, /ops/finance (đối soát), /ops/reports, /ops/compare (so sánh cơ sở), /manage/reports, /manage/approvals.
15. Business logic nằm ở	policy/policy.controller.ts · domain/pricing.service.ts (giá, khuyến mãi, VAT) · packages/shared/src/policy.ts (schema + mặc định) · billing/billing.controller.ts (miễn giảm, credit note, đối soát) · report/report.service.ts (báo cáo, xuất CSV/XLSX/PDF) · jobs/jobs.service.ts (activateScheduledPolicies, report.daily-metrics).
16. Quy tắc nghiệp vụ quan trọng	1. Ba khái niệm tiền tách bạch: Phát hành (tổng hóa đơn), Thu được (thanh toán trừ hoàn), Ghi nhận (doanh thu theo thời gian đã cung cấp dịch vụ). Tiền cọc là nợ phải trả, không phải doanh thu. 2. Giá hợp đồng đang chạy không đổi giữa kỳ; chỉ đổi khi gia hạn theo renewal_price_rule (tăng tối đa 10%). 3. Mọi thay đổi giá/chính sách/phí ghi audit_logs kèm lý do. 4. Không cộng dồn hai khuyến mãi trừ khi cờ stackable. 5. Phí trễ hạn: GREATER_OF(50.000 ₫, 10% tiền thuê), trần 500.000 ₫, mỗi hóa đơn một lần.
17. Lỗi có thể xảy ra	• Đổi chính sách xong nhưng báo giá chưa đổi: bản mới mới ở SCHEDULED; job policy.activate-scheduled chạy mỗi 15 phút nên bản hẹn ngày sẽ được kích hoạt sau khi đến ngày hiệu lực; khi demo nên xuất bản hiệu lực ngay.
18. Cách kiểm thử	Unit test: policyParamsSchema, pricing & promotions, delinquency lateFee, renewal price (test/unit/shared.test.ts). E2E: f4-f7-lifecycle.ts (34 kiểm tra, chưa chạy lại được trên Worker do giới hạn đăng nhập). Test case TC-F4-01 → TC-F4-10.
19. Cách demo	Phúc, tài khoản bom@vaultra.test, mục 5.2 bước F4.
Trạng thái triển khai	**Đã kiểm chứng** 08/11/2026: 16/16 unit test đạt; đối soát VNPay/tiền mặt chạy trên bản deploy (MATCHED). **Chưa xác minh**: f4-f7-lifecycle trên bản hiện tại.

Ví dụ thực tế và kịch bản thao tác

BOM muốn tăng ân hạn quá hạn từ 5 lên 7 ngày cho toàn hệ thống từ ngày 01/12. BOM tạo phiên bản mới, sửa grace_days = 7, nhập lý do "Hỗ trợ khách dịp cuối năm", xem trước: 12 hợp đồng gia hạn sắp tới sẽ chịu bản mới. Xuất bản với hiệu lực 01/12. Hợp đồng đang chạy vẫn giữ 5 ngày cho đến kỳ gia hạn.

#	Thao tác	Kết quả mong đợi
1	BOM mở /ops/policies, tạo phiên bản mới	Bản DRAFT sao chép tham số hiện hành
2	Sửa một tham số, nhập lý do, lưu	Validate; sai thứ tự mốc quá hạn bị báo lỗi
3	Bấm Xem tác động	Hiện số đơn/hợp đồng chịu ảnh hưởng và khác biệt từng tham số
4	Xuất bản	Trạng thái SCHEDULED hoặc ACTIVE
5	Mở /ops/finance → Chạy đối soát ngày có giao dịch	Lô đối soát Khớp/Có chênh lệch

20. Câu hỏi giảng viên có thể đặt ra

- Vì sao chính sách phải có phiên bản? (Không hồi tố, truy được hợp đồng chịu bản nào.)
- Nếu BOM nhập grace_days lớn hơn lockout_day thì sao? (Schema chặn.)
- Doanh thu khác tiền thu thế nào? Cọc có phải doanh thu không?
- Đối soát dùng để làm gì? Trên bản demo lấy dữ liệu cổng ở đâu? (Nói thật: mô phỏng.)

> **Điểm cần lưu ý khi bảo vệ**
> Phúc nên mở file packages/shared/src/policy.ts khi được hỏi "tham số nằm ở đâu". Về đối soát: nói rõ bản demo dùng sao kê mô phỏng từ dữ liệu hệ thống; có thể dán sao kê thủ công để tạo chênh lệch.

3.5 Flow 5 — Quản lý kho và nhân sự cơ sở (Facility Storage and Staff Management)

Người trình bày chính: **Trường** · Độ khó: Trung bình – Khó · Màn hình chính: /manage/floor, /manage/units, /manage/staff, /manage/schedule, /manage/tasks, /manage/maintenance

1. Mục đích nghiệp vụ	FM quản lý tài sản kho (khu/tầng, loại kho, từng kho, sơ đồ, trạng thái, bảo trì), phân bổ kho cho khách và quản lý nhân viên (ca làm, công việc, hiệu suất) tại cơ sở được phân công.
2. Vấn đề thực tế	Kho trên sổ phải khớp thực tế (kho hỏng không được bán); nhân viên cần biết hôm nay làm gì; FM cần thấy cơ sở đang đầy bao nhiêu và việc nào quá hạn.
3. Actor	FACILITY_MANAGER (chủ trì) · FACILITY_STAFF (thực hiện việc, cập nhật trạng thái kho) · BUSINESS_OPS_MANAGER (tạo/đóng cơ sở) · SYSTEM_ADMIN (tạo tài khoản, gán cơ sở) · Hệ thống (sinh việc tự động).
4–5. Điều kiện bắt đầu / tiên quyết	Cơ sở đã được BOM tạo; FM đã được Admin gán vai trò FACILITY_MANAGER cho cơ sở đó.

Sơ đồ quy trình (luồng chính)



Hình 3.5 — Luồng chính F5

Thuật toán phân bổ dễ giải thích: tập ứng viên = kho cùng cơ sở, cùng loại, AVAILABLE, không cờ vận hành, không allocation mở, không work order mở; xếp theo allocation_priority tăng dần → kho trống lâu nhất trước (last_vacated_at) → mã kho. Gán trong giao dịch có khóa hàng; UNIQUE(open_unit_key) là lớp bảo vệ cuối.



AVAILABLE ↔ MAINTENANCE / OUT_OF_SERVICE (kèm lý do) · OCCUPIED ↔ OVERLOCKED (quá hạn) · OUT_OF_SERVICE → RETIRED · không chuyển kho đang thuê sang bảo trì trực tiếp

Máy trạng thái units.status

8. Luồng thay thế	• Work order bảo trì: REPORTED → APPROVED → SCHEDULED → IN_PROGRESS → COMPLETED → VERIFIED (POST /work-orders/:id/:action), có ảnh trước/sau. • Kế hoạch bảo trì định kỳ (maintenance_plans) sinh work order theo chu kỳ (job maintenance.plans). • Nhân viên xin đổi ca (POST /shift-swaps) → FM duyệt. • Chuyển hàng loạt việc khi nhân viên nghỉ (POST /tasks/bulk-reassign).
9. Ngoại lệ	• Trùng mã kho trong cơ sở → UNIT_CODE_DUPLICATE. • Chuyển trạng thái không hợp lệ → 409 INVALID_STATUS_TRANSITION. • Đổi loại kho khi còn allocation mở → UNIT_HAS_OPEN_ALLOCATION. • Ca chồng nhau của cùng người → SHIFT_OVERLAP; giao việc cho người ngoài cơ sở → ASSIGNEE_NOT_IN_FACILITY.
10. Điều kiện hoàn thành	Kho có trạng thái đúng thực tế, đơn xác nhận được phân bổ, mọi khung hẹn có người trực, việc xong đúng hạn.
11. Dữ liệu trước → sau	unit: theo máy trạng thái + unit_status_history · unit_allocation: — → ACTIVE → RELEASED · staff_shift: PLANNED → PUBLISHED · work_task: OPEN → ASSIGNED → IN_PROGRESS → DONE/BLOCKED · work_order: REPORTED → … → VERIFIED.
12. Bảng database	zones, unit_types, units, unit_status_history, floor_plans, unit_shapes, unit_allocations, shift_templates, staff_shifts, shift_swap_requests, work_tasks, task_events, work_orders, maintenance_plans, facility_notices.
13. API / backend	POST /facilities/:id/zones · POST /facilities/:id/unit-types · PATCH /unit-types/:id · POST /facilities/:id/units/bulk · PATCH /units/:id · POST /units/:id/status · GET /units/:id/history · GET/PUT /zones/:zoneId/floor-plan · GET /reservations/:id/allocation-suggestions · POST /reservations/:id/allocate · /facilities/:id/shift-templates · /facilities/:id/shifts · POST /shifts/publish · GET /me/shifts · /tasks (claim, start, complete, block, cancel, assign) · /work-orders · /facilities/:id/maintenance-plans · GET /dashboard/facility/:id.

14. Màn hình frontend	/manage/floor (sơ đồ + chỉnh sửa bố cục), /manage/unit-types, /manage/units, /manage/staff, /manage/schedule, /manage/tasks, /manage/maintenance, /manage (Cockpit); nhân viên: /staff/floor, /staff/tasks, /staff/schedule.
15. Business logic nằm ở	facility/facility.controller.ts (kho, loại kho, sơ đồ, trạng thái) · workforce/workforce.controller.ts (ca, việc, work order, kế hoạch bảo trì) · reservation/reservation.service.ts (gợi ý và gán kho) · packages/shared/src/states.ts (máy trạng thái).
16. Quy tắc nghiệp vụ quan trọng	1. Không đổi unit_type của kho có allocation mở; không chuyển kho OCCUPIED/RESERVED sang MAINTENANCE trực tiếp (phải đổi kho cho khách trước). 2. Nhân viên chỉ cập nhật việc được giao hoặc việc OPEN của cơ sở (tự nhận). 3. Mọi đổi trạng thái kho ghi unit_status_history và phát sự kiện realtime cho màn hình sơ đồ. 4. FM không tạo tài khoản nhân viên; Admin tạo và gán cơ sở.
17. Lỗi có thể xảy ra	• Di chuột qua kho đang có khách trên sơ đồ từng làm trắng trang (render object khách) — đã sửa 08/11/2026 và thêm ErrorBoundary. • Ca làm việc chưa ràng buộc nghiệp vụ check-in/thu tiền (đặc tả không yêu cầu; nếu giảng viên hỏi, trả lời đúng như vậy).
18. Cách kiểm thử	E2E f4-f7-lifecycle.ts (phần F5), unit test máy trạng thái (states). Test case TC-F5-01 → TC-F5-10.
19. Cách demo	Trường, tài khoản fm.hn@vaultra.test, mục 5.2 bước F5.
Trạng thái triển khai	**Đã kiểm chứng**: 4 unit test máy trạng thái đạt (08/11/2026); tạo việc + realtime đã thử trên bản deploy. **Chưa xác minh**: e2e F5 trên bản hiện tại.

Ví dụ thực tế và kịch bản thao tác

Cơ sở Cầu Giấy nhận thêm 10 kho "Tủ khóa 1m²". FM tạo hàng loạt A-041 → A-050, đặt lên sơ đồ khu A. Một kho bị thấm nước: nhân viên đổi trạng thái sang Bảo trì kèm lý do, tạo work order; sau khi sửa xong FM xác minh, kho về Trống. FM lập ca sáng 07:00–15:00 cho hai nhân viên và giao việc kiểm tra định kỳ.

#	Thao tác	Kết quả mong đợi
1	FM mở /manage/floor, chọn khu	Sơ đồ màu theo trạng thái, số liệu lấp đầy
2	Bấm một kho trống → đổi sang Bảo trì kèm lý do	Màu đổi; lịch sử trạng thái ghi lý do
3	/manage/tasks → Tạo việc, giao cho nhân viên	Nhân viên (cửa sổ khác) thấy việc mới không cần F5
4	Nhân viên nhận → bắt đầu → hoàn thành	Việc DONE, ghi thời gian thực tế
5	/manage/schedule → lập và xuất bản ca	Nhân viên xem ở /staff/schedule

20. Câu hỏi giảng viên có thể đặt ra

- Thuật toán chọn kho để phân bổ là gì? Vì sao không chọn ngẫu nhiên?
- Kho đang có khách bị hỏng thì làm sao? (Cờ vận hành, work order; muốn đưa vào bảo trì phải đổi kho cho khách.)
- Làm sao biết ai đổi trạng thái kho, khi nào, vì sao? (unit_status_history + audit_logs.)
- Nhân viên A có sửa được việc của nhân viên B không?

> **Điểm cần lưu ý khi bảo vệ**
> Mở sơ đồ /manage/floor và giải thích bằng màu sắc; đó là phần trực quan nhất của F5. Thuật toán phân bổ nói theo ba tiêu chí xếp hạng.

3.6 Flow 6 — Gia hạn và xử lý quá hạn (Storage Renewal and Overdue

Handling)

Người trình bày chính: **Trường** · Độ khó: Khó · Màn hình chính: /app/contracts/:id (Gia hạn), /manage/renewals, /manage/delinquency, /ops/delinquency

1. Mục đích nghiệp vụ	Gia hạn hợp đồng trước khi hết hạn (khách tự gia hạn, nhân viên gia hạn hộ, tự động), xử lý hợp đồng hết hạn mà không gia hạn, và chuỗi xử lý nợ quá hạn từ nhắc nhở đến chấm dứt do vi phạm.
2. Vấn đề thực tế	Khách quên gia hạn thì kho bị bỏ trống hoặc tranh chấp; khách nợ kéo dài thì doanh nghiệp mất tiền nhưng không thể tự ý thu hồi kho — cần quy trình có từng mốc, có bằng chứng thông báo, có người duyệt.
3. Actor	CUSTOMER · FACILITY_STAFF (gia hạn hộ, gắn/tháo khóa chặn) · FACILITY_MANAGER (chặn gia hạn, ghi cam kết trả, xác nhận chấm dứt) · BUSINESS_OPS_MANAGER (phê duyệt chấm dứt, xử lý tài sản) · Hệ thống (job mỗi giờ).
4–5. Điều kiện bắt đầu / tiên quyết	Gia hạn: hợp đồng ACTIVE, còn ≤ 30 ngày (renewal_window_days) đến ngày kết thúc. Quá hạn: có hóa đơn ISSUED/PARTIALLY_PAID đã qua due_date.

Sơ đồ quy trình (luồng chính)



Hình 3.6 — Luồng chính F6

Chấm dứt do vi phạm **không bao giờ tự động**: hệ thống chỉ tạo yêu cầu, FM xác nhận và BOM duyệt. Ở bất kỳ mốc nào trước DEFAULTED, khách trả đủ thì về CURRENT, PIN mở lại tự động và tạo việc tháo khóa chặn.

`CURRENT` → `GRACE` → `LATE` → `LOCKED_OUT` → `FINAL_NOTICE` → `DEFAULT_PENDING` → `DEFAULTED`

Mọi giai đoạn trước DEFAULTED → CURRENT khi trả đủ

contracts.delinquency_status (mốc mặc định: 5/10/30/45 ngày)

8. Luồng thay thế	• Bật/tắt tự gia hạn (PATCH /contracts/:id/auto-renew). • FM chặn gia hạn (POST /contracts/:id/renewal-block) kèm lý do, ví dụ kho sắp cải tạo. • Hết hạn không gia hạn: HOLDOVER_MONTHLY (tiếp tục theo tháng) hoặc REQUIRE_MOVE_OUT (job renewal.expiries). • FM ghi cam kết trả (POST /contracts/:id/promise-to-pay) và ghi chú thu nợ (collection-notes).
9. Ngoại lệ	• Gia hạn khi đang nợ → 409 RENEWAL_BLOCKED_OUTSTANDING_BALANCE. • OTP sai/hết hạn hoặc không khớp thời hạn đã chọn → phải gửi lại. • Không khóa truy cập khi hóa đơn đang có ticket tranh chấp (dispute_hold). • Chưa mở cửa sổ gia hạn → "Chưa mở cửa sổ gia hạn cho hợp đồng này".

10. Điều kiện hoàn thành	Gia hạn: kỳ mới liền mạch, không gián đoạn truy cập. Quá hạn: khách trả đủ và về CURRENT, hoặc hợp đồng chấm dứt đúng quy trình có bằng chứng.
11. Dữ liệu trước → sau	renewal_offer: — → OPEN → ACCEPTED/LAPSED · contract_terms: thêm kỳ mới · contracts.current_end_date dời · access_credentials.valid_to gia hạn · delinquency_events ghi mỗi lần chuyển mốc · access_credential: ACTIVE ↔ SUSPENDED · overlock tạo/gỡ · termination_request: — → FM_CONFIRMED → APPROVED · contract → TERMINATED.
12. Bảng database	renewal_offers, contract_terms, contract_amendments, invoices (RENEWAL, dòng LATE_FEE), delinquency_events, collection_notes, promise_to_pay, overlocks, termination_requests, abandoned_goods_records, access_credentials, notifications.
13. API / backend	GET /contracts/:id/renewal-offer · POST /contracts/:id/renew/otp · POST /contracts/:id/renew · PATCH /contracts/:id/auto-renew · POST /contracts/:id/renewal-block · GET /delinquency · GET/POST /contracts/:id/collection-notes · POST /contracts/:id/promise-to-pay · POST /overlocks · POST /overlocks/:id/remove · GET /termination-requests · POST /termination-requests/:id/confirm · POST /termination-requests/:id/decide · /abandoned-goods.
14. Màn hình frontend	Khách: /app/contracts/:id → tab Gia hạn. Quản lý: /manage/renewals, /manage/delinquency (Collections). BOM: /ops/delinquency (duyệt chấm dứt).
15. Business logic nằm ở	lifecycle/renewal.service.ts (offer, OTP, gia hạn, hết hạn) · lifecycle/delinquency.service.ts (runJob, đánh giá mốc, phí trễ, khóa/mở) · packages/shared (targetStage, lateFee — có unit test).
16. Quy tắc nghiệp vụ quan trọng	1. Job delinquency.run chạy mỗi giờ, idempotent: chỉ ghi khi giai đoạn mục tiêu khác hiện tại; mỗi lần chuyển tạo delinquency_events. 2. Phí trễ một lần mỗi hóa đơn, tôn trọng trần 500.000 ₫; phí khóa chặn 100.000 ₫. 3. Khách từ LATE trở lên bị chặn đặt kho mới và gia hạn. 4. Giá gia hạn: giá niêm yết hiện hành, tăng tối đa 10%; chưa báo trước đủ 30 ngày thì giữ giá cũ. 5. Phụ lục gia hạn chấp nhận bằng OTP gửi email.
17. Lỗi có thể xảy ra	• Job delinquency.run và renewal.windows chạy mỗi giờ trên bản demo (cron), nhưng bản deploy không cho tua thời gian (RISK-10): các mốc D+6, D+10… chỉ xảy ra theo ngày thật, không thể hiện trong buổi demo. Bằng chứng dùng unit test + e2e trên bản Node (có ENABLE_TEST_CLOCK). • Email OTP gia hạn từng không đến do SMTP từ chối đăng nhập (535) — đã thêm thử lại; xem notification_deliveries.
18. Cách kiểm thử	Unit test: "delinquency targetStage and lateFee", "renewal price calculation". E2E f4-f7-lifecycle.ts (phần F6, dùng đồng hồ thử ENABLE_TEST_CLOCK trên bản Node). Test case TC-F6-01 → TC-F6-10.
19. Cách demo	Trường, khách từ F1/F2 (gia hạn), fm.hn@vaultra.test (màn hình công nợ), mục 5.2 bước F6.
Trạng thái triển khai	**Đã kiểm chứng**: unit test mốc quá hạn, phí trễ, giá gia hạn đạt (08/11/2026); gửi OTP gia hạn đã gửi được trên bản deploy (người dùng thử). Cron chạy delinquency.run thành công trên bản deploy (08/11/2026). **Giới hạn**: không tua thời gian được nên không demo trực tiếp các mốc.

Ví dụ thực tế và kịch bản thao tác

Hợp đồng của anh Minh hết hạn 07/02/2027. Từ 08/01 tab Gia hạn mở với giá kỳ mới 319.000 ₫/tháng (+10%). Anh chọn 1 tháng, nhận OTP qua email, xác nhận và thanh toán. Nếu anh không trả hóa đơn kỳ: ngày thứ 6 bị cộng phí trễ 50.000 ₫, ngày thứ 10 PIN bị chặn và nhân viên gắn khóa chặn; trả đủ thì PIN mở lại ngay.

#	Thao tác	Kết quả mong đợi
1	Khách mở /app/contracts/:id → Gia hạn	Hiện giá kỳ mới, các thời hạn cho phép, ngày hết hạn mới dự kiến

#	Thao tác	Kết quả mong đợi
2	Bấm Gửi mã, nhập OTP từ email, bấm Chấp nhận & gia hạn	Tạo hóa đơn RENEWAL
3	Thanh toán hóa đơn	Ngày hết hạn hợp đồng dời; có phụ lục PDF
4	FM mở /manage/delinquency	Danh sách hợp đồng theo giai đoạn quá hạn, ghi chú thu nợ
5	(Bản Node) chạy job delinquency.run với đồng hồ thử	Hợp đồng chuyển mốc, PIN SUSPENDED ở LOCKED_OUT

20. Câu hỏi giảng viên có thể đặt ra

- Vì sao không tự chấm dứt hợp đồng khi khách nợ 45 ngày?
- Khách trả tiền ngay sau khi bị khóa thì sao?
- Job chạy mỗi giờ, chạy hai lần có cộng phí trễ hai lần không? (Idempotent, mỗi hóa đơn một lần.)
- Trên bản demo làm sao thấy chuỗi quá hạn? (Nói thật giới hạn; trình bày bằng unit test/e2e.)

> **Điểm cần lưu ý khi bảo vệ**
> Thuộc bảng mốc D+1/D+6/D+10/D+30/D+45 và ý nghĩa từng mốc. Chủ động nói giới hạn job trên bản Worker trước khi bị hỏi; đưa bằng chứng unit test.

3.7 Flow 7 — Hỗ trợ và xử lý sự cố (Support Request and Issue Handling)

Người trình bày chính: **Khang** · Độ khó: Dễ – Trung bình · Màn hình chính: /app/support, /staff/tickets, /manage/tickets, /ops/sla

1. Mục đích nghiệp vụ	Tiếp nhận, phân loại, phân công, xử lý, theo dõi SLA và đóng yêu cầu hỗ trợ/sự cố về kho, khóa, mã truy cập, thanh toán, tài sản lưu trữ, kể cả sự cố do nhân viên phát hiện.
2. Vấn đề thực tế	Khách gọi điện báo sự cố thì không ai theo dõi được đã xử lý chưa, mất bao lâu; doanh nghiệp cần cam kết thời gian phản hồi (SLA) và đo được hiệu suất nhân viên.
3. Actor	CUSTOMER (tạo, trao đổi, xác nhận, đánh giá) · FACILITY_STAFF (tạo hộ, nhận, xử lý) · FACILITY_MANAGER (phân công, leo thang, quyết định bồi thường) · BUSINESS_OPS_MANAGER (leo thang cấp 2 về tiền) · Hệ thống (SLA, nhắc, tự đóng).
4–5. Điều kiện bắt đầu / tiên quyết	Khách đăng nhập; ticket gắn cơ sở (có thể gắn hợp đồng của chính khách); hoặc nhân viên/FM phát hiện sự cố.

Sơ đồ quy trình (luồng chính)



Hình 3.7 — Luồng chính F7

SLA tính theo giờ làm việc của cơ sở (bỏ ngoài giờ và Chủ nhật), riêng URGENT tính 24/7 — có unit test "support SLA addBusinessMinutes". Ví dụ: ACCESS_CODE ưu tiên HIGH, phản hồi 30 phút, giải quyết 4 giờ.



IN_PROGRESS ↔ WAITING_CUSTOMER · RESOLVED → REOPENED → IN_PROGRESS · chưa đóng → CANCELLED (gộp trùng: merge)

Máy trạng thái support_tickets.status

8. Luồng thay thế	• Nhân viên tạo ticket hộ khách tại quầy (channel COUNTER). • Leo thang: Staff → FM (vượt thẩm quyền hoặc sắp trễ SLA), FM → BOM (bồi thường vượt hạn mức). • Gộp ticket trùng (merge); ticket phụ CANCELLED. • Trợ lý AI gợi ý câu trả lời và tóm tắt ticket cho nhân viên (assistant).
9. Ngoại lệ	• Ghi chú nội bộ (is_internal) không bao giờ trả về API của khách. • Ticket PAYMENT gắn hóa đơn → dispute_hold, F6 không khóa truy cập vì hóa đơn đang tranh chấp. • Khách chỉ gắn ticket vào hợp đồng của mình; gắn hợp đồng người khác → 404. • Không xóa ticket; chỉ CANCELLED.
10. Điều kiện hoàn thành	Ticket CLOSED có resolution_code, trong SLA, khách đánh giá ≥ 4. Cảnh báo: vượt SLA (sla_breached), bị mở lại, đánh giá ≤ 2.
11. Dữ liệu trước → sau	support_ticket: — → NEW → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED · ticket_messages, ticket_events (mỗi thay đổi), ticket_ratings · có thể sinh work_orders, fee_waivers, refunds, credential mới.
12. Bảng database	support_tickets, ticket_messages, ticket_events, sla_policies, ticket_ratings, media_files, work_tasks (SUPPORT_TICKET), work_orders, fee_waivers, refunds, access_credentials.
13. API / backend	POST /tickets · GET /tickets · GET /tickets/:id · POST /tickets/:id/messages · POST /tickets/:id/:action (claim, assign, priority, start, wait-customer, escalate, resolve, close, reopen, rate, cancel, merge) · POST /tickets/:id/actions/:kind (reissue-pin, work-order, charge, fee-waiver, refund) · GET/PATCH /sla-policies.
14. Màn hình frontend	Khách: /app/support, /app/support/:id. Nhân viên: /staff/tickets, /staff/tickets/:id. Quản lý: /manage/tickets. BOM: /ops/sla (cấu hình SLA).
15. Business logic nằm ở	support/support.service.ts (tạo, chuyển trạng thái, SLA, hành động liên kết, slaJob) · support/support.controller.ts · assistant/assistant.service.ts (gợi ý trả lời).
16. Quy tắc nghiệp vụ quan trọng	1. Bảng SLA mặc định: ACCESS_CODE 30 phút/4 giờ · LOCK_KEY 30 phút/8 giờ · UNIT_DAMAGE 1 giờ/24 giờ · STORED_ITEMS 1 giờ/72 giờ · PAYMENT 4 giờ/3 ngày · CONTRACT, OTHER 8 giờ/5 ngày. 2. Bắt buộc nguyên nhân khi đánh dấu đã giải quyết. 3. Khách mở lại trong 7 ngày (reopen_days); không phản hồi 72 giờ thì tự đóng. 4. Mỗi thay đổi trạng thái, người xử lý, ưu tiên ghi ticket_events.
17. Lỗi có thể xảy ra	• Job support.sla chạy mỗi 5 phút trên bản demo; cảnh báo trễ SLA và tự đóng sau 72 giờ diễn ra theo giờ thật, không thể hiện được trong buổi demo.
18. Cách kiểm thử	Unit test SLA (test/unit/shared.test.ts). E2E f4-f7-lifecycle.ts (phần F7). Test case TC-F7-01 → TC-F7-10.
19. Cách demo	Khang, customer@vaultra.test + staff.hn@vaultra.test, mục 5.2 bước F7.
Trạng thái triển khai	**Đã kiểm chứng**: unit test SLA đạt (08/11/2026). **Chưa xác minh**: luồng ticket đầy đủ trên bản deploy sau thay đổi gần nhất — nhóm cần chạy thử TC-F7-01 → 05 trước ngày bảo vệ.

Ví dụ thực tế và kịch bản thao tác

Anh Minh không mở được cổng vì PIN báo sai. Anh tạo ticket danh mục "Mã truy cập", hệ thống đặt ưu tiên Cao, hạn phản hồi 30 phút. Nhân viên trực nhận ticket, kiểm tra thấy hợp đồng không nợ, bấm "Cấp lại PIN" ngay trong ticket, ghi

nguyên nhân "khách nhập nhầm mã cũ", đánh dấu đã giải quyết. Anh Minh đánh giá 5 sao.

#	Thao tác	Kết quả mong đợi
1	Khách /app/support → Tạo yêu cầu, chọn hợp đồng, danh mục, mô tả, ảnh	Ticket SR-… trạng thái Mới, hiện hạn SLA
2	Nhân viên /staff/tickets → mở ticket → Nhận việc	ASSIGNED → IN_PROGRESS; khách thấy cập nhật không cần F5
3	Nhân viên trả lời khách, thêm ghi chú nội bộ	Khách chỉ thấy tin công khai
4	Đánh dấu đã giải quyết, nhập nguyên nhân	RESOLVED
5	Khách đánh giá 5 sao	CLOSED, có đánh giá

20. Câu hỏi giảng viên có thể đặt ra

- SLA tính thế nào khi khách tạo ticket lúc 21:00?
- Khách có xem được ghi chú nội bộ không? Làm sao đảm bảo?
- Ticket được giao cho ai nếu không có ai trực?
- Vì sao không cho xóa ticket?

> **Điểm cần lưu ý khi bảo vệ**
> Đây là Flow trực quan, Khang nên vừa làm vừa kể như một câu chuyện có thật. Dùng hai cửa sổ (khách và nhân viên) để cho thấy realtime và việc ghi chú nội bộ bị ẩn với khách.

CHƯƠNG 4

Hướng dẫn kiểm thử toàn hệ thống

> Lý thuyết đủ dùng, áp dụng trực tiếp lên VAULTRA: loại kiểm thử, 74 test case cho 7 Flow, kiểm thử nghiệp vụ chuyên sâu, Postman, SQL và test tự động có sẵn trong repo.

4.1 Các loại kiểm thử áp dụng cho VAULTRA

Loại	Khái niệm	Mục đích / khi dùng	Ví dụ trong VAULTRA	Cách làm	Kết quả mong đợi
Functional Testing	Kiểm tra chức năng làm đúng yêu cầu nghiệp vụ.	Mỗi khi hoàn thành một chức năng.	Đặt kho thành công tạo đơn PENDING_PAYMENT và hóa đơn ISSUED.	Làm theo kịch bản, so với đặc tả mục 6.	Kết quả khớp bảng "Trạng thái trước → sau".
Unit Testing	Kiểm tra một hàm/khối logic riêng lẻ, không cần database.	Logic tính toán, quy tắc.	test/unit/shared.test.ts: tính giá, phí trễ, quyết toán, mốc quá hạn, SLA.	pnpm --filter @vaultra/api test	16/16 đạt (đã chạy 08/11/2026).
Integration Testing	Kiểm tra nhiều thành phần chạy cùng nhau (API + DB + cổng).	Khi nối service với DB/cổng thanh toán.	Thanh toán: startGateway → IPN VNPay → settle → hóa đơn PAID.	Chạy test/e2e/vnpay.ts với khóa sandbox.	IPN đúng → xác nhận; IPN giả → từ chối.
System Testing	Kiểm tra toàn hệ thống trên môi trường giống thật.	Trước khi demo/bàn giao.	Chạy 7 Flow trên vaultra-844.pages.dev.	Theo kịch bản Chương 5.	Không lỗi 5xx, dữ liệu đúng ở mọi vai trò.
End-to-End Testing	Đi trọn một luồng nghiệp vụ từ đầu đến cuối qua API.	Kiểm tra luồng nhiều bước.	test/e2e/f1…f4-f7: đặt → trả tiền → check-in → trả kho.	pnpm --filter @vaultra/api test:e2e (cần MySQL).	Mọi check() in ✓.
Regression Testing	Chạy lại test sau khi sửa để chắc không làm hỏng chỗ cũ.	Sau mỗi lần sửa lỗi/thêm tính năng.	Sau khi bỏ payOS, chạy lại unit + e2e F1/F2.	Chạy lại toàn bộ bộ test.	Kết quả không kém lần trước.
Negative Testing	Cố tình nhập sai/thao tác sai.	Mọi form, mọi API có ràng buộc.	Đặt kho ngày quá khứ; gia hạn khi đang nợ.	Gửi dữ liệu sai qua giao diện và Postman.	400/409 với mã lỗi đúng, dữ liệu không đổi.
Boundary Value Analysis	Kiểm tra giá trị ở biên.	Tham số có ngưỡng.	Ngày bắt đầu = hôm nay + 60 (hợp lệ) và + 61 (không); báo trước trả kho đúng 7 ngày.	Chọn giá trị biên, biên−1, biên+1.	Biên hợp lệ chấp nhận, ngoài biên từ chối.
Equivalence Partitioning	Chia đầu vào thành nhóm tương đương, thử một đại diện mỗi nhóm.	Nhiều giá trị đầu vào.	Bậc hủy: ≥7 ngày / 2–6 ngày / <48 giờ.	Mỗi nhóm thử một ngày đại diện.	Số tiền hoàn đúng bậc.
Validation Testing	Kiểm tra ràng buộc định dạng dữ liệu.	Form và body API.	Email sai định dạng, OTP không đủ 6 số, thời hạn gia hạn không thuộc danh sách.	Gửi body thiếu/sai trường.	400 VALIDATION_FAILED, details theo trường.

Loại	Khái niệm	Mục đích / khi dùng	Ví dụ trong VAULTRA	Cách làm	Kết quả mong đợi
Authorization Testing	Kiểm tra quyền và phạm vi dữ liệu.	Mọi endpoint có @RequirePermission.	Khách A xem hợp đồng khách B; nhân viên Cầu Giấy xem đơn Quận 7.	Dùng token vai trò khác nhau.	403 (thiếu quyền) hoặc 404 (ngoài phạm vi).
Security Testing	Tìm lỗ hổng: giả mạo, tiêm lệnh, lộ dữ liệu.	Đăng nhập, thanh toán, tải tệp.	IPN giả chữ ký; tải tệp .exe đổi đuôi .png; dò mật khẩu.	test/e2e/vnpay.ts, audit-findings.ts.	Bị từ chối; tài khoản khóa sau nhiều lần sai.
API Testing	Kiểm tra từng endpoint: mã trạng thái, body, header.	Khi phát triển API.	POST /reservations trả 201; thiếu token trả 401.	Postman (mục 4.4).	Đúng mã, đúng định dạng lỗi.
Database Consistency Testing	Kiểm tra dữ liệu trong DB khớp nghiệp vụ.	Sau thao tác tiền/trạng thái.	Tổng payment_allocations = amount_paid của hóa đơn.	Chạy SQL mục 4.5.	Không có bản ghi lệch.
Performance Testing	Đo tốc độ và khả năng chịu tải.	Trước khi demo, khi nghi chậm.	100 request đặt kho đồng thời (nfr03-concurrency.ts).	Chạy test, đo thời gian; đo API bằng curl -w.	100 request ≈ 1 s, 1 thành công 99 hết chỗ (đã chạy).

4.2 Test case cho 7 Flow

Cột **Actual** để trống và **Status** = Not Tested: nhóm tự điền sau khi chạy. Severity: Critical (mất tiền/sai dữ liệu/bảo mật), High (sai nghiệp vụ chính), Medium (sai quy tắc phụ), Low (giao diện/thông báo). Mỗi Flow phủ: happy path, negative, invalid input, boundary, permission, duplicate request, database consistency, error handling, state transition, unauthorized access.

Test case Flow 1 — Đặt kho (12 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F1-01	Happy path: đặt kho và thanh toán	Khách đã đăng nhập, < 3 đơn chờ	Chọn loại kho → ngày → 3 tháng → Đặt → VNPay thẻ NCB	customer@vaultra.test, thẻ NCB test	Đơn CONFIRMED (hoặc UNIT_ASSIGNED), hóa đơn PAID, deposits HELD		Not Tested	**Critical**
TC-F1-02	Ngày bắt đầu ở quá khứ	Đăng nhập	POST /quotes với startDate = hôm qua	startDate quá khứ	400 VALIDATION_FAILED, không tạo quote		Not Tested	**High**
TC-F1-03	Biên đặt trước 60/61 ngày	Đăng nhập	Báo giá với start = hôm nay+60, rồi +61	+60, +61	+60 chấp nhận; +61 bị từ chối		Not Tested	**Medium**
TC-F1-04	Thời hạn không cho phép	Đăng nhập	POST /quotes termMonths = 2	termMonths 2	400, chỉ nhận 1/3/6/12		Not Tested	**Medium**
TC-F1-05	Vượt 3 đơn chờ thanh toán	Khách có 3 đơn PENDING_PAYMENT	Tạo đơn thứ 4	4 quote	Từ chối, thông báo giới hạn 3 đơn		Not Tested	**High**
TC-F1-06	Bấm đặt hai lần cùng Idempotency-Key	Có quote	Gửi 2 lần POST /reservations cùng key	cùng key + body	Chỉ 1 đơn; lần 2 trả lại kết quả lần 1		Not Tested	**Critical**
TC-F1-07	Cùng key, khác nội dung	Có 2 quote	Gửi POST /reservations key K với quote A rồi quote B	key K	409 IDEMPOTENCY_CONFLICT		Not Tested	**High**
TC-F1-08	Đặt kho cuối cùng đồng thời	Loại kho còn 1	nfr03-concurrency.ts: 100 request	100 khách	1 thành công, 99 UNIT_TYPE_SOLD_OUT; không âm kho		Not Tested	**Critical**

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F1-09	IPN giả chữ ký	Có payment PENDING	Gọi /payments/vnpay/ipn với vnp_SecureHash sai	hash = 000…	RspCode 97/INVALID_SIGNATURE; đơn vẫn PENDING_PAYMENT		Not Tested	**Critical**
TC-F1-10	IPN thiếu tiền	Có payment PENDING	IPN chữ ký đúng, vnp_Amount nhỏ hơn	amount 1.000 ₫	RspCode 04 (Invalid amount); đơn không được xác nhận		Not Tested	**Critical**
TC-F1-11	Khách xem đơn của khách khác	Hai tài khoản khách	GET /reservations/:id của người khác	id đơn khác	404 NOT_FOUND		Not Tested	**High**
TC-F1-12	Hủy và xem tiền hoàn theo bậc	Đơn CONFIRMED, start sau ≥ 7 ngày	GET cancellation-preview → cancel	đơn hợp lệ	Hoàn 100% (trừ phí quản trị), kho được giải phóng		Not Tested	**High**

Test case Flow 2 — Check-in và bàn giao (11 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F2-01	Happy path check-in 4 bước	Đơn MOVE_IN_SCHEDULED trong cửa sổ	Scan → xác minh → gán kho → ký → biên bản 4 ảnh → cấp PIN → hoàn tất	staff.hn, 4 ảnh	Contract ACTIVE, unit OCCUPIED, reservation CHECKED_IN, credential ACTIVE		Not Tested	**Critical**
TC-F2-02	Cấp PIN khi chưa ký	Phiên đang mở, chưa ký	POST /checkin-sessions/:id/issue-access	—	409 CHECKIN_PRECONDITION_FAILED, missing [CONTRACT_NOT_SIGNED]		Not Tested	**Critical**
TC-F2-03	Hoàn tất khi thiếu biên bản	Đã ký, chưa có biên bản	POST …/complete	—	409, danh sách bước còn thiếu		Not Tested	**High**
TC-F2-04	Biên bản dưới 4 ảnh	Đến bước 4	Lưu biên bản 3 ảnh	3 ảnh	Từ chối, yêu cầu đủ ảnh		Not Tested	**Medium**
TC-F2-05	Ảnh chưa tải xong	Ảnh media PENDING	Lưu biên bản ngay khi upload chưa complete	ảnh PENDING	"Tệp chưa tải lên xong", không lưu		Not Tested	**Medium**
TC-F2-06	Tệp giả ảnh	Đến bước 4	Tải tệp .exe đổi đuôi .png	file giả	complete từ chối: nội dung không khớp định dạng		Not Tested	**High**
TC-F2-07	Nhân viên cơ sở khác	staff Quận 7	Bắt đầu phiên cho đơn Cầu Giấy	đơn khác cơ sở	404		Not Tested	**High**
TC-F2-08	OTP ký sai	Đã gửi OTP	POST …/sign với OTP sai	000000	Từ chối, hợp đồng vẫn PENDING_SIGNATURE		Not Tested	**High**
TC-F2-09	Hủy phiên	Phiên IN_PROGRESS	POST …/abort lý do CUSTOMER_LEFT	—	Phiên ABORTED; đơn về MOVE_IN_SCHEDULED; không có credential		Not Tested	**Medium**
TC-F2-10	Đổi kho rẻ hơn	Bước gán kho	Đổi sang loại kho giá thấp hơn	kho rẻ	Tạo hoàn tiền mặt phần chênh; allocation cũ RELEASED		Not Tested	**High**
TC-F2-11	Toàn vẹn PDF	Hợp đồng đã ký	Tải PDF, tính SHA-256, so với contract_documents.sha256	PDF	Hai mã băm giống nhau		Not Tested	**Medium**

Test case Flow 3 — Kho đang thuê và trả kho (11 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F3-01	Xem PIN	Hợp đồng ACTIVE có PIN	Tab Truy cập	khách chủ HĐ	Hiện PIN 6 số; ghi audit access.pin_viewed		Not Tested	**High**
TC-F3-02	Đổi PIN	ACTIVE	Bấm Đổi PIN	—	PIN cũ REVOKED, PIN mới ACTIVE		Not Tested	**High**

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F3-03	Mở/đóng kho từ xa	ACTIVE, không nợ	Mở kho → Đóng kho	—	Trạng thái OPEN → CLOSED; access_events UNIT_OPEN/UNIT_CLOSE		Not Tested	**Medium**
TC-F3-04	Mở kho khi đang bị khóa do nợ	delinquency LOCKED_OUT	remote-unlock	—	Từ chối "tạm khóa do hóa đơn quá hạn"		Not Tested	**High**
TC-F3-05	Trả nhiều hóa đơn một lần	2 hóa đơn ISSUED	POST /invoices/pay 2 id	2 hóa đơn	Phân bổ hạn cũ trước; cả hai PAID		Not Tested	**High**
TC-F3-06	Phụ phí ngoài danh mục	Staff	POST /contracts/:id/charges feeTypeId không tồn tại	id sai	404/400, không tạo hóa đơn		Not Tested	**Medium**
TC-F3-07	Trả kho báo trước < 7 ngày	Khách	Yêu cầu trả kho ngày mai	hôm nay+1	Từ chối, nêu ngày sớm nhất		Not Tested	**Medium**
TC-F3-08	Biên 7 ngày	Khách	Yêu cầu trả kho hôm nay+7	+7	Chấp nhận; ước tính hiển thị		Not Tested	**Medium**
TC-F3-09	Quyết toán đúng công thức	Cọc 690.000, nợ 50.000	Kiểm tra → chốt	không hư hại	net = 640.000; hoàn tiền chờ duyệt		Not Tested	**Critical**
TC-F3-10	Kho về Trống khi còn PIN	Chưa chốt kiểm tra	POST /units/:id/final-check	—	Từ chối vì còn credential ACTIVE		Not Tested	**High**
TC-F3-11	Hủy trả kho đúng ngày	Ngày trả = hôm nay	POST /move-out-requests/:id/cancel	—	"Đã đến ngày trả kho, không thể hủy"		Not Tested	**Low**

Test case Flow 4 — Chính sách, phí, doanh thu (10 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F4-01	Tạo và xuất bản phiên bản	BOM	Tạo bản mới → sửa grace_days → xuất bản	grace_days 7	Bản mới ACTIVE/SCHEDULED; bản cũ SUPERSEDED khi kích hoạt		Not Tested	**High**
TC-F4-02	Sai thứ tự mốc quá hạn	BOM, bản DRAFT	grace_days = 12 > lockout_day 10	12	400 POLICY_INVALID, chỉ ra trường lỗi		Not Tested	**High**
TC-F4-03	Xem trước tác động	Bản DRAFT	GET /policy-versions/:id/impact	—	Số hợp đồng/đơn chịu ảnh hưởng + diff tham số		Not Tested	**Medium**
TC-F4-04	FM sửa chính sách	fm.hn	PATCH /policy-versions/:id	—	403 FORBIDDEN		Not Tested	**High**
TC-F4-05	Hợp đồng cũ không đổi	Có HĐ ACTIVE	Xuất bản bản mới; xem HĐ cũ	—	HĐ vẫn policy_version_id cũ		Not Tested	**Critical**
TC-F4-06	Giá ngoài khung	FM	Đặt giá loại kho > max khung	giá vượt	Tạo price_change_request PENDING, giá chưa đổi		Not Tested	**Medium**
TC-F4-07	Khuyến mãi hết hạn	Có mã hết hạn	POST /promotions/validate	mã cũ	PROMO_INVALID		Not Tested	**Medium**
TC-F4-08	Miễn giảm vượt hạn mức FM	Đề xuất 2.000.000 ₫	FM duyệt	2.000.000	Chuyển BOM duyệt		Not Tested	**High**
TC-F4-09	Đối soát khớp	Ngày có giao dịch VNPay	POST /reconciliations provider VNPAY	ngày 08/11	Lô MATCHED, cổng = hệ thống		Not Tested	**Medium**
TC-F4-10	Đối soát có chênh lệch	Như trên	Dán sao kê với 1 dòng số tiền khác	mã,số_tiền	HAS_MISMATCH; dòng AMOUNT_MISMATCH có nút giải trình		Not Tested	**Medium**

Test case Flow 5 — Kho và nhân sự (10 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F5-01	Tạo kho hàng loạt	FM	Tiền tố A-, dải 41–50	A-041…A-050	10 kho AVAILABLE trong 1 giao dịch		Not Tested	**Medium**

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F5-02	Trùng mã kho	Đã có A-041	Tạo lại A-041	A-041	UNIT_CODE_DUPLICATE, không tạo kho nào		Not Tested	**Medium**
TC-F5-03	Chuyển trạng thái hợp lệ	Kho AVAILABLE	POST /units/:id/status MAINTENANCE + lý do	lý do	Thành công; ghi unit_status_history		Not Tested	**Medium**
TC-F5-04	Chuyển trạng thái sai	Kho OCCUPIED	Đổi sang AVAILABLE	—	409 INVALID_STATUS_TRANSITION		Not Tested	**High**
TC-F5-05	Gợi ý phân bổ	Đơn CONFIRMED	GET allocation-suggestions	—	Top 3 kho đúng loại, AVAILABLE, kèm lý do		Not Tested	**Medium**
TC-F5-06	Hai allocation mở cùng kho	Kho đã có allocation ACTIVE	Allocate đơn khác vào kho đó	—	Bị chặn (UNIQUE open_unit_key / UNIT_HAS_OPEN_ALLOCATION)		Not Tested	**Critical**
TC-F5-07	Ca chồng nhau	NV đã có ca 07–15	Tạo ca 14–22 cùng ngày	—	SHIFT_OVERLAP		Not Tested	**Medium**
TC-F5-08	Giao việc cho người ngoài cơ sở	FM Cầu Giấy	Assign cho NV Quận 7	—	ASSIGNEE_NOT_IN_FACILITY		Not Tested	**Medium**
TC-F5-09	Nhân viên sửa việc người khác	Việc giao cho NV A	NV B bấm hoàn thành	—	Từ chối		Not Tested	**High**
TC-F5-10	Realtime sơ đồ/việc	Hai cửa sổ FM, NV	FM tạo việc	—	Cửa sổ NV tự hiện việc mới (không F5)		Not Tested	**Low**

Test case Flow 6 — Gia hạn và quá hạn (10 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F6-01	Gia hạn happy path	HĐ ACTIVE trong cửa sổ, không nợ	Gửi OTP → nhập → Chấp nhận → thanh toán	OTP từ email	current_end_date dời; contract_terms thêm kỳ		Not Tested	**High**
TC-F6-02	Gia hạn khi đang nợ	Có hóa đơn quá hạn	POST /contracts/:id/renew	—	409 RENEWAL_BLOCKED_OUTSTANDING_BALANCE		Not Tested	**High**
TC-F6-03	OTP sai	Đã gửi OTP	Nhập OTP sai	000000	Từ chối; đếm số lần thử		Not Tested	**Medium**
TC-F6-04	OTP cho thời hạn khác	OTP gửi cho 1 tháng	Gia hạn 3 tháng với OTP đó	—	"OTP … không khớp thời hạn đã chọn"		Not Tested	**Medium**
TC-F6-05	Thời hạn không hợp lệ	—	termMonths = 5	5	400 "Thời hạn gia hạn không hợp lệ"		Not Tested	**Low**
TC-F6-06	Mốc quá hạn D+6 (bản Node, đồng hồ thử)	Hóa đơn quá hạn 6 ngày	Chạy job delinquency.run	test clock	LATE, thêm dòng LATE_FEE đúng công thức		Not Tested	**High**
TC-F6-07	Chạy job hai lần	Như TC-F6-06	Chạy delinquency.run lần 2	—	Không cộng phí trễ lần 2		Not Tested	**Critical**
TC-F6-08	D+10 khóa truy cập	Quá hạn 10 ngày	Chạy job	—	Credential SUSPENDED; task OVERLOCK; PIN bị từ chối		Not Tested	**High**
TC-F6-09	Trả đủ sau khi bị khóa	LOCKED_OUT	Thanh toán đủ nợ + phí	—	CURRENT; credential ACTIVE; task REMOVE_OVERLOCK		Not Tested	**Critical**
TC-F6-10	FM tự chấm dứt không qua BOM	DEFAULT_PENDING	FM gọi termination-requests/:id/decide	—	403 (chỉ BOM duyệt)		Not Tested	**High**

Test case Flow 7 — Hỗ trợ (10 ca)

ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Actual	Status	Severity
TC-F7-01	Tạo ticket happy path	Khách có hợp đồng	Tạo ticket ACCESS_CODE kèm ảnh	mô tả + 1 ảnh	Ticket NEW, mã SR-..., hạn SLA 30 phút/4 giờ		Not Tested	**High**
TC-F7-02	Gắn hợp đồng người khác	Khách A	POST /tickets contractId của khách B	id khác	404		Not Tested	**High**
TC-F7-03	Quá 5 tệp đính kèm	—	Tạo ticket 6 tệp	6 ảnh	Từ chối		Not Tested	**Low**
TC-F7-04	Ghi chú nội bộ bị ẩn	NV thêm ghi chú nội bộ	Khách GET /tickets/:id	—	Không có tin is_internal		Not Tested	**Critical**
TC-F7-05	Giải quyết thiếu nguyên nhân	IN_PROGRESS	resolve không có resolution code	—	400		Not Tested	**Medium**
TC-F7-06	Đánh giá ngoài 1–5	RESOLVED	rate = 6	6	400		Not Tested	**Low**
TC-F7-07	Mở lại sau 7 ngày	RESOLVED > 7 ngày	reopen	—	Từ chối		Not Tested	**Low**
TC-F7-08	Khách gọi hành động của nhân viên	Token khách	POST /tickets/:id/actions/refund	—	403		Not Tested	**High**
TC-F7-09	SLA ngoài giờ	Tạo ticket 21:00	Xem first_response_due_at	—	Hạn tính từ giờ mở cửa hôm sau (trừ URGENT)		Not Tested	**Medium**
TC-F7-10	Cấp lại PIN trong ticket	Ticket ACCESS_CODE, HĐ không nợ	actions/reissue-pin	—	PIN mới ACTIVE; ticket ghi sự kiện		Not Tested	**Medium**

4.3 Kiểm thử nghiệp vụ chuyên sâu

Nghiệp vụ	Đầu vào / quy tắc	Hợp lệ	Không hợp lệ / biên	Kết quả mong đợi	Dữ liệu DB cần kiểm
Khả dụng loại kho	available = sellable − commitments trong [s, e)	Còn 1 kho, đặt 1	Còn 0; đơn PENDING chưa hết giờ giữ vẫn tính	Không bao giờ âm	reservations, contracts, units theo unit_type_id
Báo giá	Kỳ đầu + cọc 1 tháng + phí quản trị; VAT gồm trong giá (prices_include_vat)	Mã giảm % có trần	Mã hết lượt, sai cơ sở	Tổng = Σ dòng, làm tròn đồng	quotes.line_items, due_now_total
Thanh toán	IPN có chữ ký, đúng số tiền, chưa xử lý	IPN hợp lệ	Sai chữ ký; thiếu tiền; gửi 2 lần	Chỉ ghi nhận 1 lần	payments, payment_allocations, payment_webhook_events
Hủy đơn	Bậc hủy theo số giờ đến start_date	≥ 7 ngày	Đúng 48 giờ (biên)	Hoàn đúng bậc; phí quản trị không hoàn	refunds, deposits.forfeited_amount
Cấp mã truy cập	SIGNED + hóa đơn nhận kho PAID	Đủ điều kiện	Thiếu một điều kiện	409 kèm missing[]	access_credentials, contracts.status
Quyết toán cọc	net = cọc − nợ − hư hại − vệ sinh − phí trả sớm + hoàn trả trước	net > 0	net < 0; net = 0	Dương → refund; âm → hóa đơn SETTLEMENT	deposit_settlements, refunds, invoices
Phí trễ	GREATER_OF(50.000; 10% tiền thuê), trần 500.000	Tiền thuê 290.000 → 50.000	Tiền thuê 6.000.000 → 500.000 (trần)	Mỗi hóa đơn 1 lần	invoice_items (LATE_FEE)

Nghiệp vụ	Đầu vào / quy tắc	Hợp lệ	Không hợp lệ / biên	Kết quả mong đợi	Dữ liệu DB cần kiểm
Mốc quá hạn	D+6 LATE, D+10 LOCKED_OUT, D+30, D+45	Ngày 10 đúng mốc	Ngày 9 (chưa khóa)	Chuyển đúng giai đoạn, không nhảy cóc	delinquency_events, contracts.delinquency_status
Gia hạn	Giá tăng ≤ 10%, không nợ, có offer	Không nợ	Đang nợ; OTP sai thời hạn	Kỳ mới liền mạch	contract_terms, renewal_offers
SLA	Giờ làm việc cơ sở; URGENT 24/7	Tạo trong giờ	Tạo 21:00, Chủ nhật	Hạn đúng	support_tickets.first_response_due_at

Các điểm phải kiểm chéo frontend – backend – database

- Số tiền hiển thị trên giao diện = tổng dòng hóa đơn trong DB = số tiền trên PDF hóa đơn.
- Trạng thái hiển thị (badge) = cột status trong DB = bước tiếp theo được phép theo states.ts.
- Thao tác bị chặn ở giao diện vẫn phải bị chặn khi gọi thẳng API (Postman với token vai trò khác).
- Gửi lại cùng request (F5 trang thanh toán, bấm hai lần) không tạo bản ghi thứ hai.

4.4 Kiểm thử API bằng Postman

1. **Xác định API cần test**: tra danh sách ở Chương 3 (mục 13 mỗi Flow) hoặc mở tài liệu OpenAPI của bản Node: `GET /v1/openapi.json`, `GET /v1/docs`.
2. **Base URL**: tạo Environment với biến `baseUrl` = `https://vaultra-gateway.zerngulon.workers.dev/v1` (bản demo) hoặc `http://localhost:4000/v1` (bản Node).
3. **Đăng nhập**: `POST {{baseUrl}}/auth/login`, Body raw JSON `{"email":"customer@vaultra.test","password":"<mật khẩu demo>"}`. Bản demo giới hạn 10 lần đăng nhập/phút/IP.
4. **Lấy token**: tab Tests của request login: `pm.environment.set('token', pm.response.json().accessToken);`
5. **Authorization**: ở Collection chọn Bearer Token = `{{token}}`. Access token ngắn hạn; hết hạn thì đăng nhập lại.
6. **Các phương thức**: GET `/contracts` · POST `/tickets` · PUT `/me/notification-preferences` · PATCH `/contracts/:id/auto-renew` · DELETE `/media/:id`.
7. **Kiểm Status Code**: Tests → `pm.test("201", () => pm.response.to.have.status(201));`
8. **Kiểm Response Body**: `pm.expect(pm.response.json().status).to.eql("PENDING_PAYMENT");`
9. **Kiểm dữ liệu trả về**: so trường với DB (mục 4.5); id trả về là public_id (ULID 26 ký tự).
10. **Dữ liệu không hợp lệ**: bỏ trường bắt buộc → mong đợi 400, `error.code = "VALIDATION_FAILED"`, `error.details[]` liệt kê trường.
11. **Không có token**: tắt Authorization → 401 UNAUTHENTICATED.
12. **Sai quyền**: đăng nhập customer rồi gọi `POST /policy-versions/:id/publish` → 403; gọi `GET /contracts/:id` của khách khác → 404.
13. **Lỗi backend**: lỗi 5xx trả `{"error":{"code":"INTERNAL_ERROR","message":"Lỗi hệ thống","requestId":…}}`; ghi lại requestId để tra log (wrangler tail hoặc log Node).
14. **Request trùng lặp**: thêm Header `Idempotency-Key: {{$guid}}` cố định cho 2 lần gửi `POST /reservations` → lần 2 trả lại kết quả lần 1; đổi body giữ key → 409 IDEMPOTENCY_CONFLICT.

```http
# Bộ request mẫu cho Flow 1 (thay {{...}} bằng biến Postman)
POST {{baseUrl}}/auth/login
{"email":"customer@vaultra.test","password":"{{demoPassword}}"}

GET  {{baseUrl}}/catalog/facilities
GET  {{baseUrl}}/catalog/facilities/{{slug}}          # lấy unitTypes[].id
POST {{baseUrl}}/quotes
{"unitTypeId":"{{unitTypeId}}","startDate":"2026-11-20","termMonths":3,"billingMode":"MONTHLY"}
```

```
POST {{baseUrl}}/reservations          Header: Idempotency-Key: {{key}}
{"quoteId":"{{quoteId}}"}
POST {{baseUrl}}/reservations/{{rid}}/pay   Header: Idempotency-Key: {{key2}}
# → redirectUrl (trang VNPay sandbox)
GET  {{baseUrl}}/reservations/{{rid}}
```

4.5 Kiểm tra database

Kết nối: bản Node dùng MySQL theo DATABASE_URL trong apps/api/.env (ví dụ `mysql://root:@127.0.0.1:3306/vaultra`); dùng MySQL Workbench/DBeaver hoặc `npx prisma studio` trong apps/api. Bản demo Worker không có MySQL: dữ liệu xem qua API hoặc tải snapshot `wrangler r2 object get vaultra/_db/vaultra-diff-v1.json --remote`. Các câu SQL dưới đây chạy trên bản Node/MySQL.

SQL

```sql
-- 1. Đơn đặt thuê và lịch sử trạng thái (F1)
SELECT r.code, r.status, r.start_date, r.end_date, r.hold_expires_at, q.due_now_total
FROM reservations r JOIN quotes q ON q.id = r.quote_id
WHERE r.code = 'RSV-2026-000105';
SELECT from_status, to_status, reason, created_at FROM reservation_status_history
WHERE reservation_id = (SELECT id FROM reservations WHERE code = 'RSV-2026-000105') ORDER BY id;

-- 2. Không có kho nào có hai allocation mở (phải trả về 0 dòng)
SELECT unit_id, COUNT(*) AS open_cnt FROM unit_allocations
WHERE status = 'ACTIVE' GROUP BY unit_id HAVING COUNT(*) > 1;

-- 3. Tiền phân bổ khớp số đã trả của hóa đơn (phải trả về 0 dòng)
SELECT i.code, i.amount_paid, COALESCE(SUM(pa.amount),0) AS allocated
FROM invoices i LEFT JOIN payment_allocations pa ON pa.invoice_id = i.id
GROUP BY i.id HAVING i.amount_paid <> COALESCE(SUM(pa.amount),0);

-- 4. Ràng buộc tổng hóa đơn (phải trả về 0 dòng)
SELECT code, subtotal, discount_total, tax_total, total FROM invoices
WHERE total <> subtotal - discount_total + tax_total;

-- 5. Hợp đồng ACTIVE phải có kho OCCUPIED/OVERLOCKED và ít nhất 1 credential (F2)
SELECT c.code, u.code AS unit, u.status,
  (SELECT COUNT(*) FROM access_credentials a WHERE a.contract_id = c.id AND a.status = 'ACTIVE') AS active_creds
FROM contracts c JOIN units u ON u.id = c.unit_id WHERE c.status = 'ACTIVE';

-- 6. Hợp đồng đã ký có tài liệu và mã băm (F2)
SELECT c.code, d.type, d.sha256, s.signed_at, s.ip_address
FROM contracts c JOIN contract_documents d ON d.contract_id = c.id
LEFT JOIN contract_signatures s ON s.document_id = d.id WHERE c.code = 'CT-2026-000062';

-- 7. Quyết toán cọc (F3)
SELECT c.code, s.deposit_amount, s.outstanding_applied, s.damage_total, s.cleaning_total,
       s.prepaid_refund, s.net_amount, s.status, s.dispute_deadline
FROM deposit_settlements s JOIN contracts c ON c.id = s.contract_id ORDER BY s.id DESC LIMIT 5;

-- 8. Phiên bản chính sách đang hiệu lực (F4)
SELECT p.code, v.version_no, v.status, v.effective_from, v.change_reason
FROM policy_versions v JOIN policies p ON p.id = v.policy_id ORDER BY v.id DESC;

-- 9. Lịch sử chuyển trạng thái kho (F5)
SELECT u.code, h.from_status, h.to_status, h.reason, h.created_at
FROM unit_status_history h JOIN units u ON u.id = h.unit_id ORDER BY h.id DESC LIMIT 20;

-- 10. Chuỗi quá hạn (F6)
SELECT c.code, c.delinquency_status, e.from_stage, e.to_stage, e.days_overdue, e.created_at
FROM delinquency_events e JOIN contracts c ON c.id = e.contract_id ORDER BY e.id DESC LIMIT 20;

-- 11. Ticket, SLA, sự kiện (F7)
SELECT code, status, priority, first_response_due_at, first_response_at, resolution_due_at, sla_breached
FROM support_tickets ORDER BY id DESC LIMIT 10;

-- 12. Phát hiện trùng: hai PIN đang hoạt động trùng nhau trong một cơ sở (phải 0 dòng)
SELECT facility_id, pin_lookup_hash, COUNT(*) FROM access_credentials
```

```
WHERE status IN ('ACTIVE','SUSPENDED') AND type = 'PIN_CODE'
GROUP BY facility_id, pin_lookup_hash HAVING COUNT(*) > 1;

-- 13. Email không gửi được
SELECT d.id, d.status, d.recipient, d.subject, d.attempts, d.last_error
FROM notification_deliveries d WHERE d.status IN ('QUEUED','FAILED') ORDER BY d.id DESC LIMIT 20;

-- 14. Nhật ký hành động của một người
SELECT occurred_at, action, entity_type, entity_ref, reason
FROM audit_logs WHERE actor_user_id = (SELECT id FROM users WHERE email = 'fm.hn@vaultra.test')
ORDER BY id DESC LIMIT 30;
```

Kiểm tra transaction

Cách đơn giản để thấy tính nguyên tử: chạy TC-F2-09 (hủy phiên check-in giữa chừng) rồi chạy SQL số 5 — không được có hợp đồng ACTIVE thiếu credential hoặc kho vẫn RESERVED mà reservation đã CHECKED_IN. Kiểm tra khóa ngoại: SHOW CREATE TABLE unit_allocations; xem FOREIGN KEY và UNIQUE(open_unit_key). Tên cột trong các câu SQL đã đối chiếu với schema.prisma ngày 08/11/2026; SQL chưa được chạy trên MySQL trong lúc soạn tài liệu.

4.6 Kiểm thử tự động có trong repo

Bộ test	Framework / cách chạy	Nội dung	Kết quả gần nhất
test/unit/shared.test.ts	node:test qua tsx · `pnpm --filter @vaultra/api test`	16 test: tư vấn kích thước, 8 máy trạng thái, schema chính sách, giá & khuyến mãi, phí trễ & mốc quá hạn, giá gia hạn, quyết toán, SLA, gộp quyền RBAC, mã hóa AES-256-GCM	**16/16 đạt** 08/11/2026
test/audit/audit.unit.ts	`pnpm --filter @vaultra/api test:audit`	5 test thuật toán tiền/ngày	**5/5 đạt** 08/11/2026
test/e2e/f1-reservation.ts	tsx, gọi API thật · `pnpm … test:e2e` (cần MySQL + API đang chạy)	Đặt kho, thanh toán, idempotency	10/12 trên Worker cục bộ 08/11 (2 lỗi do dữ liệu cục bộ, chưa xác minh)
test/e2e/nfr03-concurrency.ts	như trên	100 request đồng thời cho kho cuối	**5/5 đạt** (1 ok, 99 sold-out, 970 ms)
test/e2e/f2-checkin.ts	như trên	Lịch hẹn, check-in, ký OTP, biên bản, kích hoạt	17/24 trên Worker (OTP đọc từ hộp thư giả của bản Node)
test/e2e/f3-moveout.ts	như trên	Trả kho, kiểm tra, quyết toán	**15/15 đạt**
test/e2e/f4-f7-lifecycle.ts	như trên	Chính sách, đối soát, ca/việc/bảo trì, quá hạn, gia hạn, ticket	**Chưa chạy lại** (bị giới hạn đăng nhập 429 trên Worker)
test/e2e/rbac.ts	như trên	401/403/404/409 theo vai trò và phạm vi	**Chưa chạy lại** (429)
test/e2e/vnpay.ts	như trên, cần VNPAY_HASH_SECRET	URL thanh toán, chữ ký IPN/return, số tiền	**Chưa chạy lại** sau khi bỏ payOS
test/audit/audit-findings.ts	tsx · AUDIT_SKIP để bỏ ca	40 kiểm tra lỗi bảo mật/nghiệp vụ đã sửa	Chưa chạy lại sau thay đổi gần nhất

Còn thiếu / nên ưu tiên

- Chạy lại toàn bộ e2e trên bản Node + MySQL (không bị giới hạn đăng nhập), ghi kết quả vào bảng test case.
- Chưa có test tự động cho giao diện (không có Playwright/Cypress trong repo); kiểm thử giao diện hiện là thủ công.
- Chưa có test tự động cho realtime trên Worker và cho bộ gửi email trên Worker (đã kiểm tra thủ công 08/11/2026).
- Ưu tiên: thanh toán (vnpay.ts), quyền (rbac.ts), quá hạn (f4-f7-lifecycle.ts) vì liên quan tiền và bảo mật.

CHƯƠNG 5

Kịch bản demo thực tế

> Thứ tự demo được sắp để dữ liệu của bước trước dùng cho bước sau: đơn đặt ở F1 được check-in ở F2, hợp đồng đó được gia hạn ở F6. Không phải tạo dữ liệu nhiều lần.

5.1 Thứ tự trình bày và demo

#	Người	Nội dung	Phút	Dữ liệu tạo ra / dùng lại
1	Khang	Mở đầu, tổng quan, kiến trúc mức tổng quan	3	—
2	Khang	F1 Đặt kho (demo)	3	Tạo đơn RSV-… (1 tháng, bắt đầu hôm nay)
3	Triển	F2 Check-in (demo)	5	Dùng RSV-… → hợp đồng CT-… ACTIVE
4	Triển	F3 Đang thuê & trả kho (demo)	4	CT-… (PIN, mở/đóng kho) + một hợp đồng khác để trả kho
5	Phúc	F4 Chính sách, phí, doanh thu (demo) + tổng kết kiểm thử	7	Phiên bản chính sách mới, lô đối soát
6	Khang	F7 Hỗ trợ (demo)	2	Ticket SR-… trên CT-…
7	Trường	F5 Kho & nhân sự (demo)	4	Đổi trạng thái kho, việc mới
8	Trường	F6 Gia hạn (demo) & quá hạn (giải thích + bằng chứng test), kết luận	4	Gia hạn CT-… 1 tháng

Cách bố trí màn hình

Máy demo mở 3 cửa sổ trình duyệt: (1) cửa sổ thường — khách; (2) cửa sổ ẩn danh — nhân viên staff.hn; (3) một trình duyệt khác — quản lý fm.hn / bom. Đăng nhập sẵn trước giờ bảo vệ 10 phút. Hộp thư email của tài khoản khách mở ở tab riêng.

5.2 Kịch bản chi tiết từng Flow

F1 — Đặt kho · Khang

1. Người demo	Khang
2. Tài khoản	customer@vaultra.test
3. Role	CUSTOMER
4. Chuẩn bị	Khách có < 3 đơn chờ thanh toán (hủy đơn cũ ở /app/reservations nếu cần). Thẻ NCB test. Chọn thời hạn **1 tháng**, bắt đầu **hôm nay** để hợp đồng nằm trong cửa sổ gia hạn khi Trường demo F6.

5. Các bước	1. Trang chủ → Đặt kho ngay → chọn Vaultra Cầu Giấy (cơ sở của staff.hn). 2. Chọn loại kho còn trống → xem dải "Lịch trống 8 tuần" và bảng giá. 3. Ngày bắt đầu hôm nay, thời hạn 1 tháng, theo tháng. 4. Chọn ngày hẹn và khung giờ còn suất, đồng ý điều khoản. 5. Thanh toán trực tuyến & giữ chỗ → VNPay: 9704198526191432198 · NGUYEN VAN A · 07/15 · OTP 123456. 6. Về trang kết quả → mở đơn ở /app/reservations: ghi lại mã RSV-… cho Triển.
6. Quan sát	Đơn "Đã xác nhận" (hoặc đã gán kho), hóa đơn đã thanh toán, có lịch hẹn; email xác nhận đến hộp thư của tài khoản.
7. Database đối chiếu	reservations (status), invoices (PAID), payments (SUCCEEDED, provider VNPAY), deposits (HELD).
8. Thuyết minh	"*Khách chọn loại kho, hệ thống tính giá từng dòng và giữ chỗ 15 phút. Tiền chỉ được ghi nhận khi VNPay gọi lại máy chủ với chữ ký hợp lệ, không phải khi khách quay về trang.*"
9. Lỗi cần tránh	Chọn ngày bắt đầu xa → hợp đồng không vào cửa sổ gia hạn; quên hủy đơn chờ cũ → bị chặn đơn thứ 4.
10. Khi demo thất bại	VNPay sandbox chậm/lỗi: chuyển sang đơn đã thanh toán sẵn (chuẩn bị từ hôm trước) và giải thích luồng bằng màn hình đơn; hoặc nhân viên ghi nhận tiền mặt ở F2.

F2 — Check-in & bàn giao · Triển

1. Người demo	Triển
2. Tài khoản	staff.hn@vaultra.test (+ khách ở cửa sổ ẩn danh)
3. Role	FACILITY_STAFF
4. Chuẩn bị	Mã RSV-… từ F1. 4 ảnh có sẵn trên máy (cửa, sàn, tường, trần). Hộp thư của khách mở sẵn để ký.
5. Các bước	1. /staff/scan → nhập mã RSV-… → Bắt đầu check-in. 2. Bước 1: loại giấy tờ + 4 số cuối → xác nhận. 3. Bước 2: giữ kho được gán (hoặc minh họa đổi kho). 4. Bước 3: Gửi link ký → khách mở email, ký trên điện thoại/cửa sổ khác → bước 3 tự cập nhật. 5. Bước 4: checklist + tải 4 ảnh → Lưu biên bản → cấp PIN → Hoàn tất. 6. Cửa sổ khách: /app/contracts → tab Truy cập: PIN hiển thị.
6. Quan sát	Hợp đồng "Đang thuê", kho OCCUPIED, PIN 6 số chỉ hiện trên web; xem được PDF hợp đồng ở tab Tài liệu.
7. Database đối chiếu	contracts (ACTIVE), contract_documents (sha256), access_credentials (ACTIVE), handover_records, units (OCCUPIED).
8. Thuyết minh	"*Backend chặn cứng: chưa ký hoặc chưa trả đủ thì không cấp mã — giao diện không quyết định việc này.*"
9. Lỗi cần tránh	Ảnh chưa tải xong đã bấm lưu; quên mở hộp thư khách.
10. Khi demo thất bại	Email ký chậm: dùng ký OTP tại quầy (gửi OTP, khách nhập mã và vẽ chữ ký). Nếu lỗi bước 4: mở hợp đồng đã check-in sẵn để trình bày kết quả.

F3 — Kho đang thuê & trả kho · Triển

1. Người demo	Triển
2. Tài khoản	customer@vaultra.test; staff.hn@vaultra.test
3. Role	CUSTOMER, FACILITY_STAFF

4. Chuẩn bị	Hợp đồng vừa tạo ở F2 (để xem PIN, mở/đóng kho) và một hợp đồng ACTIVE khác của khách (ví dụ CT-2026-000016) để gửi yêu cầu trả kho, tránh ảnh hưởng demo gia hạn của Trường.
5. Các bước	1. Khách: tab Truy cập → Mở kho → trạng thái "Đang mở" → Đóng kho. 2. Tab Hóa đơn: xem hóa đơn, tải PDF. 3. Hợp đồng thứ hai → tab Trả kho → chọn ngày (≥ 7 ngày) → xem ước tính → Gửi. 4. Nhân viên: /staff/moveouts → mở yêu cầu → kiểm tra, (tùy chọn) ghi hư hại có ảnh → chốt; khách ký đồng ý. 5. Quản lý: chốt quyết toán; nhân viên kiểm tra cuối.
6. Quan sát	PIN bị thu hồi sau chốt; kho "Chờ kiểm tra" rồi "Trống"; quyết toán có số tiền hoàn.
7. Database đối chiếu	access_events, move_out_requests, deposit_settlements (net_amount), refunds, units.
8. Thuyết minh	*"Công thức: cọc trừ nợ, trừ hư hại, trừ vệ sinh, cộng tiền trả trước chưa dùng. Khách có 3 ngày khiếu nại."*
9. Lỗi cần tránh	Gửi trả kho trên hợp đồng mà Trường cần để gia hạn.
10. Khi demo thất bại	Nếu chốt lỗi: dùng yêu cầu trả kho đã chuẩn bị ở trạng thái INSPECTED để trình bày quyết toán.

F4 — Chính sách, phí, doanh thu · Phúc

1. Người demo	Phúc
2. Tài khoản	bom@vaultra.test
3. Role	BUSINESS_OPS_MANAGER
4. Chuẩn bị	Biết trước tham số sẽ sửa (ví dụ grace_days 5 → 7). Có ít nhất một ngày có giao dịch VNPay và tiền mặt.
5. Các bước	1. /ops/policies → tạo phiên bản mới → sửa tham số, nhập lý do. 2. Thử nhập sai thứ tự mốc quá hạn để thấy validate. 3. Xem tác động → xuất bản. 4. /ops/finance → Chạy đối soát (VNPay, ngày có giao dịch) → mở lô; chạy thêm với "Tiền mặt / chuyển khoản tại quầy". 5. /ops/compare hoặc /ops/reports: so sánh cơ sở, xuất CSV.
6. Quan sát	Phiên bản mới ở trạng thái mới; lô đối soát Khớp; báo cáo có số liệu.
7. Database đối chiếu	policy_versions, reconciliation_batches, reconciliation_items, audit_logs (lý do thay đổi).
8. Thuyết minh	*"Chính sách là dữ liệu có phiên bản, không hồi tố; hợp đồng giữ snapshot chính sách lúc ký."*
9. Lỗi cần tránh	Xuất bản chính sách sai lên bản demo rồi quên khôi phục.
10. Khi demo thất bại	Nếu xuất bản lỗi: trình bày bản DRAFT và màn hình xem tác động. Sau buổi demo tạo bản mới khôi phục giá trị cũ.

F5 — Kho & nhân sự · Trường

1. Người demo	Trường
2. Tài khoản	fm.hn@vaultra.test (+ staff.hn ở cửa sổ khác)
3. Role	FACILITY_MANAGER
4. Chuẩn bị	Một kho trống để đổi trạng thái; nhân viên đang đăng nhập ở cửa sổ khác để thấy realtime.

5. Các bước	1. /manage/floor → giới thiệu màu trạng thái, chỉ số lấp đầy. 2. Chọn kho trống → đổi sang Bảo trì kèm lý do → xem lịch sử. 3. /manage/tasks → tạo việc, giao cho nhân viên → cửa sổ nhân viên tự hiện. 4. Nhân viên nhận → bắt đầu → hoàn thành. 5. /manage/schedule → ca làm việc.
6. Quan sát	Màu kho đổi; việc mới hiện bên nhân viên không cần F5; lịch sử ghi lý do.
7. Database đối chiếu	units, unit_status_history, work_tasks, task_events.
8. Thuyết minh	"*Mọi đổi trạng thái kho đi qua máy trạng thái và ghi lịch sử; chuyển sai bị từ chối.*"
9. Lỗi cần tránh	Đổi trạng thái kho đang có khách (sẽ bị chặn — có thể dùng làm ví dụ negative).
10. Khi demo thất bại	Nếu realtime chậm: bấm tải lại và giải thích cơ chế.

F6 — Gia hạn & quá hạn · Trường

1. Người demo	Trường
2. Tài khoản	customer@vaultra.test; fm.hn@vaultra.test
3. Role	CUSTOMER, FACILITY_MANAGER
4. Chuẩn bị	Hợp đồng 1 tháng vừa tạo (trong cửa sổ gia hạn) — kiểm tra trước ngày bảo vệ rằng tab Gia hạn mở được. Hộp thư khách mở sẵn để lấy OTP.
5. Các bước	1. Khách: hợp đồng → tab Gia hạn → chọn 1 tháng → Gửi mã. 2. Nhập OTP từ email → Chấp nhận & gia hạn → thanh toán hóa đơn gia hạn. 3. Ngày hết hạn mới hiện trên hợp đồng. 4. FM: /manage/delinquency → giải thích các giai đoạn và thao tác ghi chú thu nợ. 5. Mở bảng mốc D+1…D+45 (slide) và kết quả unit test làm bằng chứng.
6. Quan sát	Hợp đồng có ngày hết hạn mới; màn hình công nợ hiển thị danh sách theo giai đoạn.
7. Database đối chiếu	renewal_offers, contract_terms, contracts.current_end_date, invoices (RENEWAL).
8. Thuyết minh	"*Chấm dứt do nợ không bao giờ tự động: hệ thống đề xuất, FM xác nhận, BOM duyệt.*"
9. Lỗi cần tránh	Hứa sẽ cho xem hợp đồng tự chuyển LATE/LOCKED_OUT trên bản demo — bản deploy không tua thời gian được.
10. Khi demo thất bại	OTP không đến: kiểm tra spam; dùng hợp đồng đã gia hạn sẵn để trình bày kết quả.

F7 — Hỗ trợ · Khang

1. Người demo	Khang
2. Tài khoản	customer@vaultra.test; staff.hn@vaultra.test
3. Role	CUSTOMER, FACILITY_STAFF
4. Chuẩn bị	Một ảnh minh họa sự cố. Hai cửa sổ: khách và nhân viên.
5. Các bước	1. Khách: /app/support → Tạo yêu cầu → chọn hợp đồng, danh mục Mã truy cập, mô tả, ảnh. 2. Chỉ ra mã SR-… và hạn SLA. 3. Nhân viên: /staff/tickets → mở ticket → Nhận việc → trả lời; thêm ghi chú nội bộ. 4. Cửa sổ khách: thấy câu trả lời, không thấy ghi chú nội bộ. 5. Nhân viên: Đánh dấu đã giải quyết (nhập nguyên nhân) → khách đánh giá 5 sao.
6. Quan sát	Ticket đi đủ trạng thái, có đánh giá; ghi chú nội bộ bị ẩn với khách.

7. Database đối chiếu	support_tickets, ticket_messages (is_internal), ticket_events, ticket_ratings.
8. Thuyết minh	*"SLA tính theo giờ làm việc; ghi chú nội bộ không bao giờ trả về API của khách."*
9. Lỗi cần tránh	Chọn nhầm cơ sở khác với nhân viên đang đăng nhập.
10. Khi demo thất bại	Nếu tạo ticket lỗi: mở ticket mẫu có sẵn của khách để trình bày.

5.3 Checklist trước ngày bảo vệ

Hạng mục	Cách kiểm tra	Người	Xong
Website hoạt động	Mở https://vaultra-844.pages.dev, Ctrl+F5	Khang	☐
Backend hoạt động	Mở …workers.dev/v1/health → {"status":"ok"}	Phúc	☐
Database/dữ liệu	Đăng nhập 5 tài khoản, mở danh sách hợp đồng/đơn	Trường	☐
Tài khoản các role	customer, staff.hn, fm.hn, bom, admin đăng nhập được	Khang	☐
API quan trọng	Đặt kho → thanh toán thẻ test → check-in một lần trọn vẹn	Triển	☐
Email	Nhận được email xác nhận đặt kho và OTP gia hạn trong ≤ 1 phút	Triển	☐
Dữ liệu demo	Hủy đơn chờ thừa; chuẩn bị 1 đơn đã thanh toán, 1 hợp đồng đã check-in, 1 yêu cầu trả kho INSPECTED làm dự phòng	Triển	☐
Không lỗi console nghiêm trọng	F12 → Console trên các màn hình demo	Phúc	☐
Không dữ liệu test gây ảnh hưởng	Xóa/tắt bài tri thức thử, ticket thử; tên hiển thị đúng	Khang	☐
Các Flow đã chạy thử	Chạy toàn bộ Chương 5 một lần trọn vẹn ngày hôm trước, bấm giờ	Cả nhóm	☐
Phương án dự phòng	Video quay màn hình từng Flow + ảnh chụp kết quả + slide mốc F6	Trường	☐
Mạng	Có 4G dự phòng; thử VNPay sandbox mở được	Khang	☐

CHƯƠNG 6

Kịch bản thuyết trình cho 4 thành viên

> Lời thoại viết để nói thành tiếng, có thể học thuộc. Phần trong [ngoặc vuông] là thao tác trên máy. Thời lượng đã tính sẵn; tập với đồng hồ.

6.1 Khang — Mở đầu, tổng quan, Flow 1 và Flow 7

KHANG · MỞ ĐẦU (1 PHÚT)

Em chào thầy cô ạ. Nhóm em gồm bốn bạn: Triển, Phúc, Trường và em là Khang. Hôm nay nhóm em xin trình bày dự án VAULTRA — hệ thống cho thuê và quản lý kho tự quản.

Thầy cô có thể hình dung thế này: khi mình chuyển nhà hoặc cửa hàng hết chỗ để hàng, mình cần thuê một cái kho nhỏ. Bình thường phải gọi điện hỏi còn kho không, giá bao nhiêu, rồi đến tận nơi làm giấy tờ. VAULTRA cho phép làm tất cả những việc đó trên mạng, và giúp cơ sở kho quản lý từ lúc khách đặt đến lúc khách trả kho.

KHANG · TỔNG QUAN VÀ KIẾN TRÚC (2 PHÚT)

Hệ thống có năm vai trò: khách thuê, nhân viên cơ sở, quản lý cơ sở, quản lý vận hành doanh nghiệp và quản trị hệ thống. Mỗi vai trò có một khu vực giao diện riêng, chỉ thấy đúng chức năng của mình.

Đề bài yêu cầu bảy quy trình. Em tóm tắt bằng vòng đời một cái kho: khách đặt kho, đến nhận kho, dùng kho hằng tháng, gia hạn hoặc trả kho. Song song là chính sách giá, quản lý kho và nhân sự, và hỗ trợ khách. [Mở Hình 1.1]

Về kỹ thuật, giao diện viết bằng React, máy chủ viết bằng NestJS, dữ liệu lưu trong MySQL. Bản demo hôm nay chạy trên Cloudflare. Điểm nhóm em chú trọng nhất là: một cái kho không bao giờ được giao cho hai người, và mọi đồng tiền đều truy được nguồn gốc.

Bây giờ em xin demo quy trình đầu tiên — đặt kho.

KHANG · FLOW 1 ĐẶT KHO — VỪA NÓI VỪA LÀM (3 PHÚT)

[Mở trang chủ, bấm Đặt kho ngay] Em đóng vai một khách hàng. Em chọn cơ sở Cầu Giấy và loại kho nhỏ.

[Chỉ vào dải lịch trống] Ở đây hệ thống cho biết mỗi ngày còn bao nhiêu kho trống. Con số này tính từ số kho thực tế trừ đi các đơn và hợp đồng đang giữ kho trong cùng khoảng thời gian.

[Chọn ngày, 1 tháng] Em chọn bắt đầu hôm nay, thuê một tháng. Bảng giá hiện từng dòng: tiền thuê, tiền cọc, phí quản trị, thuế. Khách biết chính xác mình trả gì.

[Chọn khung giờ, bấm thanh toán] Khi em bấm đặt, hệ thống giữ chỗ cho em mười lăm phút. Nếu em không trả tiền, chỗ được nhả ra cho người khác.

Một câu hỏi hay gặp là: nếu hai người cùng bấm đặt cái kho cuối cùng thì sao? Nhóm em đã viết một bài test cho đúng tình huống đó: một trăm yêu cầu cùng lúc, kết quả chỉ một người đặt được, chín mươi chín người nhận thông báo hết kho.

[Thanh toán thẻ test VNPay] Em thanh toán bằng thẻ thử của VNPay. Lưu ý là hệ thống không tin trang mà trình duyệt quay về; tiền chỉ được ghi nhận khi VNPay gọi thẳng máy chủ kèm chữ ký hợp lệ.

[Mở đơn] Đơn đã được xác nhận, có hóa đơn đã thanh toán và lịch hẹn đến nhận kho. Đơn này bạn Triển sẽ dùng để làm thủ tục nhận kho. Em mời Triển.

KHANG · FLOW 7 HỖ TRỢ (2 PHÚT, SAU PHẦN CỦA PHÚC)

Em quay lại với quy trình hỗ trợ khách. Giả sử khách không mở được cổng vì mã PIN báo sai.

[Cửa sổ khách: tạo ticket danh mục Mã truy cập] Khách tạo yêu cầu, chọn hợp đồng, mô tả và đính kèm ảnh. Hệ thống cấp mã SR và tự tính hạn phản hồi: loại mã truy cập là ưu tiên cao, phải phản hồi trong ba mươi phút theo giờ làm việc của cơ sở.

[Cửa sổ nhân viên] Nhân viên thấy yêu cầu mới, bấm nhận việc và trả lời khách. Em thêm một ghi chú nội bộ — ghi chú này chỉ nhân viên thấy. [Chuyển sang cửa sổ khách] Khách thấy câu trả lời nhưng không thấy ghi chú nội bộ.

[Nhân viên đánh dấu đã giải quyết, khách đánh giá 5 sao] Khi xong, nhân viên phải ghi nguyên nhân, khách đánh giá. Mọi thay đổi đều được ghi lịch sử để đo hiệu suất. Em xin chuyển lời cho bạn Trường.

Câu hỏi Khang có thể được hỏi

Câu hỏi	Trả lời mẫu (ngắn, dễ nhớ)
Vì sao giữ chỗ 15 phút?	Đủ thời gian để khách thanh toán nhưng không để kho bị giữ ảo lâu. Con số nằm trong chính sách, quản lý vận hành đổi được.
100 người cùng đặt kho cuối thì sao?	Hệ thống khóa loại kho trong lúc đếm và tạo đơn, nên các yêu cầu xếp hàng lần lượt. Nhóm có bài test 100 yêu cầu: 1 thành công, 99 hết kho.
Khách sửa đường link trả về để giả là đã thanh toán?	Không được, vì hệ thống chỉ ghi nhận tiền khi VNPay gọi máy chủ kèm chữ ký; chữ ký sai thì bị từ chối.
Bấm Đặt hai lần có ra hai đơn không?	Không. Mỗi lần bấm có một mã chống trùng; gửi lại cùng mã thì máy chủ trả lại kết quả cũ.
SLA là gì?	Là cam kết thời gian phản hồi và giải quyết. Ví dụ lỗi mã truy cập phải phản hồi trong 30 phút, giải quyết trong 4 giờ.
Khách có xem được ghi chú nội bộ không?	Không. Máy chủ lọc bỏ ghi chú nội bộ trước khi trả dữ liệu cho khách, không chỉ ẩn trên giao diện.
Hệ thống có mấy vai trò, khác nhau thế nào?	Năm vai trò. Khách chỉ thấy dữ liệu của mình; nhân viên và quản lý chỉ thấy cơ sở được giao; quản lý vận hành thấy toàn hệ thống về kinh doanh; quản trị lo tài khoản và phân quyền.
Kiến trúc gồm những gì?	Giao diện React, máy chủ NestJS, cơ sở dữ liệu MySQL; bản demo chạy trên Cloudflare. Hai bên nói chuyện qua API, có thêm kênh realtime để màn hình tự cập nhật.
Em đóng góp phần nào trong dự án?	Trả lời bằng phần việc thật Khang đã làm (giao diện, triển khai, tích hợp…) và mở đúng màn hình/mã nguồn tương ứng.
Bản demo khác bản đầy đủ chỗ nào?	Bản demo chạy trên Cloudflare nên dữ liệu không nằm trong MySQL mà trong bộ nhớ và được lưu lên kho lưu trữ R2; bản đầy đủ dùng MySQL. Các bài test chống trùng chạy trên bản MySQL.
Nếu khách hủy đơn thì được hoàn bao nhiêu?	Theo bậc: hủy trước từ 7 ngày hoàn hết; 2–6 ngày hoàn nửa tiền cọc; dưới 48 giờ mất cọc. Phí quản trị không hoàn.
Realtime là gì?	Khi một người thao tác, màn hình của người khác tự cập nhật mà không phải F5 — như lúc em chuyển sang cửa sổ khách và thấy câu trả lời ngay.

6.2 Triển — Flow 2 và Flow 3

TRIỂN · MỞ PHẦN (30 GIÂY)

Cảm ơn Khang. Em là Triển, em phụ trách hai quy trình nối tiếp nhau: nhận kho và quản lý kho trong lúc khách thuê, đến khi trả kho.

TRIỂN · FLOW 2 CHECK-IN (4 PHÚT)

Khách vừa đặt đơn đã đến cơ sở. [Cửa sổ nhân viên, /staff/scan] Nhân viên nhập mã đơn và bắt đầu phiên nhận kho. Phiên gồm bốn bước.

Bước một, rà soát: nhân viên đối chiếu giấy tờ, hệ thống chỉ lưu loại giấy tờ và bốn số cuối, không lưu ảnh giấy tờ để bảo vệ dữ liệu cá nhân.

Bước hai, gán kho: hệ thống đã đề xuất sẵn kho cụ thể. Nếu kho thực tế có vấn đề, nhân viên đổi sang kho khác cùng loại; đổi sang kho đắt hơn thì phát sinh hóa đơn chênh lệch, rẻ hơn thì tạo khoản hoàn tiền.

Bước ba, ký hợp đồng: [Gửi link ký, cửa sổ khách mở email và ký] Hợp đồng được sinh thành PDF, kèm mã băm SHA-256. Chỉ cần sửa một ký tự trong file là mã băm thay đổi, nên chứng minh được hợp đồng không bị chỉnh sửa sau khi ký.

Bước bốn, biên bản hiện trạng và cấp mã: [Tải 4 ảnh, lưu biên bản] Bắt buộc tối thiểu bốn ảnh, mỗi ảnh lưu thời điểm và mã băm — đây là bằng chứng khi trả kho. [Cấp PIN, Hoàn tất]

Điểm quan trọng: nếu nhân viên cố cấp mã khi khách chưa ký hoặc chưa trả đủ tiền, máy chủ trả lỗi và liệt kê bước còn thiếu. Giao diện không quyết định chuyện này, máy chủ quyết định.

[Cửa sổ khách, tab Truy cập] Khách thấy mã PIN sáu số. Mã này được mã hóa trong cơ sở dữ liệu và không bao giờ gửi qua email.

TRIỂN · FLOW 3 ĐANG THUÊ VÀ TRẢ KHO (4 PHÚT)

Trong lúc thuê, khách quản lý mọi thứ ở trang hợp đồng. [Bấm Mở kho, rồi Đóng kho] Khách có thể mở và đóng kho từ xa; mỗi lần đều được ghi vào nhật ký ra vào. Khóa thông minh thật nằm ngoài phạm vi đề bài nên đây là mô phỏng, nhưng quy tắc giống hệt cổng PIN: đang bị khóa do nợ thì không mở được.

[Tab Hóa đơn] Mỗi kỳ hệ thống phát hành hóa đơn trước bảy ngày. Khách trả một lần cho nhiều hóa đơn được; tiền phân bổ vào hóa đơn cũ nhất trước.

[Hợp đồng thứ hai, tab Trả kho] Khi muốn trả kho, khách chọn ngày, phải báo trước ít nhất bảy ngày, và thấy ngay ước tính quyết toán.

Công thức quyết toán là: tiền cọc, trừ nợ còn lại, trừ phí hư hại, trừ phí vệ sinh, cộng tiền trả trước chưa dùng nếu chính sách cho phép. Ví dụ cọc 690 nghìn, còn nợ 50 nghìn, không hư hại thì hoàn 640 nghìn.

[Cửa sổ nhân viên, kiểm tra trả kho] Nhân viên so ảnh lúc nhận và lúc trả theo từng mục. Phí hư hại bắt buộc có ảnh. Khi chốt, mọi mã truy cập bị thu hồi ngay. Khách có ba ngày để khiếu nại trước khi quyết toán được chốt.

Kho chỉ quay về trạng thái trống khi đủ ba điều kiện: không còn mã truy cập hoạt động, đã có biên bản trả kho, và nhân viên xác nhận đã vệ sinh. Em xin mời bạn Phúc trình bày phần chính sách và doanh thu.

Câu hỏi Triển có thể được hỏi

Câu hỏi	Trả lời mẫu
Cấp PIN trước khi ký được không?	Không. Máy chủ trả 409 CHECKIN_PRECONDITION_FAILED kèm danh sách điều kiện còn thiếu.
Làm sao chứng minh hợp đồng không bị sửa?	PDF lưu kèm SHA-256 trong contract_documents; chữ ký lưu thời điểm, IP, mã băm của tài liệu đã ký.
PIN lưu dạng gì? Admin xem được không?	Mã hóa AES-256-GCM; chỉ khách và nhân viên cơ sở có quyền giải mã, mỗi lần xem ghi audit. Admin không có quyền nghiệp vụ này.

Câu hỏi	Trả lời mẫu
Hai PIN trùng nhau thì sao?	Cột active_pin_key có ràng buộc UNIQUE trong phạm vi cơ sở; trùng thì sinh lại.
Check-in lỗi giữa chừng có để lại dữ liệu nửa vời không?	Không. Bước hoàn tất là một giao dịch; hủy phiên thì đơn về trạng thái cũ.
Công thức quyết toán? Âm thì sao?	net = cọc − nợ − hư hại − vệ sinh − phí trả sớm + hoàn trả trước; âm thì tạo hóa đơn quyết toán cho phần thiếu.
Vì sao chờ 3 ngày mới chốt?	Để khách khiếu nại; trong thời gian đó tiền cọc chưa bị khấu trừ chính thức.
Phụ phí có tùy ý không?	Không; chỉ chọn từ danh mục phí do quản lý vận hành quản lý, sửa số tiền cần quyền quản lý cơ sở.
Trả một phần hóa đơn?	Hóa đơn thành PARTIALLY_PAID, phần còn lại vẫn tính vào chuỗi quá hạn.
Hoàn tiền làm thế nào?	Tạo yêu cầu hoàn tiền chờ duyệt theo hạn mức; thực hiện chuyển khoản/tiền mặt và ghi mã tham chiếu (mark-manual).
Mở kho từ xa có thật không?	Là mô phỏng (ghi access_events); khóa thông minh thật nằm ngoài phạm vi giai đoạn 1 theo đặc tả.
Email ký hợp đồng không đến thì sao?	Dùng ký OTP tại quầy; nhật ký gửi thư ở notification_deliveries cho biết lỗi.
Ảnh hiện trạng có thể là file giả?	Máy chủ kiểm tra magic bytes; nội dung không khớp định dạng thì bị từ chối.
Kho hỏng phát hiện lúc check-in?	Báo kho không đạt: kho chuyển bảo trì, đổi sang kho cùng loại ngay trong phiên.
Khách không đến buổi kiểm tra trả kho?	Lịch hẹn bị đánh dấu lỡ hẹn, hợp đồng vẫn tính phí; khách đặt lại lịch.

6.3 Phúc — Flow 4 và tổng kết kiểm thử

PHÚC · FLOW 4 (5 PHÚT)

Cảm ơn Triển. Em là Phúc. Ở hai quy trình vừa rồi có rất nhiều con số: giữ chỗ mười lăm phút, báo trước bảy ngày, phí trễ hạn năm mươi nghìn. Em trình bày nơi các con số đó được quản lý.

Các con số này không viết cứng trong mã nguồn mà là chính sách có phiên bản. [Mở /ops/policies] Quản lý vận hành tạo phiên bản mới từ bản đang dùng, sửa tham số và bắt buộc nhập lý do.

[Nhập sai: ân hạn 12 ngày lớn hơn ngày khóa 10] Hệ thống kiểm tra cả quan hệ giữa các tham số: ân hạn phải nhỏ hơn ngày khóa truy cập, nhỏ hơn ngày thông báo cuối, nhỏ hơn ngày chấm dứt. Nhập sai thì bị từ chối ngay.

[Xem tác động] Trước khi xuất bản, quản lý xem có bao nhiêu đơn, bao nhiêu hợp đồng sẽ chịu bản mới. [Xuất bản]

Nguyên tắc quan trọng nhất là không hồi tố: hợp đồng đã ký giữ nguyên phiên bản chính sách lúc ký, đến kỳ gia hạn mới áp dụng bản mới. Khách không bị đổi điều khoản giữa chừng.

Về tiền, hệ thống tách ba khái niệm: tiền phát hành là tổng hóa đơn, tiền thu được là tiền thực nhận, doanh thu ghi nhận là phần dịch vụ đã cung cấp. Tiền cọc không phải doanh thu.

[Mở /ops/finance, chạy đối soát] Đối soát so giao dịch của cổng thanh toán với hệ thống theo từng ngày, cả tiền mặt tại quầy. Em xin nói rõ: VNPay bản thử không cho máy chủ Cloudflare tra cứu, nên trên bản demo, phần "theo cổng" được mô phỏng từ dữ liệu hệ thống. Muốn thấy chênh lệch, em dán một sao kê có số tiền khác — [dán] — hệ thống đánh dấu dòng lệch để giải trình.

PHÚC · TỔNG KẾT KIỂM THỬ (2 PHÚT)

Về kiểm thử, nhóm em có ba lớp. Lớp một là unit test cho các công thức: tính giá, khuyến mãi, phí trễ, mốc quá hạn, quyết toán, SLA, máy trạng thái — mười sáu bài, đang đạt cả mười sáu.

> Lớp hai là test đầu–cuối gọi API thật cho từng quy trình, ví dụ quy trình trả kho mười lăm trên mười lăm kiểm tra đạt, và bài một trăm yêu cầu đồng thời.
>
> Lớp ba là bộ test case thủ công cho bảy quy trình, gồm cả trường hợp nhập sai, sai quyền, gửi trùng. Bộ này nhóm chạy bằng tay và ghi kết quả thực tế. Em xin mời bạn Khang trình bày quy trình hỗ trợ.

Câu hỏi Phúc có thể được hỏi

Câu hỏi	Trả lời mẫu
Tham số chính sách nằm ở đâu?	Giá trị mặc định ở packages/shared/src/policy.ts (DEFAULT_POLICY_PARAMS), dữ liệu thật ở bảng policy_versions.params; schema kiểm tra bằng Zod.
Vì sao cần phiên bản?	Để không hồi tố và truy được hợp đồng nào chịu bản nào (contracts.policy_version_id).
Sửa bản đang ACTIVE được không?	Không; tạo bản mới hiệu lực ngay. Bản SCHEDULED hủy được trước ngày hiệu lực.
Hai bản chồng hiệu lực?	Bị chặn với POLICY_OVERLAP.
FM tự đặt giá được không?	Được trong khung giá do BOM đặt; ngoài khung tạo yêu cầu chờ BOM duyệt.
Khuyến mãi cộng dồn?	Không, trừ khi có cờ stackable.
Phí trễ tính thế nào?	Lấy giá trị lớn hơn giữa 50.000 ₫ và 10% tiền thuê, trần 500.000 ₫, mỗi hóa đơn một lần.
Doanh thu khác tiền thu?	Thu là tiền nhận được; ghi nhận là phần dịch vụ đã cung cấp. Trả trước 3 tháng thì tháng đầu mới ghi nhận 1 tháng.
Cọc có phải doanh thu?	Không, là khoản phải trả lại; chỉ phần bị khấu trừ mới thành doanh thu.
Đối soát trên bản demo có thật không?	Phần cổng là mô phỏng vì sandbox không cho Cloudflare tra cứu (lỗi 522); có thể dán sao kê thủ công. Nói rõ, không khẳng định đã nối VNPay thật.
Miễn giảm làm sao để không gian lận?	Đề xuất có lý do, duyệt theo hạn mức, sinh credit note; không sửa trực tiếp dòng hóa đơn; ghi audit.
Unit test kiểm tra gì? Chạy thế nào?	pnpm --filter @vaultra/api test; 16 test công thức và quy tắc; đạt 16/16.
E2E chạy ở đâu?	Bản Node + MySQL với API đang chạy; tsx test/e2e/run-all.ts.
Có test giao diện tự động không?	Chưa; giao diện kiểm thử thủ công theo test case.
Làm sao biết test case đã chạy?	Cột Actual/Status do nhóm điền sau khi chạy; chưa chạy thì để Not Tested.

6.4 Trường — Flow 5, Flow 6 và kết luận

> **TRƯỜNG · FLOW 5 (4 PHÚT)**
>
> Cảm ơn Khang. Em là Trường. Hai quy trình cuối là quản lý kho, nhân sự và xử lý gia hạn, quá hạn.
>
> [Mở /manage/floor] Đây là sơ đồ mặt bằng của cơ sở. Mỗi ô là một kho, màu thể hiện trạng thái: trống, đã giữ, đang thuê, bảo trì, bị khóa do nợ. Phía trên là tỷ lệ lấp đầy.
>
> [Chọn kho trống, đổi sang Bảo trì kèm lý do] Mọi đổi trạng thái đều phải đi theo máy trạng thái và có lý do. Ví dụ kho đang có khách thì không thể chuyển thẳng sang bảo trì, phải đổi kho cho khách trước. Lịch sử ghi ai đổi, lúc nào, vì sao.
>
> Khi phân bổ kho cho khách, hệ thống chọn theo ba tiêu chí dễ giải thích: mức ưu tiên do quản lý đặt, kho trống lâu nhất trước, rồi đến mã kho. Ở tầng cơ sở dữ liệu còn có ràng buộc để một kho chỉ có một phân bổ đang mở.

[Tạo việc, giao cho nhân viên; cửa sổ nhân viên tự hiện việc mới] Quản lý giao việc, nhân viên thấy ngay mà không cần tải lại trang, nhận việc, bắt đầu và hoàn thành. Thời gian thực tế được ghi để đo hiệu suất.

TRƯỜNG · FLOW 6 VÀ KẾT LUẬN (4 PHÚT)

[Cửa sổ khách, hợp đồng một tháng, tab Gia hạn] Hợp đồng sắp hết hạn thì mở cửa sổ gia hạn, giá kỳ mới tăng tối đa mười phần trăm. Khách chọn thời hạn, nhận mã OTP qua email để chấp nhận phụ lục, rồi thanh toán. [Nhập OTP, gia hạn] Ngày hết hạn mới đã được cập nhật.

Nếu khách đang nợ quá hạn thì không gia hạn được, hệ thống yêu cầu trả nợ trước.

Phần quá hạn là một chuỗi mốc: [Mở bảng mốc] sau năm ngày ân hạn, ngày thứ sáu cộng phí trễ, ngày thứ mười tạm khóa mã truy cập và nhân viên gắn khóa chặn, ngày ba mươi gửi thông báo cuối, ngày bốn lăm tạo yêu cầu chấm dứt. Ở bất kỳ mốc nào, khách trả đủ thì mã mở lại tự động.

Chấm dứt hợp đồng không bao giờ tự động: hệ thống chỉ đề xuất, quản lý cơ sở xác nhận và quản lý vận hành phê duyệt.

Em xin nói rõ một giới hạn: các tác vụ định kỳ trên bản demo chạy theo giờ thật mỗi giờ, nhưng không tua được thời gian, nên chuỗi mốc này không thể hiện trong buổi hôm nay. Bằng chứng là các unit test về mốc quá hạn và phí trễ, cùng bài test đầu–cuối chạy trên bản đầy đủ với đồng hồ thử.

Tóm lại, nhóm em đã xây dựng đủ năm vai trò và bảy quy trình của đề bài, với hai bảo đảm cốt lõi là không giao trùng kho và mọi khoản tiền truy được nguồn gốc. Hướng phát triển tiếp: chuyển dữ liệu bản demo sang cơ sở dữ liệu thật, và nối khóa thông minh. Nhóm em cảm ơn thầy cô đã lắng nghe và mong nhận được câu hỏi ạ.

Câu hỏi Trường có thể được hỏi

Câu hỏi	Trả lời mẫu
Thuật toán phân bổ?	Ứng viên cùng cơ sở, cùng loại, AVAILABLE, không cờ, không allocation/work order mở; xếp ưu tiên → trống lâu nhất → mã kho; top 3.
Hai người gán cùng một kho?	Khóa hàng khi gán + UNIQUE(open_unit_key): chỉ một allocation mở.
Kho đang thuê bị hỏng?	Gắn cờ vận hành + work order; muốn đưa vào bảo trì phải đổi kho cho khách trước.
Nhân viên sửa việc của người khác?	Không; chỉ việc được giao hoặc việc OPEN để tự nhận.
Ca làm việc có ràng buộc check-in không?	Hiện chưa; đặc tả không yêu cầu ràng buộc này. Ca dùng để lập lịch và hiển thị người đang trực.
Mốc quá hạn?	D+1 GRACE, D+6 LATE (phí trễ), D+10 LOCKED_OUT, D+30 FINAL_NOTICE, D+45 DEFAULT_PENDING.
Job chạy 2 lần có cộng phí 2 lần?	Không; job idempotent, mỗi hóa đơn một lần phí trễ, chỉ ghi khi giai đoạn thay đổi.
Vì sao không tự chấm dứt?	Hậu quả pháp lý; cần FM xác nhận và BOM duyệt, lưu bằng chứng thông báo.
Trả tiền khi đang bị khóa?	Về CURRENT, PIN mở lại tự động, tạo việc tháo khóa chặn.
Hóa đơn đang tranh chấp có bị khóa?	Không; ticket PAYMENT đặt dispute_hold.
Giá gia hạn?	Giá niêm yết hiện hành, tăng tối đa 10%; chưa báo trước đủ 30 ngày thì giữ giá cũ.
Hết hạn không gia hạn?	Holdover theo tháng (mặc định) hoặc bắt buộc trả kho, tùy chính sách.
Vì sao demo không thấy chuyển mốc?	Job chạy mỗi giờ theo giờ thật nhưng bản deploy không tua thời gian được; bằng chứng là unit test + e2e trên bản Node với đồng hồ thử.
Realtime hoạt động thế nào?	WebSocket với vé ký 60 giây; sau mỗi thao tác ghi máy chủ đẩy tín hiệu, trình duyệt tải lại dữ liệu đang hiển thị.

Câu hỏi	Trả lời mẫu
Hướng phát triển?	Dữ liệu demo sang database thật (D1/MySQL), khóa thông minh, ứng dụng di động, test giao diện tự động.

6.5 Câu chuyển tiếp giữa các phần

Từ → đến	Câu chuyển
Khang → Triển	"Đơn này bạn Triển sẽ dùng để làm thủ tục nhận kho. Em mời Triển."
Triển → Phúc	"Những con số vừa rồi — báo trước 7 ngày, 3 ngày khiếu nại — được quản lý ở đâu, em mời bạn Phúc."
Phúc → Khang	"Em xin mời bạn Khang trình bày quy trình hỗ trợ khách."
Khang → Trường	"Em xin chuyển lời cho bạn Trường với phần quản lý kho và gia hạn."
Trường → Hội đồng	"Nhóm em cảm ơn thầy cô đã lắng nghe và mong nhận được câu hỏi ạ."

CHƯƠNG 7

Ngân hàng câu hỏi giảng viên

> 111 câu hỏi chia 15 nhóm. Mỗi câu có: trả lời mẫu ngắn, phân tích, ví dụ trong VAULTRA, lưu ý và cách trả lời không nên dùng.

7.1 Tổng quan dự án

Q1 VAULTRA giải quyết bài toán gì?

Trả lời mẫu. Cho thuê và vận hành kho tự quản nhiều cơ sở: khách đặt và trả tiền online, cơ sở bàn giao có biên bản, doanh nghiệp quản lý chính sách và doanh thu trên cùng dữ liệu.

Phân tích. Ba nhóm vấn đề: khách thiếu thông tin tức thời (giá, còn kho), cơ sở thiếu quy trình bàn giao có bằng chứng, doanh nghiệp thiếu chính sách thống nhất và số liệu theo cơ sở.

Ví dụ trong VAULTRA. Khách đặt kho 2.5m² lúc 22:00 không cần gọi điện; sáng hôm sau nhân viên thấy lịch hẹn trên bảng Hôm nay.

Lưu ý. Nói bằng ví dụ người dùng, sau đó mới nói kỹ thuật.

Không nên trả lời. "Là web cho thuê kho" — quá chung, không nêu được giá trị.

Q2 Đối tượng sử dụng là ai?

Trả lời mẫu. Năm vai trò: khách thuê, nhân viên cơ sở, quản lý cơ sở, quản lý vận hành doanh nghiệp, quản trị hệ thống.

Phân tích. Vai trò định nghĩa ở packages/shared/src/permissions.ts; phạm vi dữ liệu OWN/FACILITY/GLOBAL; một tài khoản có thể có nhiều vai trò.

Ví dụ trong VAULTRA. staff.hn chỉ thấy cơ sở Cầu Giấy; bom thấy mọi cơ sở nhưng chỉ đọc dữ liệu vận hành chi tiết.

Lưu ý. Nêu được khác biệt phạm vi, không chỉ liệt kê tên.

Không nên trả lời. Nói admin "làm được mọi thứ" — sai, admin không thao tác tiền/hợp đồng.

Q3 Phạm vi dự án gồm và không gồm những gì?

Trả lời mẫu. Gồm 7 quy trình bắt buộc, thanh toán sandbox, hợp đồng điện tử, báo cáo; không gồm ứng dụng native, khóa thông minh thật, kế toán tổng hợp.

Phân tích. Đặc tả mục 1.3 liệt kê rõ; khóa thông minh có interface và mô phỏng (mở/đóng kho từ xa ghi access_events).

Ví dụ trong VAULTRA. Nút Mở kho trên web là mô phỏng, không điều khiển khóa vật lý.

Lưu ý. Chủ động nói "ngoài phạm vi giai đoạn 1" thay vì để bị hỏi dồn.

Không nên trả lời. Khẳng định đã nối khóa thông minh thật.

Q4 Điểm khác biệt chính của hệ thống?

Trả lời mẫu. Không giao trùng kho (bảo đảm ở database) và mọi đồng tiền truy được nguồn gốc.

Phân tích. Chống trùng hai lớp: khóa loại kho khi đặt + UNIQUE(open_unit_key) khi gán. Truy vết tiền: hóa đơn → dòng → payment → payment_allocations → refunds → customer_ledger_entries.

Ví dụ trong VAULTRA. Test nfr03: 100 request đồng thời, 1 thành công.

Lưu ý. Luôn kèm bằng chứng (test, bảng).

Không nên trả lời. Nói "giao diện đẹp" là điểm chính.

Q5 Hệ thống có bao nhiêu màn hình, API, bảng?

Trả lời mẫu. Khoảng 306 endpoint, 104 bảng; 6 khu vực giao diện.
Phân tích. Số liệu đếm từ mã nguồn: controller (@Get/@Post…), schema.prisma (model), các file routes.tsx.
Ví dụ trong VAULTRA. Khu vực /manage có hơn 20 route: floor, units, reservations, contracts, renewals, delinquency, moveouts…
Lưu ý. Nói số tròn, kèm "theo đếm trong mã nguồn".
Không nên trả lời. Bịa số màn hình theo đặc tả (68) mà không kiểm tra.

Q6 Bản demo đang chạy ở đâu?

Trả lời mẫu. Giao diện trên Cloudflare Pages, API trên Cloudflare Worker; bản đầy đủ chạy Node + MySQL.
Phân tích. Worker dùng chung service NestJS, thay lớp DB bằng edge-prisma (bộ nhớ + snapshot R2).
Ví dụ trong VAULTRA. vaultra-844.pages.dev và vaultra-gateway.zerngulon.workers.dev.
Lưu ý. Phải nói rõ khác biệt DB của bản demo.
Không nên trả lời. Nói bản demo dùng MySQL.

Q7 Nhóm đã làm được bao nhiêu phần trăm đặc tả?

Trả lời mẫu. Đủ 5 vai trò và 7 quy trình ở luồng chính; một số phần mở rộng [MR] và dữ liệu bản demo (mô phỏng DB) còn giới hạn.
Phân tích. Không tự nhận 100%: các giới hạn ở Chương 8 (DB mô phỏng trên Worker, không tua thời gian, đối soát mô phỏng, khóa mô phỏng).
Ví dụ trong VAULTRA. TRACEABILITY.md ghi 100% nhưng nhóm cần nói theo bằng chứng đã chạy.
Lưu ý. Trung thực tạo thiện cảm.
Không nên trả lời. "100% hoàn thành, không lỗi".

7.2 Phân tích nghiệp vụ

Q8 Vì sao khách đặt theo loại kho, không chọn đúng kho?

Trả lời mẫu. Đề bài yêu cầu kho cụ thể được phân bổ sau; giúp vận hành linh hoạt.
Phân tích. Khả dụng tính theo loại kho trong khoảng ngày; phân bổ kho cụ thể khi gần ngày nhận (allocation lead), giữ kho trống bán ngắn hạn.
Ví dụ trong VAULTRA. Đặt "Kho nhỏ 2.5m²" → hệ thống gán A-012 sau khi xác nhận.
Lưu ý. Trích đặc tả mục 1.4.
Không nên trả lời. "Vì làm cho dễ".

Q9 Vì sao phải giữ chỗ 15 phút?

Trả lời mẫu. Cân bằng giữa thời gian thanh toán và tránh giữ chỗ ảo.
Phân tích. hold_minutes là tham số chính sách; đơn PENDING_PAYMENT chưa hết giờ vẫn tính vào khả dụng; mỗi khách tối đa 3 đơn chờ.
Ví dụ trong VAULTRA. Khách bỏ dở thanh toán → sau 15 phút kho trở lại khả dụng.
Lưu ý. Biết tham số đổi được.
Không nên trả lời. Nói con số viết cứng trong code.

Q10 Tiền cọc dùng để làm gì và tính thế nào?

Trả lời mẫu. Bảo đảm cho hư hại/nợ; mặc định bằng 1 tháng tiền thuê.
Phân tích. deposit_type MONTHS_OF_RENT, deposit_value 1; cọc là nợ phải trả, quyết toán khi trả kho.
Ví dụ trong VAULTRA. Kho 690.000 ₫/tháng → cọc 690.000 ₫.
Lưu ý. Phân biệt cọc với doanh thu.
Không nên trả lời. Nói cọc là doanh thu.

Q11 Chính sách hủy hoạt động thế nào?

Trả lời mẫu. Theo số ngày đến ngày bắt đầu: ≥7 ngày hoàn 100%; 2–6 ngày hoàn 50% cọc; <48 giờ mất cọc; phí quản trị không hoàn.
Phân tích. cancellation_tiers trong policy; xem trước bằng GET /reservations/:id/cancellation-preview.
Ví dụ trong VAULTRA. Hủy trước 10 ngày → hoàn đủ trừ phí quản trị.
Lưu ý. Có test case biên 48 giờ.
Không nên trả lời. Nói "hủy là mất hết".

Q12 Vì sao phải ký hợp đồng trước khi nhận mã?

Trả lời mẫu. Ràng buộc pháp lý: chỉ giao quyền vào kho khi đã có hợp đồng hợp lệ và đã trả tiền đến hạn.
Phân tích. Backend chặn CHECKIN_PRECONDITION_FAILED; tham khảo thông lệ CubeSmart (đặc tả mục 2).
Ví dụ trong VAULTRA. Nhân viên cố cấp PIN → lỗi liệt kê CONTRACT_NOT_SIGNED.
Lưu ý. Nhấn mạnh chặn ở backend.
Không nên trả lời. "Giao diện ẩn nút nên không cấp được".

Q13 Vì sao chuỗi quá hạn có nhiều mốc?

Trả lời mẫu. Để nhắc trước, phạt nhẹ trước, khóa sau, chấm dứt cuối cùng có người duyệt — công bằng và có bằng chứng.
Phân tích. GRACE→LATE→LOCKED_OUT→FINAL_NOTICE→DEFAULT_PENDING→DEFAULTED; mỗi chuyển ghi delinquency_events.
Ví dụ trong VAULTRA. Ngày 10 PIN bị chặn; trả đủ thì mở lại.
Lưu ý. Thuộc mốc 5/10/30/45.
Không nên trả lời. Nói hệ thống tự chấm dứt hợp đồng.

Q14 Vì sao cần SLA cho ticket?

Trả lời mẫu. Cam kết thời gian phản hồi theo mức độ nghiêm trọng; đo hiệu suất.
Phân tích. sla_policies theo danh mục; tính theo giờ làm việc; URGENT 24/7; cảnh báo 80%.
Ví dụ trong VAULTRA. Không vào được cổng: phản hồi 30 phút.
Lưu ý. Kể được một ví dụ cụ thể.
Không nên trả lời. "SLA là đánh giá sao".

7.3 Câu hỏi về 7 Flow

Q15 F1: Thanh toán thành công nhưng đơn đã hết hạn giữ thì sao?

Trả lời mẫu. Đặc tả: nếu còn kho thì khôi phục đơn thành CONFIRMED, hết kho thì hoàn 100%.
Phân tích. Trạng thái EXPIRED → CONFIRMED được cho phép trong states.ts riêng cho trường hợp IPN đến muộn.
Ví dụ trong VAULTRA. Khách trả tiền ở phút 16.
Lưu ý. Nói theo đặc tả + máy trạng thái; nếu chưa test thì nói chưa test.
Không nên trả lời. Nói "tiền mất".

Q16 F1: Làm sao biết còn bao nhiêu kho?

Trả lời mẫu. Khả dụng = kho bán được − cam kết (đơn còn hiệu lực + hợp đồng) có khoảng ngày giao nhau.
Phân tích. Công thức đặc tả 7.3, quy ước nửa mở [start, end).
Ví dụ trong VAULTRA. Dải "Lịch trống 8 tuần" trên trang đặt kho.
Lưu ý. Giải thích "giao nhau" bằng ví dụ ngày.
Không nên trả lời. Nói "đếm kho trạng thái AVAILABLE" — thiếu đơn tương lai.

Q17 F2: Kho thực tế không đạt khi nhận thì xử lý sao?

Trả lời mẫu. Báo kho không đạt: kho → bảo trì, đổi kho cùng loại ngay trong phiên.
Phân tích. POST /checkin-sessions/:id/unit-unfit, swap-unit; allocation cũ RELEASED, mới ACTIVE; sinh lại phụ lục.
Ví dụ trong VAULTRA. Kho A-003 thấm nước → đổi A-004.
Lưu ý. Nói được tên hai API (unit-unfit, swap-unit) và việc allocation cũ được giải phóng.
Không nên trả lời. Nói phải hủy đơn.

Q18 F2: PIN được cấp và bảo vệ thế nào?

Trả lời mẫu. Sinh ngẫu nhiên 6 số, mã hóa AES-256-GCM, duy nhất trong cơ sở, chỉ hiển thị trên web.
Phân tích. pin_enc + pin_lookup_hash (HMAC) + active_pin_key UNIQUE; xem PIN ghi audit access.pin_viewed.
Ví dụ trong VAULTRA. Tab Truy cập của khách.
Lưu ý. Phân biệt mã hóa (giải được, dùng cho PIN cần hiển thị) và băm (không giải được, dùng cho mật khẩu).
Không nên trả lời. Nói gửi PIN qua email/SMS.

Q19 F3: Quyết toán cọc tính ra sao?

Trả lời mẫu. net = cọc − nợ − hư hại − vệ sinh − phí trả sớm + hoàn trả trước.
Phân tích. moveout.service.ts estimate(); dương → refund chờ duyệt; âm → hóa đơn SETTLEMENT; 3 ngày khiếu nại.
Ví dụ trong VAULTRA. 690.000 − 50.000 = 640.000 ₫ hoàn.
Lưu ý. Có unit test.
Không nên trả lời. Nói "trả lại toàn bộ cọc".

Q20 F3: Khách đổi PIN hoặc mất thẻ?

Trả lời mẫu. PIN cũ REVOKED, PIN mới ACTIVE; mất thẻ → LOST, tạo ticket + phí cấp lại.
Phân tích. POST /contracts/:id/access/pin/rotate; /access-credentials/:id/report-lost tạo ticket LOCK_KEY và hóa đơn phí theo danh mục.
Ví dụ trong VAULTRA. Khách bấm Đổi PIN ở tab Truy cập: mã mới hiện ngay, mã cũ bị từ chối ở cổng.
Lưu ý. Báo mất thẻ còn sinh phí cấp lại theo danh mục — nhắc để thấy liên kết F3 ↔ F7 ↔ F4.
Không nên trả lời. "Khách tự sửa PIN trong database."

Q21 F4: Sửa chính sách có ảnh hưởng hợp đồng cũ không?

Trả lời mẫu. Không; hợp đồng giữ policy_version_id lúc ký.
Phân tích. Không hồi tố; bản mới áp dụng cho báo giá/đơn mới và kỳ gia hạn.
Ví dụ trong VAULTRA. Đổi grace_days 5→7 không đổi hợp đồng đang chạy.
Lưu ý. Chỉ vào cột contracts.policy_version_id khi được hỏi bằng chứng.
Không nên trả lời. "Cập nhật cho tất cả".

Q22 F4: Khung giá để làm gì?

Trả lời mẫu. BOM giới hạn giá FM được đặt; ngoài khung cần duyệt.
Phân tích. price_bands (min/max), price_change_requests; ghi price_history.
Ví dụ trong VAULTRA. (Số giả định) Khung kho XS 250.000–350.000 ₫; FM đặt 290.000 ₫ áp dụng ngay, đặt 400.000 ₫ thì tạo yêu cầu chờ BOM.
Lưu ý. Giá đổi chỉ áp dụng cho báo giá mới, không đổi hợp đồng đang chạy.
Không nên trả lời. "FM tự đặt giá thoải mái."

Q23 F5: Không cho đổi trạng thái kho đang thuê sang bảo trì — vì sao?

Trả lời mẫu. Vì khách đang dùng; phải đổi kho cho khách trước hoặc chỉ gắn cờ vận hành.
Phân tích. Máy trạng thái units; operational_flag MAINTENANCE_SCHEDULED.
Ví dụ trong VAULTRA. Kho A-005 đang có khách bị dột: nhân viên gắn cờ và tạo work order; kho vẫn OCCUPIED.
Lưu ý. Đây là ví dụ hay để nói về máy trạng thái chặn ở backend.
Không nên trả lời. "Cứ chuyển sang bảo trì, khách tự biết."

Q24 F5: Việc (task) được tạo tự động khi nào?

Trả lời mẫu. Từ sự kiện: phân bổ, bàn giao, kiểm tra trả kho, khóa chặn, ticket, bảo trì…
Phân tích. core.createTask(...) trong các service; trạng thái OPEN→ASSIGNED→IN_PROGRESS→DONE.
Ví dụ trong VAULTRA. Yêu cầu trả kho không chọn lịch → tạo task MOVE_OUT_INSPECTION.
Lưu ý. Kể 3 loại việc tự sinh là đủ; không cần thuộc hết 9 loại.
Không nên trả lời. "Quản lý phải tạo tay mọi việc."

Q25 F6: Khách đang nợ có gia hạn được không?

Trả lời mẫu. Không — RENEWAL_BLOCKED_OUTSTANDING_BALANCE.
Phân tích. renewal.service.ts kiểm tra blockers trước khi tạo hóa đơn gia hạn.
Ví dụ trong VAULTRA. Khách còn một hóa đơn quá hạn bấm gia hạn → thông báo yêu cầu thanh toán trước.
Lưu ý. Nêu thêm: từ giai đoạn LATE khách cũng bị chặn đặt kho mới.
Không nên trả lời. "Được, nợ tính sau."

Q26 F6: Bản demo có cho thấy chuỗi quá hạn không?

Trả lời mẫu. Không thể hiện được trong buổi demo: job quá hạn chạy mỗi giờ theo giờ thật nhưng bản deploy không tua thời gian; bằng chứng là unit test và e2e bản Node.
Phân tích. jobs.service.ts chạy bằng timer trên Node và bằng cron 5 phút trên Worker (runDueJobs); thời gian thật, không tua (RISK-10).
Ví dụ trong VAULTRA. Trình chiếu kết quả unit test "delinquency targetStage and lateFee" thay vì chờ hợp đồng chuyển mốc.
Lưu ý. Nói trước khi bị hỏi.
Không nên trả lời. Nói "để em đợi một lát nó sẽ chuyển".

Q27 F7: Ghi chú nội bộ bị ẩn thế nào?

Trả lời mẫu. Máy chủ lọc is_internal trước khi trả cho khách.
Phân tích. support.service.ts; không phụ thuộc giao diện.
Ví dụ trong VAULTRA. Khách GET /tickets/:id không có tin nội bộ.
Lưu ý. Nếu được yêu cầu chứng minh, mở hai cửa sổ khách/nhân viên hoặc gọi API bằng token khách.
Không nên trả lời. "Giao diện ẩn đi".

Q28 F7: Ticket có thể sinh hành động gì?

Trả lời mẫu. Cấp lại PIN, tạo work order, phụ phí, miễn giảm, hoàn tiền.
Phân tích. POST /tickets/:id/actions/:kind; bản ghi liên kết ticket_id.
Ví dụ trong VAULTRA. Ticket báo kho dột → nhân viên bấm "Tạo work order" ngay trong ticket; work order mang ticket_id.
Lưu ý. Hành động liên kết đều theo hạn mức duyệt của F4 (miễn giảm, hoàn tiền).
Không nên trả lời. "Ticket chỉ để chat."

7.4 Database

Q29 Vì sao dùng MySQL?

Trả lời mẫu. Đề bài yêu cầu MySQL; cần giao dịch, khóa hàng, ràng buộc, cột sinh.
Phân tích. InnoDB REPEATABLE READ + SELECT … FOR UPDATE; UNIQUE trên cột sinh để chống trùng.
Ví dụ trong VAULTRA. open_unit_key, active_pin_key.
Lưu ý. Nhớ nói thêm bản demo Cloudflare không dùng MySQL và lý do.
Không nên trả lời. Nói "vì quen".

Q30 Vì sao có cả id và public_id?

Trả lời mẫu. id nội bộ để join nhanh; public_id ULID khó đoán dùng ra ngoài, chống IDOR.
Phân tích. Đặc tả 5.3; API trả public_id ở trường id.
Ví dụ trong VAULTRA. 01M6SX1TRC5GMZ7DBQZX5SY6JG.
Lưu ý. IDOR = đoán ID tăng dần để xem dữ liệu người khác; ULID khó đoán và có thứ tự thời gian.
Không nên trả lời. "Để cho đẹp."

Q31 Làm sao chống một kho có hai phân bổ?

Trả lời mẫu. Cột sinh open_unit_key = IF(status='ACTIVE', unit_id, NULL) + UNIQUE.
Phân tích. MySQL cho nhiều NULL nên chỉ 1 allocation ACTIVE/kho, kể cả khi code sai.
Ví dụ trong VAULTRA. SQL 4.5 số 2 phải trả 0 dòng.
Lưu ý. Đây là câu "ăn điểm".
Không nên trả lời. Nói chỉ kiểm tra ở code.

Q32 Tiền lưu kiểu gì? Vì sao không dùng float?

Trả lời mẫu. DECIMAL(15,2); tính bằng decimal.js.
Phân tích. Float sai số nhị phân (0.1+0.2); làm tròn đến đồng ROUND_HALF_UP ở từng dòng.
Ví dụ trong VAULTRA. CHECK total = subtotal − discount + tax_total.
Lưu ý. Có thể minh họa 0.1 + 0.2 = 0.30000000000000004 trong JavaScript.
Không nên trả lời. "Dùng number của JavaScript là đủ."

Q33 Có xóa dữ liệu không?

Trả lời mẫu. Không xóa dữ liệu tiền, hợp đồng, biên bản, nhật ký; danh mục xóa mềm.
Phân tích. deleted_at; trạng thái CANCELLED/VOID thay cho xóa.
Ví dụ trong VAULTRA. Ticket không xóa, chỉ CANCELLED.
Lưu ý. Liên hệ yêu cầu truy vết tiền và nhật ký kiểm toán.
Không nên trả lời. "Có, xóa cho gọn database."

Q34 Lịch sử thay đổi lưu ở đâu?

Trả lời mẫu. Các bảng *_status_history, ticket_events, task_events, audit_logs.
Phân tích. Ghi cùng giao dịch với thay đổi.
Ví dụ trong VAULTRA. unit_status_history có from/to/reason/actor.
Lưu ý. Nêu một bảng cụ thể và cột reason để thấy có lý do thay đổi.
Không nên trả lời. "Không lưu, chỉ lưu trạng thái hiện tại."

Q35 Số dư khách hàng tính thế nào?

Trả lời mẫu. Tổng customer_ledger_entries (nợ dương, có âm).
Phân tích. Không lưu số dư rời rạc không đối chiếu được.
Ví dụ trong VAULTRA. Khách trả thừa 100.000 ₫ → dòng có âm 100.000 ₫, lần sau tự trừ vào hóa đơn.
Lưu ý. Xem qua GET /customers/:id/ledger.
Không nên trả lời. "Có một cột balance tự cộng trừ."

Q36 Có bao nhiêu bảng, chia nhóm thế nào?

Trả lời mẫu. 104 bảng, 6 nhóm: tài khoản, cơ sở/kho, đặt thuê/hợp đồng, tiền, vận hành/hỗ trợ, hệ thống.
Phân tích. schema.prisma; đặc tả mục 10.
Ví dụ trong VAULTRA. Nhóm tiền có 21 bảng theo đặc tả mục 10.5.
Lưu ý. Không cần thuộc hết; nhớ 10 bảng chính của Flow mình phụ trách.
Không nên trả lời. Đọc thuộc lòng danh sách 104 bảng.

7.5 Backend

Q37 Vì sao chọn NestJS?

Trả lời mẫu. Cấu trúc module, dependency injection, decorator cho phân quyền và validate; TypeScript dùng chung với frontend.
Phân tích. Modular monolith: một tiến trình, nhiều module, chung DB để giao dịch tiền nhất quán.
Ví dụ trong VAULTRA. reservation, checkin, contract, billing… mỗi module một thư mục.
Lưu ý. So sánh ngắn với Express: NestJS có sẵn cấu trúc, phù hợp dự án nhiều module.
Không nên trả lời. "Vì nó phổ biến."

Q38 Business logic nằm ở đâu?

Trả lời mẫu. Trong các *.service.ts; controller chỉ validate và gọi service.
Phân tích. Máy trạng thái và tham số chính sách ở packages/shared để frontend dùng chung.
Ví dụ trong VAULTRA. moveout.service.ts estimate().
Lưu ý. Mở một service cụ thể khi được hỏi, ví dụ checkin.service.ts.
Không nên trả lời. Nói logic nằm ở frontend.

Q39 Validate dữ liệu đầu vào thế nào?

Trả lời mẫu. Zod schema ở từng endpoint; sai trả 400 VALIDATION_FAILED kèm details.
Phân tích. parse(z.object(...), body).
Ví dụ trong VAULTRA. OTP phải đúng 6 ký tự.
Lưu ý. Validate ở backend là bắt buộc; validate ở form chỉ để báo lỗi sớm.
Không nên trả lời. "Validate ở giao diện là đủ."

Q40 Job nền gồm những gì?

Trả lời mẫu. 16 job: hết hạn giữ chỗ, hóa đơn kỳ, quá hạn, gia hạn, tự chốt quyết toán, SLA, kích hoạt chính sách, số liệu ngày...
Phân tích. jobs.service.ts; idempotent; Node chạy bằng timer, Worker chạy từ cron mỗi 5 phút.
Ví dụ trong VAULTRA. Lượt cron đầu tiên trên bản demo (08/11/2026): expire-holds xử lý 7 đơn, appointment.reminders 7, workforce.overdue-tasks 15.
Lưu ý. Nêu giới hạn.
Không nên trả lời. "Không cần job, người dùng tự bấm."

Q41 Outbox là gì, vì sao cần?

Trả lời mẫu. Ghi sự kiện cùng giao dịch, gửi sau; giao dịch hủy thì sự kiện cũng mất.
Phân tích. events.service.ts, bảng outbox_events; tránh gửi thông báo cho thao tác thất bại.
Ví dụ trong VAULTRA. Hoàn tất check-in ghi sự kiện cùng giao dịch; nếu giao dịch lỗi, khách không nhận thông báo "đã nhận kho" sai.
Lưu ý. Bộ điều phối dùng SKIP LOCKED để nhiều instance không gửi trùng.
Không nên trả lời. "Gửi thông báo ngay trong code là được."

Q42 Idempotency được cài ở đâu?

Trả lời mẫu. common/idempotency.interceptor.ts + decorator @Idempotent(); bảng idempotency_keys.
Phân tích. Cùng key + cùng body → trả lại kết quả cũ; khác body → 409.
Ví dụ trong VAULTRA. POST /reservations, /payments/offline.
Lưu ý. Header Idempotency-Key bắt buộc cho các API tạo tiền (đặc tả 7.2).
Không nên trả lời. "Frontend khóa nút là đủ."

Q43 Email gửi thế nào?

Trả lời mẫu. Thông báo xếp hàng notification_deliveries rồi bộ gửi gửi; OTP gửi ngay; thử lại có giãn cách.
Phân tích. notification.service.ts; Worker dùng SMTP qua cloudflare:sockets.
Ví dụ trong VAULTRA. Đặt kho xong: email "Xác nhận đặt kho" vào hàng đợi; mã OTP gia hạn gửi ngay.
Lưu ý. Lịch sử gửi xem ở notification_deliveries.
Không nên trả lời. "Gửi email đồng bộ trong mỗi request."

7.6 Frontend

Q44 Cấu trúc frontend?

Trả lời mẫu. React + Vite; mỗi vai trò một thư mục pages và file routes.tsx.
Phân tích. App.tsx định tuyến theo không gian làm việc; lib/api.ts, lib/auth.ts, lib/realtime.ts.
Ví dụ trong VAULTRA. /app, /staff, /manage, /ops, /admin.
Lưu ý. Biết đường dẫn tới màn hình mình demo.
Không nên trả lời. "Một file App lớn."

Q45 Quản lý dữ liệu từ server thế nào?

Trả lời mẫu. TanStack Query: cache, tải lại sau thao tác (invalidate), realtime kích hoạt tải lại.
Phân tích. useGet/useAct trong lib/hooks.ts.
Ví dụ trong VAULTRA. Đổi PIN xong, danh sách credential tự tải lại.
Lưu ý. Realtime dùng chung cơ chế invalidate.
Không nên trả lời. "Mỗi màn hình tự gọi fetch rồi lưu state."

Q46 Frontend ẩn nút thì đủ bảo mật chưa?

Trả lời mẫu. Chưa; backend kiểm tra lại mọi quyền.
Phân tích. Đặc tả 12.8: frontend chỉ cải thiện trải nghiệm.
Ví dụ trong VAULTRA. Gọi Postman bằng token khách → 403/404.
Lưu ý. Chuẩn bị sẵn một request Postman để chứng minh.
Không nên trả lời. "Đủ vì người dùng không thấy nút".

Q47 Nếu một màn hình lỗi thì sao?

Trả lời mẫu. ErrorBoundary hiển thị thẻ lỗi có nút Thử lại, không trắng cả trang.
Phân tích. ui/ErrorBoundary.tsx bọc App.
Ví dụ trong VAULTRA. Sơ đồ kho từng trắng trang khi di chuột vào kho có khách; nay chỉ hiện thẻ lỗi (và lỗi gốc đã sửa).
Lưu ý. Nhắc là đã sửa cả nguyên nhân, không chỉ che lỗi.
Không nên trả lời. "Không bao giờ lỗi."

Q48 Trạng thái tải dữ liệu hiển thị thế nào?

Trả lời mẫu. Skeleton/spinner khi tải, thông báo lỗi + Thử lại khi lỗi, trạng thái rỗng khi không có dữ liệu.
Phân tích. Component Async trong ui/index.tsx.
Ví dụ trong VAULTRA. Danh sách hợp đồng trống hiện "Chưa có kho đang thuê" kèm hướng dẫn.
Lưu ý. Nếu tải chậm trên bản demo, giải thích độ trễ của Worker.
Không nên trả lời. "Màn hình trắng nghĩa là đang tải."

Q49 Có hỗ trợ đa ngôn ngữ không?

Trả lời mẫu. Có vi/en (lib/i18n.ts), đồng bộ với PATCH /me.
Phân tích. Chuỗi giao diện tách khỏi mã; ngôn ngữ người dùng lưu ở hồ sơ.
Ví dụ trong VAULTRA. Nút VI/EN trên thanh đầu trang.
Lưu ý. Nội dung hợp đồng/PDF hiện bằng tiếng Việt.
Không nên trả lời. "Có dịch tự động toàn bộ."

7.7 Authentication và Authorization

Q50 Đăng nhập hoạt động thế nào?

Trả lời mẫu. Email + mật khẩu băm Argon2; trả access JWT ngắn hạn và refresh token.
Phân tích. auth.service.ts; giới hạn tần suất; khóa tài khoản khi sai nhiều.
Ví dụ trong VAULTRA. Sai mật khẩu nhiều lần → tài khoản bị khóa tạm; đăng nhập quá 10 lần/phút → 429.
Lưu ý. Lịch sử đăng nhập lưu ở login_history, khách xem được của mình.
Không nên trả lời. "Mật khẩu so sánh trực tiếp chuỗi."

Q51 Access token và refresh token khác nhau?

Trả lời mẫu. Access ngắn hạn gửi mỗi request; refresh dài hạn dùng để lấy access mới.
Phân tích. Refresh xoay vòng; dùng lại token cũ → thu hồi cả phiên (TOKEN_REUSE).
Ví dụ trong VAULTRA. Access hết hạn sau vài phút, trình duyệt tự gọi /auth/refresh rồi gửi lại request.
Lưu ý. Phát hiện dùng lại refresh token cũ là dấu hiệu bị đánh cắp.
Không nên trả lời. "Một token dùng mãi."

Q52 Token lưu ở đâu?

Trả lời mẫu. Access trong bộ nhớ; refresh trong cookie httpOnly và có bản dự phòng trong localStorage.
Phân tích. Bản dự phòng localStorage là điểm yếu đã ghi nhận (SEC-10) vì Brave/Safari chặn cookie bên thứ ba.
Ví dụ trong VAULTRA. Đăng nhập Google trên Brave cần bản dự phòng vì cookie bên thứ ba bị chặn.
Lưu ý. Nói thật điểm yếu và lý do.
Không nên trả lời. "Lưu hết trong localStorage là an toàn."

Q53 Google OAuth hoạt động thế nào?

Trả lời mẫu. Authorization Code + PKCE, state ký HMAC trong cookie 10 phút.
Phân tích. googleStart/googleCallback; email trùng tài khoản thường phải xác nhận liên kết.
Ví dụ trong VAULTRA. Bấm "Đăng nhập với Google" → trang Google → quay về /auth/callback → vào tài khoản.
Lưu ý. Lỗi đăng nhập Google trên bản demo từng do phiên không được lưu, đã sửa 08/11/2026.
Không nên trả lời. "Lấy mật khẩu Google của người dùng."

Q54 RBAC kết hợp phạm vi thế nào?

Trả lời mẫu. Quyền × phạm vi (OWN/FACILITY/GLOBAL); AccessContext gắn vào mỗi request.
Phân tích. @RequirePermission + ctx.assertScope/ctx.scope.
Ví dụ trong VAULTRA. staff.hn có quyền checkin.perform nhưng chỉ trên cơ sở Cầu Giấy.
Lưu ý. Hai trục: được làm gì × trên dữ liệu nào.
Không nên trả lời. "Chỉ cần kiểm tra vai trò."

Q55 Vì sao ngoài phạm vi trả 404 mà không phải 403?

Trả lời mẫu. Để không lộ bản ghi có tồn tại.
Phân tích. Đặc tả 5.3.
Ví dụ trong VAULTRA. Khách A mở hợp đồng của khách B → "Không tồn tại hoặc ngoài phạm vi truy cập".
Lưu ý. 403 vẫn dùng khi thiếu quyền chức năng.
Không nên trả lời. "Vì lỗi."

Q56 Thu hồi quyền có hiệu lực ngay không?

Trả lời mẫu. Có: thu hồi refresh token và đẩy session.revoked qua WebSocket.
Phân tích. realtime.hub revokeUser; frontend tự đăng xuất.
Ví dụ trong VAULTRA. Admin khóa tài khoản nhân viên → tab nhân viên đang mở tự đăng xuất.
Lưu ý. Cần kết nối realtime; nếu mất kết nối thì lần gọi API kế tiếp bị từ chối.
Không nên trả lời. "Phải đợi token hết hạn."

Q57 Nhân viên có tự xử lý tiền trên hợp đồng của chính mình không?

Trả lời mẫu. Đặc tả cấm (actor ≠ chủ hợp đồng).
Phân tích. Nếu chưa kiểm tra mã nguồn chỗ này, nói "đặc tả yêu cầu, em xin kiểm tra lại cài đặt".
Ví dụ trong VAULTRA. Nhân viên kiêm khách thuê kho ở chính cơ sở mình.
Lưu ý. Trung thực.
Không nên trả lời. Khẳng định đã cài khi chưa kiểm tra.

7.8 API

Q58 Quy ước API?

Trả lời mẫu. REST JSON dưới /v1; lỗi thống nhất {error:{code,message,details,requestId}}.
Phân tích. Mã HTTP theo đặc tả 7.5.
Ví dụ trong VAULTRA. POST /v1/reservations sai body → {"error":{"code":"VALIDATION_FAILED",...}}.
Lưu ý. Nhớ requestId dùng để tra log.
Không nên trả lời. "Mỗi API trả lỗi một kiểu."

Q59 Có tài liệu API không?

Trả lời mẫu. Bản Node có OpenAPI: GET /v1/openapi.json và /v1/docs.
Phân tích. platform.controller.ts.
Ví dụ trong VAULTRA. Mở /v1/docs trên bản Node để xem danh sách endpoint.
Lưu ý. Kiểm tra trên bản Node trước khi nói có trên bản demo.
Không nên trả lời. "Không có tài liệu."

Q60 Mã 409 dùng khi nào?

Trả lời mẫu. Xung đột nghiệp vụ: sai trạng thái, hết kho, thiếu điều kiện, dữ liệu cũ.
Phân tích. INVALID_STATUS_TRANSITION, UNIT_TYPE_SOLD_OUT, CHECKIN_PRECONDITION_FAILED...
Ví dụ trong VAULTRA. Cấp PIN khi chưa ký → 409 CHECKIN_PRECONDITION_FAILED.
Lưu ý. Phân biệt 400 (dữ liệu sai), 403 (thiếu quyền), 404 (ngoài phạm vi), 409 (xung đột nghiệp vụ).
Không nên trả lời. "409 là lỗi máy chủ."

Q61 Có giới hạn tần suất không?

Trả lời mẫu. Có; đăng nhập 10 lần/phút/IP trên bản demo; header RateLimit-*.
Phân tích. rate-limiter.ts; vượt trả 429 RATE_LIMITED.
Ví dụ trong VAULTRA. Chạy test e2e liên tục trên bản demo gặp 429 do giới hạn đăng nhập.
Lưu ý. Retry-After cho biết bao lâu thử lại.
Không nên trả lời. "Không giới hạn."

Q62 Webhook/IPN VNPay xử lý thế nào?

Trả lời mẫu. Kiểm chữ ký HMAC-SHA512, tìm payment theo txnRef, so số tiền, settle() idempotent.
Phân tích. RspCode 00/02/97/01/04/99.
Ví dụ trong VAULTRA. IPN có hash sai → RspCode 97, đơn vẫn chờ thanh toán.
Lưu ý. Trang return chỉ để hiển thị; IPN mới là nguồn sự thật.
Không nên trả lời. "Tin trang return."

Q63 Upload tệp qua API thế nào?

Trả lời mẫu. Xin URL ký → PUT tệp → complete để kiểm tra → READY.
Phân tích. media.service.ts.
Ví dụ trong VAULTRA. Ảnh hiện trạng check-in đi đủ 3 bước; lưu biên bản khi ảnh chưa READY bị báo "Tệp chưa tải lên xong".
Lưu ý. Tệp lớn không đi qua JSON; URL ký có hạn.
Không nên trả lời. "Gửi base64 trong body."

7.9 Bảo mật

Q64 Chống SQL injection thế nào?

Trả lời mẫu. Dùng Prisma với tham số hóa; raw query dùng template có tham số.
Phân tích. $queryRaw`... ${x}` tham số hóa.
Ví dụ trong VAULTRA. Ô tìm kiếm hợp đồng nhận chuỗi lạ vẫn chỉ là tham số, không ghép vào câu SQL.
Lưu ý. Có thể minh họa bằng tìm kiếm chứa dấu nháy đơn.
Không nên trả lời. "Dùng MySQL nên không bị."

Q65 Chống XSS?

Trả lời mẫu. React tự escape; SVG tải lên không hiển thị inline, có CSP sandbox.
Phân tích. platform.controller.ts getBlob đặt CSP và Content-Disposition cho SVG.
Ví dụ trong VAULTRA. Tên khách chứa thẻ <script> hiển thị như chữ thường.
Lưu ý. Không dùng dangerouslySetInnerHTML cho dữ liệu người dùng.
Không nên trả lời. "Trình duyệt tự chặn."

Q66 Mật khẩu lưu thế nào?

Trả lời mẫu. Băm Argon2 (memoryCost 19456, timeCost 2).
Phân tích. Argon2 là hàm băm chậm có salt, chống dò hàng loạt.
Ví dụ trong VAULTRA. Cột users.password_hash chỉ chứa chuỗi băm.
Lưu ý. Không ai, kể cả admin, xem được mật khẩu gốc; chỉ đặt lại.
Không nên trả lời. Nói mã hóa có thể giải ngược.

Q67 Dữ liệu nhạy cảm?

Trả lời mẫu. PIN, số giấy tờ mã hóa AES-256-GCM; BOM xem PII dạng che.
Phân tích. util.ts encryptField/maskPhone/maskEmail; có unit test.
Ví dụ trong VAULTRA. Báo cáo BOM hiện số điện thoại 09******12.
Lưu ý. Khóa mã hóa đặt bằng secret FIELD_ENCRYPTION_KEY, không nằm trong mã.
Không nên trả lời. "Lưu thô cho tiện."

Q68 Chống dò mật khẩu?

Trả lời mẫu. Giới hạn tần suất + khóa tài khoản theo số lần sai.
Phân tích. audit-findings SEC-05.
Ví dụ trong VAULTRA. Đăng nhập sai nhiều lần liên tiếp → tài khoản bị khóa tạm thời.
Lưu ý. Có thêm Turnstile (tùy chọn) chống bot.
Không nên trả lời. "Không cần vì mật khẩu dài."

Q69 Có bí mật nào lộ trong mã nguồn không?

Trả lời mẫu. Có điểm cần sửa: client secret Google và khóa Gemini ghi cứng trong apps/gateway/src/shims/config.ts; mật khẩu demo chung.
Phân tích. RISK-04/05 ở Chương 8; cần chuyển sang secret của Cloudflare và đổi khóa.
Ví dụ trong VAULTRA. File config của gateway có client secret Google và khóa Gemini mặc định.
Lưu ý. Nói thật nếu bị hỏi; đừng mở file đó khi trình chiếu.
Không nên trả lời. Phủ nhận.

Q70 Chống CSRF?

Trả lời mẫu. API dùng Bearer token trong header, không dựa vào cookie cho request nghiệp vụ; CORS chỉ cho origin trong danh sách.
Phân tích. origin.ts isAllowedOrigin.
Ví dụ trong VAULTRA. Trang web lạ không gửi được header Authorization của người dùng.
Lưu ý. Refresh token cookie SameSite=None nên CORS danh sách trắng là quan trọng.
Không nên trả lời. "Không cần quan tâm."

Q71 Tải tệp độc hại?

Trả lời mẫu. Kiểm magic bytes, giới hạn kích thước theo mục đích, SHA-256.
Phân tích. Không tin phần mở rộng hay Content-Type do client gửi; đọc byte đầu để xác định loại tệp thật.
Ví dụ trong VAULTRA. Tệp .exe đổi đuôi .png bị từ chối khi complete.
Lưu ý. SVG tải về được ép tải xuống thay vì hiển thị.
Không nên trả lời. "Kiểm tra đuôi file là đủ."

7.10 Testing

Q72 Nhóm kiểm thử những gì?

Trả lời mẫu. Unit test công thức (16), e2e từng Flow, test đồng thời, test bảo mật, test case thủ công.
Phân tích. Chương 4.
Ví dụ trong VAULTRA. Unit test 16/16 đạt; f3-moveout 15/15; nfr03 5/5 (08/11/2026).
Lưu ý. Nêu kết quả thật kèm ngày.
Không nên trả lời. "Test hết rồi".

Q73 Làm sao chứng minh chức năng chạy đúng?

Trả lời mẫu. Chạy test tự động trước hội đồng hoặc cho xem log kết quả; đối chiếu DB bằng SQL.
Phân tích. pnpm --filter @vaultra/api test (≈2 giây).
Ví dụ trong VAULTRA. Đặt kho trên web → GET /reservations/:id → SQL bảng reservations cùng trạng thái.
Lưu ý. Chuẩn bị sẵn lệnh và cửa sổ terminal.
Không nên trả lời. "Vì demo không lỗi."

Q74 Unit test và e2e khác nhau?

Trả lời mẫu. Unit kiểm một hàm; e2e đi trọn luồng qua API và DB.
Phân tích. Unit nhanh, cô lập, chạy mọi lúc; e2e chậm hơn, cần môi trường, bắt lỗi ghép nối.
Ví dụ trong VAULTRA. Unit: tính phí trễ; e2e: đặt kho → thanh toán → check-in.
Lưu ý. Hai lớp bổ sung nhau.
Không nên trả lời. "Giống nhau."

Q75 Test đồng thời làm thế nào?

Trả lời mẫu. Gửi 100 request đặt kho cuối cùng cùng lúc, đếm kết quả.
Phân tích. nfr03-concurrency.ts; kết quả 1 ok, 99 sold-out.
Ví dụ trong VAULTRA. Kết quả: 100 request trong 970 ms → 1 thành công, 99 hết kho.
Lưu ý. Bằng chứng mạnh nhất cho cam kết không bán trùng.
Không nên trả lời. "Mở hai tab bấm cùng lúc."

Q76 Có test giao diện tự động không?

Trả lời mẫu. Chưa; giao diện kiểm thử thủ công.
Phân tích. Kiểm thử giao diện thủ công theo test case Chương 4 và kịch bản demo.
Ví dụ trong VAULTRA. Chưa có Playwright/Cypress trong package.json.
Lưu ý. Thừa nhận và nêu hướng (Playwright).
Không nên trả lời. "Có, test hết rồi."

Q77 Test case nào thất bại gần đây?

Trả lời mẫu. Trên Worker: 2 kiểm tra idempotency f1 (do dữ liệu cục bộ hết chỗ) và 7 kiểm tra f2 (OTP đọc hộp thư giả).
Phân tích. Nguyên nhân do môi trường test, chưa xác minh lại trên MySQL.
Ví dụ trong VAULTRA. f2-checkin: "OTP found in mail sink" lỗi vì Worker gửi email thật.
Lưu ý. Nói rõ nguyên nhân môi trường và kế hoạch chạy lại trên MySQL.
Không nên trả lời. Giấu kết quả thất bại.

Q78 Regression làm thế nào?

Trả lời mẫu. Chạy lại toàn bộ bộ test sau mỗi thay đổi lớn (ví dụ bỏ payOS).
Phân tích. Sau mỗi thay đổi có rủi ro, chạy lại unit + e2e liên quan.
Ví dụ trong VAULTRA. Sau khi bỏ payOS, test thanh toán chuyển sang mô phỏng IPN VNPay.
Lưu ý. Ghi kết quả từng lần chạy.
Không nên trả lời. "Chỉ test chỗ mới sửa."

7.11 Kiến trúc hệ thống

Q79 Vì sao modular monolith, không microservices?

Trả lời mẫu. Quy mô đồ án; giao dịch tiền cần chung DB; dễ triển khai và kiểm thử.
Phân tích. Module tách rõ, có thể tách dịch vụ sau.
Ví dụ trong VAULTRA. Hoàn tất check-in cập nhật hợp đồng, kho, đơn, lịch hẹn trong một giao dịch — microservices sẽ cần saga phức tạp.
Lưu ý. Ranh giới module đã rõ nên tách sau được.
Không nên trả lời. "Microservices luôn tốt hơn."

Q80 Bản Worker khác bản Node thế nào?

Trả lời mẫu. Cùng mã nghiệp vụ; Worker thay DB bằng edge-prisma (bộ nhớ + R2), thay email/storage bằng shim.
Phân tích. apps/gateway; wrangler.toml alias.
Ví dụ trong VAULTRA. Worker không có MySQL; dữ liệu lưu thành file JSON trên R2.
Lưu ý. Bằng chứng giao dịch/khóa mạnh nhất lấy từ bản Node.
Không nên trả lời. "Giống hệt nhau."

Q81 Realtime hoạt động thế nào?

Trả lời mẫu. WebSocket; vé ký 60 giây; server đẩy data.changed sau mỗi thao tác ghi.
Phân tích. Node: ws hub; Worker: Durable Object FacilityHub.
Ví dụ trong VAULTRA. FM tạo việc → cửa sổ nhân viên hiện việc mới không cần F5.
Lưu ý. Tín hiệu không chứa dữ liệu, chỉ báo "có thay đổi".
Không nên trả lời. "Trình duyệt tự hỏi máy chủ mỗi giây."

Q82 Mở rộng khi nhiều cơ sở?

Trả lời mẫu. Dữ liệu đã phân theo facility_id; có thể nhiều instance nhờ khóa DB và outbox SKIP LOCKED.
Phân tích. Đặc tả 18.7.
Ví dụ trong VAULTRA. Thêm cơ sở mới chỉ là thêm dữ liệu facilities, không sửa mã.
Lưu ý. Bản demo Worker chưa sẵn sàng nhiều instance (ARCH-01).
Không nên trả lời. "Mua máy chủ mạnh hơn."

Q83 Điểm yếu kiến trúc lớn nhất?

Trả lời mẫu. Bản demo Worker lưu dữ liệu trong bộ nhớ + snapshot R2: nhiều instance có thể ghi đè nhau.
Phân tích. ARCH-01; đã thêm khóa trong một instance, chưa chặn được giữa các instance.
Ví dụ trong VAULTRA. Đã thấy trường hợp ảnh mất trạng thái READY và phiên Google mất do snapshot; đã sửa trong một instance.
Lưu ý. Tự nêu kèm hướng khắc phục (D1/Durable Object/MySQL).
Không nên trả lời. "Không có điểm yếu."

Q84 Vì sao ảnh lưu R2?

Trả lời mẫu. Lưu trữ đối tượng rẻ, có URL ký; không làm phình DB.
Phân tích. Database chỉ lưu metadata và khóa đối tượng; tệp nằm ở kho đối tượng.
Ví dụ trong VAULTRA. media_files lưu object_key, sha256, mime_type.
Lưu ý. Ảnh riêng tư tải bằng URL ký có hạn.
Không nên trả lời. "Lưu ảnh trong database."

7.12 Công nghệ sử dụng

Q85 Liệt kê stack chính?

Trả lời mẫu. React 18, Vite, Tailwind, TanStack Query, Zustand; NestJS 11, Prisma 7, MySQL 8, Zod, jose, Argon2, decimal.js, pdfkit; Cloudflare Pages/Worker/R2/Durable Object; VNPay sandbox.
Phân tích. package.json của apps/web, apps/api.
Ví dụ trong VAULTRA. Mở apps/api/package.json khi được hỏi phiên bản.
Lưu ý. Chỉ nói công nghệ có trong package.json.
Không nên trả lời. Nói Redis/Kafka (không dùng).

Q86 Vì sao Prisma?

Trả lời mẫu. Schema rõ, type-safe, migration; vẫn viết raw SQL khi cần FOR UPDATE.
Phân tích. Kiểu dữ liệu sinh từ schema giúp phát hiện lỗi khi biên dịch.
Ví dụ trong VAULTRA. Truy vấn khóa hàng FOR UPDATE viết bằng $queryRaw.
Lưu ý. Prisma không thay thế được hiểu biết SQL.
Không nên trả lời. "Để khỏi viết SQL."

Q87 Vì sao TanStack Query?

Trả lời mẫu. Cache, retry, invalidate; giảm code tải dữ liệu.
Phân tích. Tách trạng thái server khỏi trạng thái giao diện.
Ví dụ trong VAULTRA. useGet("/contracts") cache theo đường dẫn.
Lưu ý. Realtime chỉ cần invalidate.
Không nên trả lời. "Vì bắt buộc."

Q88 PDF sinh bằng gì?

Trả lời mẫu. pdfkit với font DejaVu hỗ trợ tiếng Việt; lưu SHA-256.
Phân tích. media/pdf.service.ts.
Ví dụ trong VAULTRA. Hợp đồng CT-... có trang ký, chân trang "Trang x / y".
Lưu ý. Font nhúng để tiếng Việt không lỗi.
Không nên trả lời. "Chụp màn hình thành PDF."

Q89 AI dùng gì?

Trả lời mẫu. Gemini với kho tri thức (RAG) do quản trị biên tập.
Phân tích. assistant.service.ts, kb.service.ts.
Ví dụ trong VAULTRA. Quản trị thêm bài tri thức → chatbot trả lời dựa trên đoạn đó.
Lưu ý. AI không tự thực hiện thao tác tiền.
Không nên trả lời. "AI tự học mọi thứ."

7.13 Xử lý lỗi và ngoại lệ

Q90 Backend lỗi bất ngờ trả gì?

Trả lời mẫu. "Lỗi hệ thống" kèm requestId; chi tiết chỉ ghi log server.
Phân tích. Không lộ stack trace.
Ví dụ trong VAULTRA. Màn hình hiện "Lỗi hệ thống"; tra requestId trong log Worker.
Lưu ý. Lỗi nghiệp vụ có mã riêng, không dùng thông điệp chung.
Không nên trả lời. "Trả nguyên stack trace."

Q91 Thanh toán lỗi?

Trả lời mẫu. Payment FAILED, đơn vẫn chờ, khách thử lại đến hết giờ giữ; khách nhận email thanh toán không thành công.
Phân tích. Payment là bản ghi riêng theo từng lần thử (attempt_no); hóa đơn không đổi.
Ví dụ trong VAULTRA. Thẻ test hết hạn → VNPay trả lỗi → bấm thanh toán lại.
Lưu ý. Tiền không bao giờ được ghi nhận khi chưa có IPN hợp lệ.
Không nên trả lời. "Hủy đơn luôn."

Q92 Email lỗi?

Trả lời mẫu. Ghi FAILED/last_error, bộ gửi thử lại tối đa 5 lần có giãn cách; OTP thử lại ngay 1 lần.
Phân tích. Lỗi gửi không làm hỏng thao tác chính; được ghi lại để thử lại và tra cứu.
Ví dụ trong VAULTRA. OTP gia hạn từng bị SMTP từ chối (535); lần thử sau thành công.
Lưu ý. Lỗi SMTP không còn chép mật khẩu vào nhật ký.
Không nên trả lời. "Email lỗi thì bỏ qua."

Q93 Request quá lâu?

Trả lời mẫu. Frontend dừng sau 40 giây và báo "Máy chủ phản hồi quá chậm".
Phân tích. lib/api.ts.
Ví dụ trong VAULTRA. Nút "Lưu" không quay mãi; sau 40 giây hiện thông báo.
Lưu ý. Có thể bấm lại an toàn nhờ idempotency với API tạo tiền.
Không nên trả lời. "Đợi đến khi xong."

Q94 Hai người sửa cùng bản ghi?

Trả lời mẫu. Khóa lạc quan bằng cột version: người sau nhận 409 STALE_RESOURCE.
Phân tích. Cập nhật kèm WHERE version = ?, lệch thì báo để tải lại dữ liệu mới.
Ví dụ trong VAULTRA. Hai FM cùng sửa một loại kho.
Lưu ý. Khác với khóa hàng dùng cho tiền/kho.
Không nên trả lời. "Người sau ghi đè."

Q95 Chuyển trạng thái sai?

Trả lời mẫu. 409 INVALID_STATUS_TRANSITION theo states.ts.
Phân tích. Bảng chuyển hợp lệ khai báo một chỗ (states.ts), có unit test.
Ví dụ trong VAULTRA. Đổi kho OCCUPIED sang AVAILABLE bị từ chối.
Lưu ý. Đây là chỗ minh họa negative test dễ nhất.
Không nên trả lời. "Giao diện không cho bấm."

7.14 Câu hỏi phản biện chuyên sâu

Q96 Tại sao nhóm chọn cách thiết kế này?

Trả lời mẫu. Bám đề bài và đặc tả; ưu tiên tính đúng của tiền và kho hơn tính năng phụ.
Phân tích. Mỗi quyết định có lý do: phiên bản chính sách (không hồi tố), outbox (không gửi thông báo sai), UNIQUE (chống trùng).
Ví dụ trong VAULTRA. Chính sách có phiên bản thay vì sửa trực tiếp để hợp đồng cũ không bị đổi điều khoản.
Lưu ý. Trả lời theo từng quyết định cụ thể.
Không nên trả lời. "Vì thầy dạy vậy".

Q97 Nếu có nhiều người thao tác cùng lúc thì sao?

Trả lời mẫu. Thao tác tiền/kho chạy trong giao dịch có khóa hàng; sửa chồng dùng version.
Phân tích. Bản Node + MySQL bảo đảm; bản Worker có khóa trong một instance, giữa nhiều instance còn rủi ro (ARCH-01).
Ví dụ trong VAULTRA. nfr03.
Lưu ý. Phân biệt bản Node và bản demo.
Không nên trả lời. "Không bao giờ có chuyện đó."

Q98 Hai người cùng thực hiện một nghiệp vụ trên cùng dữ liệu?

Trả lời mẫu. Người thứ hai nhận 409 (sai trạng thái/dữ liệu cũ) hoặc kết quả cũ (idempotency).
Phân tích. Ví dụ hai nhân viên cùng nhận một task: chỉ một người claim được.
Ví dụ trong VAULTRA. Hai nhân viên cùng bấm "Nhận việc": người thứ hai thấy việc đã có người nhận.
Lưu ý. Nêu cơ chế cụ thể cho từng loại nghiệp vụ.
Không nên trả lời. "Ai bấm sau thì thắng."

Q99 Database lỗi giữa giao dịch?

Trả lời mẫu. Giao dịch rollback, không có trạng thái nửa vời; outbox không phát sự kiện.
Phân tích. Prisma $transaction; edge-prisma có undo log.
Ví dụ trong VAULTRA. Hoàn tất check-in.
Lưu ý. Bản Worker mô phỏng rollback bằng undo log trong edge-prisma.
Không nên trả lời. "Dữ liệu bị lưu một nửa, phải sửa tay."

Q100 Backend lỗi nhưng frontend báo thành công?

Trả lời mẫu. Frontend chỉ báo thành công khi API trả 2xx; lỗi hiện toast.
Phân tích. useAct onSuccess chỉ chạy khi mutation thành công.
Ví dụ trong VAULTRA. Lỗi xóa tài liệu RAG: frontend dùng sai trường id nên API báo không tồn tại — giao diện hiện đúng thông báo lỗi, không báo thành công; đã sửa.
Lưu ý. Nếu từng gặp lỗi kiểu này (ví dụ KB delete dùng sai id), kể và nói đã sửa.
Không nên trả lời. "Không thể xảy ra."

Q101 Người dùng cố gọi API không có quyền?

Trả lời mẫu. 403 thiếu quyền, 404 ngoài phạm vi; ghi log.
Phân tích. Kiểm tra quyền chạy trước mọi logic; phạm vi kiểm tra trong truy vấn.
Ví dụ trong VAULTRA. Postman token khách gọi publish policy.
Lưu ý. Chuẩn bị request Postman minh họa.
Không nên trả lời. "Người dùng không biết API."

Q102 Làm sao tránh dữ liệu trùng?

Trả lời mẫu. Idempotency-Key, UNIQUE (open_unit_key, active_pin_key, mã kho), kiểm tra trùng IPN theo txnId.
Phân tích. Kết hợp ba lớp: chống gửi lặp (idempotency), ràng buộc DB (UNIQUE), kiểm tra nghiệp vụ (giao dịch khóa).
Ví dụ trong VAULTRA. IPN VNPay gửi hai lần → lần hai ghi nhận ALREADY_CONFIRMED.
Lưu ý. Đưa SQL phát hiện trùng ở mục 4.5.
Không nên trả lời. "Kiểm tra bằng mắt."

Q103 Làm sao đảm bảo tính nhất quán?

Trả lời mẫu. Giao dịch, ràng buộc DB, máy trạng thái, sổ cái; SQL đối soát mục 4.5.
Phân tích. Không có cập nhật rời rạc cho dữ liệu liên quan nhau; mọi thứ chung một giao dịch hoặc qua outbox.
Ví dụ trong VAULTRA. Hóa đơn có CHECK tổng tiền; tiền phân bổ khớp số đã trả.
Lưu ý. Chạy SQL đối soát định kỳ.
Không nên trả lời. "Tin vào code."

Q104 Flow bị gián đoạn giữa chừng?

Trả lời mẫu. Trạng thái lưu từng bước (phiên check-in), có thể tiếp tục hoặc hủy; hủy thì trả về trạng thái trước.
Phân tích. Thiết kế theo trạng thái: mỗi bước ghi trạng thái nên tiếp tục được sau sự cố.
Ví dụ trong VAULTRA. Mất mạng ở bước ký: mở lại phiên check-in, bước 1–2 vẫn đã hoàn tất.
Lưu ý. Bước cuối (hoàn tất) là một giao dịch duy nhất.
Không nên trả lời. "Làm lại từ đầu."

Q105 Chức năng chạy trên giao diện nhưng dữ liệu không được lưu?

Trả lời mẫu. Đối chiếu DB bằng SQL/API sau thao tác; bản Worker từng gặp lỗi mất dữ liệu do snapshot và đã sửa.
Phân tích. Ví dụ lỗi ảnh "chưa tải xong" và đăng nhập Google mất phiên do request GET có ghi không được lưu — đã sửa 08/11/2026.
Ví dụ trong VAULTRA. Ảnh check-in từng hiện thumbnail nhưng media vẫn PENDING do snapshot bị ghi đè; phát hiện qua thông báo khi lưu biên bản.
Lưu ý. Trung thực, nêu cách phát hiện và sửa.
Không nên trả lời. "Chưa từng xảy ra."

Q106 Nhóm đã kiểm thử những trường hợp nào?

Trả lời mẫu. Happy path, sai dữ liệu, biên, quyền, trùng request, đồng thời, chữ ký thanh toán giả.
Phân tích. Chương 4 + kết quả test tự động.
Ví dụ trong VAULTRA. 74 test case cho 7 Flow + test tự động.
Lưu ý. Nói số đã chạy thật, không gộp với số đã viết.
Không nên trả lời. "Mọi trường hợp."

Q107 Làm sao chứng minh chức năng thực sự hoạt động đúng?

Trả lời mẫu. Chạy test trước hội đồng, demo trực tiếp, mở DB/SQL đối chiếu, mở mã nguồn đoạn xử lý.
Phân tích. Bằng chứng độc lập với lời nói: test chạy lại được, dữ liệu kiểm tra được.
Ví dụ trong VAULTRA. Chạy pnpm test trước hội đồng mất khoảng 2 giây.
Lưu ý. Ba lớp bằng chứng: test, demo, dữ liệu.
Không nên trả lời. "Tin em đi".

7.15 Câu hỏi riêng cho từng thành viên

Q108 (Khang) Nếu khách không có thẻ ngân hàng?

Trả lời mẫu. Nhân viên đặt hộ tại quầy và ghi nhận tiền mặt/chuyển khoản (POST /payments/offline).
Phân tích. Cùng luồng đặt kho, khác bước thanh toán; vẫn có hóa đơn, biên nhận.
Ví dụ trong VAULTRA. Nhân viên tạo hồ sơ khách tại quầy, thu tiền mặt, in biên nhận.
Lưu ý. Ghi nhận tiền mặt cũng bắt buộc Idempotency-Key.
Không nên trả lời. "Không phục vụ được."

Q109 (Triển) Nếu khách từ chối ký hợp đồng tại quầy?

Trả lời mẫu. Hủy phiên lý do CONTRACT_DECLINED, xử lý như hủy đơn theo chính sách hủy.
Phân tích. Phiên check-in không được hoàn tất nửa chừng; đơn quay về trạng thái trước và áp chính sách hủy.
Ví dụ trong VAULTRA. Bấm "Hủy phiên nhận kho" chọn lý do "Khách từ chối hợp đồng".
Lưu ý. Không cấp mã truy cập vì hợp đồng chưa SIGNED.
Không nên trả lời. "Vẫn giao kho, ký sau."

Q110 (Phúc) Nếu BOM đặt phí trễ 0 thì sao?

Trả lời mẫu. Chính sách cho phép nếu schema hợp lệ; mốc khóa vẫn chạy; nên kiểm tra validate miền giá trị trước khi trả lời.
Phân tích. Tham số là dữ liệu; schema chỉ chặn giá trị sai kiểu/miền, quyết định kinh doanh thuộc BOM.
Ví dụ trong VAULTRA. late_fee_fixed = 0 và late_fee_pct = 0 → không phát sinh dòng LATE_FEE nhưng mốc khóa D+10 vẫn áp dụng.
Lưu ý. Mở policyParamsSchema để kiểm tra miền giá trị trước khi trả lời chắc chắn.
Không nên trả lời. "Hệ thống bị lỗi."

Q111 (Trường) Nhân viên nghỉ đột xuất, việc của họ thì sao?

Trả lời mẫu. FM chuyển hàng loạt việc (POST /tasks/bulk-reassign); bị gỡ khỏi cơ sở thì việc về OPEN.
Phân tích. Việc gắn người làm qua assignee_id; chuyển hàng loạt trong một thao tác và ghi task_events.
Ví dụ trong VAULTRA. FM chọn nhân viên nghỉ và người thay trong màn hình Công việc.
Lưu ý. Nhân viên mới phải thuộc cùng cơ sở.
Không nên trả lời. "Để việc đó quá hạn."

CHƯƠNG 8

Phân tích lỗi và rủi ro

> Chỉ liệt kê vấn đề có bằng chứng trong mã nguồn, dữ liệu hoặc lần chạy thử; mỗi mục ghi nơi kiểm chứng. Đây là những điểm giảng viên có thể phát hiện — nhóm nên chủ động nắm trước.

8.1 Bảng tổng hợp rủi ro

ID	Vấn đề	Nhóm lỗi	Severity	Bằng chứng
RISK-01	Dữ liệu bản demo nằm trong bộ nhớ Worker + snapshot R2	Lỗi xử lý dữ liệu đồng thời / transaction	High	apps/gateway/src/edge-prisma.ts, apps/gateway/src/index.ts (withLock, reloadFromR2)
RISK-02	Job định kỳ trên bản Worker (đã sửa 08/11/2026)	Logic nghiệp vụ / trạng thái	Low (đã sửa)	apps/gateway/src/index.ts (runCron), apps/gateway/src/edge-app.ts (runDueJobs), log cron 08/11/2026
RISK-03	Refresh token có bản sao trong localStorage	Lỗi bảo mật	Medium	AUDIT_REPORT.md SEC-10; apps/web/src/lib/api.ts
RISK-04	Client secret Google ghi cứng trong mã gateway	Lỗi bảo mật	High	apps/gateway/src/shims/config.ts (applyEnv)
RISK-05	Khóa Gemini mặc định trong mã gateway	Lỗi bảo mật	High	apps/gateway/src/shims/config.ts (defaultGeminiKeys); AUDIT_REPORT SEC-03
RISK-06	Mật khẩu chung cho tài khoản demo	Lỗi bảo mật	Medium	AUDIT_REPORT SEC-12; apps/api/prisma/seed-demo.ts
RISK-07	Đối soát VNPay trên bản demo là mô phỏng	Frontend–backend không đồng bộ (dữ liệu bên ngoài)	Medium	apps/api/src/billing/billing.controller.ts (reconRun), payment.service.ts (fetchVnpayStatement)
RISK-08	SMTP đôi lúc từ chối đăng nhập (535)	API không xử lý lỗi / tích hợp ngoài	Medium	Dữ liệu deliveries #460, #461 (08/11/2026); notification.service.ts sendDirect
RISK-09	Dữ liệu mẫu của bản demo từng thiếu liên kết	Database không lưu đúng dữ liệu	Low (đã sửa)	apps/gateway/src/edge-prisma.ts repairIntegrity
RISK-10	Không tua thời gian được trên bản deploy	Trạng thái chuyển đổi (khó demo)	Medium	apps/gateway/src/shims/config.ts (testClock)
RISK-11	Một số API GET có ghi dữ liệu	Thiết kế API	Low	AUDIT_REPORT API-03
RISK-12	Mở/đóng kho từ xa là mô phỏng	Phạm vi	Low	apps/api/src/access/access.service.ts remoteUnlock/remoteLock
RISK-13	E2E chưa chạy lại sau các thay đổi gần nhất	Quy trình kiểm thử	Medium	Mục 4.6
RISK-14	Độ trễ API trên bản demo 0,7–1,3 giây	Hiệu năng	Low	Đo 08/11/2026
RISK-15	Realtime chỉ bật sau khi đăng nhập	Đồng bộ sau cập nhật	Low	apps/web/src/lib/realtime.ts

8.2 Chi tiết từng rủi ro

RISK-01 — Dữ liệu bản demo nằm trong bộ nhớ Worker + snapshot R2 High

1. Mô tả	Bản Cloudflare không dùng MySQL. Mỗi instance Worker giữ một bản dữ liệu trong bộ nhớ, nạp từ và ghi về một file JSON trên R2.
2. Nguyên nhân	Cloudflare không có MySQL managed; nhóm chọn mô phỏng DB để có bản demo công khai (ARCH-01 trong AUDIT_REPORT.md).
3. Mức độ nghiêm trọng	High
4. Cách phát hiện	Hai thao tác ghi gần như cùng lúc trên hai instance khác nhau; thao tác ghi xong nhưng dữ liệu "quay lại" sau vài giây.
5. Tái hiện	Khó tái hiện chủ động; đã từng thấy ảnh check-in mất trạng thái READY và phiên Google bị mất (08/11/2026).
6. Ảnh hưởng	Có thể mất một thay đổi khi tải cao; các bảo đảm khóa hàng của MySQL không áp dụng đầy đủ.
7. Khắc phục	Đã thêm khóa tuần tự cho thao tác ghi trong một instance và chỉ nạp lại khi R2 thay đổi (etag). Hướng triệt để: chuyển dữ liệu sang Durable Object/D1 hoặc chạy bản Node + MySQL.
8. Trả lời khi bị phát hiện	*"Đúng ạ, bản demo công khai dùng cơ sở dữ liệu mô phỏng vì Cloudflare không có MySQL. Các bảo đảm chống trùng được kiểm chứng trên bản Node + MySQL bằng test 100 yêu cầu đồng thời."*

RISK-02 — Job định kỳ trên bản Worker (đã sửa 08/11/2026) Low (đã sửa)

1. Mô tả	Trước 08/11/2026 các job hết hạn giữ chỗ, hóa đơn kỳ, quá hạn, gia hạn, tự chốt quyết toán, SLA, kích hoạt chính sách, số liệu ngày không chạy trên bản demo. Nay chạy từ cron mỗi 5 phút; job chỉ chạy khi đến hạn, chạy lại nếu lần trước lỗi.
2. Nguyên nhân	jobs.service.ts chỉ chạy khi config.jobs.enabled (bản Node); hàm scheduled() của Worker trước đó chỉ gửi email.
3. Mức độ nghiêm trọng	Low (đã sửa)
4. Cách phát hiện	Xem log cron (wrangler tail): dòng [cron] jobs liệt kê kết quả từng job.
5. Tái hiện	Đặt kho, bỏ thanh toán, đợi khoảng 20 phút: đơn chuyển EXPIRED.
6. Ảnh hưởng	Không còn ảnh hưởng; độ trễ tối đa khoảng 5 phút so với lịch của bản Node.
7. Khắc phục	Đã làm: EdgeApp.runDueJobs() + runCron() trong index.ts. Lượt đầu 08/11/2026 phát hiện và sửa lỗi khóa unit_type_key của daily_facility_metrics trong edge-prisma; lượt sau 16/16 job đạt.
8. Trả lời khi bị phát hiện	*"Các job định kỳ chạy trên cả hai bản; bản Cloudflare dùng cron 5 phút."*

RISK-03 — Refresh token có bản sao trong localStorage Medium

1. Mô tả	Ngoài cookie httpOnly, refresh token còn được lưu localStorage và gửi qua header X-Refresh-Token.
2. Nguyên nhân	Trình duyệt chặn cookie bên thứ ba (Brave, Safari) khi frontend và API khác tên miền.
3. Mức độ nghiêm trọng	Medium
4. Cách phát hiện	F12 → Application → Local Storage → vaultra_rt_fallback.
5. Tái hiện	Đăng nhập, mở DevTools.
6. Ảnh hưởng	Nếu trang bị XSS, kẻ tấn công có thể lấy refresh token.
7. Khắc phục	Đưa frontend và API về cùng tên miền để dùng cookie first-party, bỏ bản dự phòng.

8. Trả lời khi bị phát hiện	*"Đây là đánh đổi để đăng nhập hoạt động trên trình duyệt chặn cookie bên thứ ba; hướng khắc phục là dùng chung tên miền."*

RISK-04 — Client secret Google ghi cứng trong mã gateway High

1. Mô tả	Giá trị client secret OAuth Google nằm trong apps/gateway/src/shims/config.ts và đã có trong lịch sử git.
2. Nguyên nhân	Viết cứng khi cấu hình nhanh để đăng nhập Google chạy trên Worker.
3. Mức độ nghiêm trọng	High
4. Cách phát hiện	Mở file config.ts của gateway.
5. Tái hiện	—
6. Ảnh hưởng	Ai đọc được repo có thể giả mạo ứng dụng OAuth.
7. Khắc phục	Tạo secret mới trên Google Cloud, đọc từ wrangler secret GOOGLE_CLIENT_SECRET, xóa khỏi mã.
8. Trả lời khi bị phát hiện	*"Nhóm đã nhận diện và có kế hoạch đổi secret, chuyển sang biến môi trường bí mật."*

RISK-05 — Khóa Gemini mặc định trong mã gateway High

1. Mô tả	Danh sách khóa API Gemini mặc định nằm trong config.ts của gateway.
2. Nguyên nhân	Như RISK-04.
3. Mức độ nghiêm trọng	High
4. Cách phát hiện	Mở file.
5. Tái hiện	—
6. Ảnh hưởng	Lạm dụng hạn mức API của nhóm.
7. Khắc phục	Thu hồi khóa trên Google AI Studio, chuyển sang secret GEMINI_API_KEYS.
8. Trả lời khi bị phát hiện	*Như RISK-04.*

RISK-06 — Mật khẩu chung cho tài khoản demo Medium

1. Mô tả	Mọi tài khoản demo dùng một mật khẩu, nằm trong seed và file tkmk.txt được theo dõi bởi git.
2. Nguyên nhân	Thuận tiện cho demo.
3. Mức độ nghiêm trọng	Medium
4. Cách phát hiện	Đăng nhập thử bất kỳ tài khoản demo.
5. Tái hiện	—
6. Ảnh hưởng	Người ngoài có thể vào bản demo với vai trò admin.
7. Khắc phục	Đổi mật khẩu trên bản deploy sau buổi bảo vệ; xóa tkmk.txt khỏi repo.
8. Trả lời khi bị phát hiện	*"Đây là tài khoản demo; trên môi trường thật mỗi người tự đặt mật khẩu qua lời mời."*

RISK-07 — Đối soát VNPay trên bản demo là mô phỏng Medium

1. Mô tả	Cột "Theo cổng" trên bản demo lấy từ dữ liệu hệ thống khi không tra được VNPay.
2. Nguyên nhân	API tra cứu (querydr) của VNPay sandbox trả 522 khi gọi từ Cloudflare; từ máy cá nhân gọi được.
3. Mức độ nghiêm trọng	Medium

4. Cách phát hiện	Chạy đối soát: luôn Khớp nếu không dán sao kê.
5. Tái hiện	POST /reconciliations không có rows.
6. Ảnh hưởng	Không phát hiện được chênh lệch thật trên bản demo.
7. Khắc phục	Bật VNPAY_QUERY_API=true trên bản Node; hoặc nhập sao kê do VNPay cung cấp.
8. Trả lời khi bị phát hiện	*"Bản demo mô phỏng sao kê vì sandbox không cho máy chủ Cloudflare kết nối; có thể dán sao kê để thấy chênh lệch."*

RISK-08 — SMTP đôi lúc từ chối đăng nhập (535) Medium

1. Mô tả	Một số email OTP gia hạn ghi FAILED với lỗi "535 Incorrect authentication data", lần sau gửi được.
2. Nguyên nhân	Chưa xác định: có thể nhà cung cấp giới hạn số lần đăng nhập SMTP hoặc mật khẩu thay đổi.
3. Mức độ nghiêm trọng	Medium
4. Cách phát hiện	Xem notification_deliveries có status FAILED.
5. Tái hiện	Gửi OTP liên tục nhiều lần.
6. Ảnh hưởng	Khách không nhận OTP, không ký/gia hạn được.
7. Khắc phục	Đã thêm thử lại ngay 1 lần cho email trực tiếp, bộ gửi thử lại 5 lần có giãn cách; nên dùng nhà cung cấp email giao dịch.
8. Trả lời khi bị phát hiện	*"Email là dịch vụ ngoài; hệ thống ghi nhận lỗi và cho gửi lại; tại quầy có phương án ký OTP."*

RISK-09 — Dữ liệu mẫu của bản demo từng thiếu liên kết Low (đã sửa)

1. Mô tả	83/104 đơn đặt thuê tham chiếu báo giá không tồn tại; một số hóa đơn/sổ cái thuộc khách đã bị xóa.
2. Nguyên nhân	Snapshot mẫu xuất từ DB phát triển không giữ ràng buộc khóa ngoại.
3. Mức độ nghiêm trọng	Low (đã sửa)
4. Cách phát hiện	Trang hủy đơn cũ lỗi 500 (cancellation-preview).
5. Tái hiện	—
6. Ảnh hưởng	Màn hình lỗi với dữ liệu cũ.
7. Khắc phục	Hàm repairIntegrity() dựng lại báo giá thiếu và dọn bản ghi mồ côi khi nạp dữ liệu.
8. Trả lời khi bị phát hiện	*"Lỗi dữ liệu mẫu đã được phát hiện bằng kiểm tra toàn vẹn và sửa tự động."*

RISK-10 — Không tua thời gian được trên bản deploy Medium

1. Mô tả	Đồng hồ thử (ENABLE_TEST_CLOCK) chỉ bật ở môi trường phát triển.
2. Nguyên nhân	Bảo vệ dữ liệu thật khỏi bị tua giờ.
3. Mức độ nghiêm trọng	Medium
4. Cách phát hiện	—
5. Tái hiện	—
6. Ảnh hưởng	Không demo trực tiếp được mốc D+6, D+10…
7. Khắc phục	Demo bằng bản Node cục bộ có test clock hoặc trình bày kết quả test.

8. Trả lời khi bị phát hiện	*"Tua thời gian chỉ bật ở môi trường thử để không ảnh hưởng dữ liệu thật."*

RISK-11 — Một số API GET có ghi dữ liệu Low

1. Mô tả	GET appointment-slots tự tạo khung giờ mặc định; POST /quotes công khai tạo bản ghi.
2. Nguyên nhân	Tiện cho cơ sở mới chưa cấu hình.
3. Mức độ nghiêm trọng	Low
4. Cách phát hiện	Đọc mã.
5. Tái hiện	—
6. Ảnh hưởng	Bảng phình, khó cache.
7. Khắc phục	Tạo khung giờ khi cấu hình cơ sở; giới hạn tần suất báo giá.
8. Trả lời khi bị phát hiện	*"Nhóm đã ghi nhận trong báo cáo kiểm tra (API-03)."*

RISK-12 — Mở/đóng kho từ xa là mô phỏng Low

1. Mô tả	Nút Mở/Đóng kho ghi access_events, không điều khiển khóa thật.
2. Nguyên nhân	Khóa thông minh ngoài phạm vi giai đoạn 1.
3. Mức độ nghiêm trọng	Low
4. Cách phát hiện	—
5. Tái hiện	—
6. Ảnh hưởng	Không có.
7. Khắc phục	Nối AccessControlProvider khi có phần cứng.
8. Trả lời khi bị phát hiện	*"Đặc tả để khóa thông minh ngoài phạm vi, nhóm làm sẵn giao diện và nhật ký."*

RISK-13 — E2E chưa chạy lại sau các thay đổi gần nhất Medium

1. Mô tả	f4-f7-lifecycle, rbac, vnpay, audit-findings chưa chạy lại sau khi bỏ payOS và sửa Worker.
2. Nguyên nhân	Giới hạn đăng nhập 429 khi chạy trên Worker; chưa dựng lại MySQL.
3. Mức độ nghiêm trọng	Medium
4. Cách phát hiện	—
5. Tái hiện	—
6. Ảnh hưởng	Có thể có lỗi hồi quy chưa phát hiện.
7. Khắc phục	Chạy pnpm test:e2e trên bản Node + MySQL trước ngày bảo vệ và ghi kết quả.
8. Trả lời khi bị phát hiện	*"Kết quả gần nhất của từng bộ test ghi ở mục 4.6, kèm những bộ chưa chạy lại."*

RISK-14 — Độ trễ API trên bản demo 0,7–1,3 giây Low

1. Mô tả	Một số API (danh sách hợp đồng) mất khoảng 1 giây.
2. Nguyên nhân	Dữ liệu trong bộ nhớ, instance mới phải nạp snapshot.
3. Mức độ nghiêm trọng	Low
4. Cách phát hiện	curl -w "%{time_total}".

5. Tái hiện	Mở danh sách hợp đồng nhiều lần.
6. Ảnh hưởng	Màn hình hiện khung tải lâu hơn.
7. Khắc phục	Làm nóng trước demo; chuyển sang DB thật.
8. Trả lời khi bị phát hiện	*"Đây là độ trễ của bản demo mô phỏng; bản Node + MySQL nhanh hơn."*

RISK-15 — Realtime chỉ bật sau khi đăng nhập Low

1. Mô tả	Trang công khai (đặt kho) không tự cập nhật số kho trống khi người khác đặt.
2. Nguyên nhân	Vé realtime cấp cho người đã đăng nhập.
3. Mức độ nghiêm trọng	Low
4. Cách phát hiện	—
5. Tái hiện	Hai trình duyệt chưa đăng nhập.
6. Ảnh hưởng	Số kho trống có thể cũ vài giây; khi đặt vẫn kiểm tra lại ở backend.
7. Khắc phục	Thêm kênh công khai cho số liệu khả dụng.
8. Trả lời khi bị phát hiện	*"Dữ liệu hiển thị có thể cũ nhưng lúc đặt backend luôn kiểm tra lại, nên không bán trùng."*

8.3 Lỗi đã phát hiện và đã sửa trong quá trình hoàn thiện

Danh sách dưới đây là bằng chứng nhóm có quy trình phát hiện – sửa lỗi. Nếu giảng viên hỏi "đã gặp lỗi gì khó nhất", đây là nguồn câu trả lời thật.

Triệu chứng	Nguyên nhân gốc	Cách sửa
Sơ đồ kho trắng trang khi di chuột vào kho có khách	Tooltip hiển thị cả object khách thay vì tên	Hiển thị customer.full_name; thêm ErrorBoundary
Xóa/bật tắt tài liệu RAG báo "không tồn tại"	Giao diện dùng public_id, API trả id	Dùng đúng trường id; sửa luôn lỗi chỉnh sửa tạo bản trùng
Đăng nhập Google báo lỗi kết nối	Callback (GET) tạo phiên nhưng không được lưu lên R2	Lưu thay đổi của mọi request có ghi dữ liệu
Tab Truy cập báo "Lỗi hệ thống"	Mã PIN (byte) bị ghi sai định dạng vào snapshot	Sửa tuần tự hóa và tự sửa dữ liệu cũ
Ảnh hiện trạng báo "Tệp chưa tải lên xong"	Request đồng thời ghi đè snapshot	Khóa tuần tự thao tác ghi
Email sự kiện không được gửi	Worker không có bộ gửi email	Bộ gửi chạy sau thao tác ghi và theo cron
Nút bấm treo	Gửi SMTP giữ khóa	Gửi email ngoài khóa; khóa tự nhường sau 6 giây; timeout 40 giây ở frontend
Job định kỳ không chạy trên bản demo (giữ chỗ không hết hạn, không nhắc hạn, không chuyển mốc quá hạn)	Worker chưa gọi JobsService	Chạy job đến hạn từ cron 5 phút; sửa lỗi khóa số liệu ngày phát hiện ở lượt chạy đầu

8.4 Đối chiếu các nhóm lỗi thường bị hỏi

Nhóm lỗi	Hiện trạng trong VAULTRA
Frontend và backend không đồng bộ	Đã gặp (RAG id); realtime + invalidate giảm dữ liệu cũ; RISK-15.
Database không lưu đúng	Bản Node: giao dịch + ràng buộc. Bản demo: RISK-01, RISK-09.
Logic nghiệp vụ sai	Công thức chính có unit test; chưa phát hiện sai công thức.
Chuyển trạng thái không hợp lệ	states.ts + unit test; API trả 409.
Phân quyền sai	rbac.ts có 54 kiểm tra (theo TRACEABILITY), chưa chạy lại (RISK-13).
Thiếu validation	Zod ở mọi controller; test case invalid input ở Chương 4.
API không xử lý lỗi	Bộ lọc lỗi chung; email/thanh toán có trạng thái lỗi riêng.
Request trùng lặp	Idempotency-Key, UNIQUE, kiểm tra IPN trùng.
Lỗi transaction	Bản Node: $transaction; bản demo: undo log.
Lỗi bảo mật	RISK-03 → RISK-06.
Xử lý đồng thời	nfr03 đạt trên Node; RISK-01 trên Worker.
Lỗi hiển thị	Đã sửa sơ đồ trắng trang; ErrorBoundary.
Đồng bộ sau cập nhật	Realtime data.changed (08/11/2026).
Xử lý ngoại lệ	Abort phiên check-in, rollback giao dịch, thử lại email.

CHƯƠNG 9

Checklist ôn tập

> Đánh dấu khi đã nói lại được bằng lời của mình, không phải khi đã đọc qua.

9.1 Checklist riêng từng thành viên

Khang — F1, F7, mở đầu

Hạng mục	Nội dung	✓
Kiến thức bắt buộc	• Bài toán, 5 vai trò, phạm vi • Hình 1.1 và Hình 2.1 • Giữ chỗ 15 phút, khả dụng theo loại kho • Chống đặt trùng (khóa + UNIQUE + test 100 request) • IPN có chữ ký, return chỉ hiển thị • Idempotency-Key • SLA theo giờ làm việc, ghi chú nội bộ • Trạng thái đơn và ticket	☐
Chức năng phải demo	• Đặt kho + thanh toán thẻ test • Tạo ticket → nhân viên xử lý → đánh giá	☐
API cần biết	• POST /quotes, POST /reservations, POST /reservations/:id/pay, GET /payments/vnpay/ipn • POST /tickets, POST /tickets/:id/messages, POST /tickets/:id/:action	☐
Bảng database	• reservations, quotes, invoices, payments, deposits, unit_allocations • support_tickets, ticket_messages, ticket_events	☐
Trạng thái cần nhớ	• PENDING_PAYMENT → CONFIRMED → UNIT_ASSIGNED → MOVE_IN_SCHEDULED → CHECKED_IN • NEW → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED	☐
Câu hỏi luyện trả lời	• 12 câu ở mục 6.1 • Nhóm 7.1, 7.2, F1/F7 ở 7.3	☐
Lỗi có thể gặp khi demo	• Thiếu suất hẹn, quá 3 đơn chờ, VNPay chậm	☐
Học thuộc	• Lời mở đầu, câu chuyển tiếp, số liệu 306 API/104 bảng/100 request	☐

Triển — F2, F3

Hạng mục	Nội dung	✓
Kiến thức bắt buộc	• Wizard 4 bước và điều kiện cứng • PDF + SHA-256, chữ ký OTP/email • PIN mã hóa, active_pin_key • Biên bản ≥ 4 ảnh, magic bytes • Hóa đơn kỳ, phân bổ thanh toán • Công thức quyết toán, 3 ngày khiếu nại • 3 điều kiện kho về Trống	☐
Chức năng phải demo	• Check-in đơn của Khang • Xem PIN, mở/đóng kho • Trả kho + quyết toán	☐
API cần biết	• /checkin-sessions/* • /contracts/:id/access* • /contracts/:id/move-out*, /move-out-requests/*, /settlements/:id/finalize, /units/:id/final-check	☐
Bảng database	• checkin_sessions, contracts, contract_documents, contract_signatures, inspections, access_credentials, handover_records • move_out_requests, deposit_settlements, damage_items, refunds	☐
Trạng thái cần nhớ	• contracts: PENDING_SIGNATURE → SIGNED → ACTIVE → MOVE_OUT_SCHEDULED → ENDED → CLOSED • move_out_requests: SCHEDULED → INSPECTED → COMPLETED	☐
Câu hỏi luyện trả lời	• 15 câu ở mục 6.2 • F2/F3 ở 7.3, nhóm Database, Bảo mật	☐
Lỗi có thể gặp khi demo	• Email ký chậm, ảnh chưa tải xong	☐
Học thuộc	• Ví dụ số quyết toán 690.000 − 50.000 = 640.000	☐

Phúc — F4, kiểm thử

Hạng mục	Nội dung	✓
Kiến thức bắt buộc	• Chính sách có phiên bản, không hồi tố • Validate quan hệ mốc quá hạn • Khung giá, yêu cầu đổi giá • Khuyến mãi, miễn giảm, credit note • Phát hành / thu được / ghi nhận • Đối soát (mô phỏng trên demo) • 15 loại kiểm thử, kết quả test thật	☐
Chức năng phải demo	• Tạo + xuất bản phiên bản • Chạy đối soát VNPay và tiền mặt • Chạy pnpm test nếu được yêu cầu	☐

Hạng mục	Nội dung	✓
API cần biết	• /policies, /policy-versions/:id/(impact|publish|cancel) • /price-bands, /price-change-requests, /promotions, /fee-waivers • /reconciliations	☐
Bảng database	• policies, policy_versions, price_bands, promotions, fee_waivers, credit_notes, reconciliation_batches, daily_facility_metrics	☐
Trạng thái cần nhớ	• policy_versions: DRAFT → SCHEDULED → ACTIVE → SUPERSEDED	☐
Câu hỏi luyện trả lời	• 15 câu ở mục 6.3 • Nhóm Testing, API, Phản biện	☐
Lỗi có thể gặp khi demo	• Xuất bản chính sách sai trên bản demo	☐
Học thuộc	• Tham số mặc định 6.4.1 quan trọng: 15/60/7/5/10/30/45	☐

Trường — F5, F6, kết luận

Hạng mục	Nội dung	✓
Kiến thức bắt buộc	• Máy trạng thái kho • Thuật toán phân bổ 3 tiêu chí • Ca, việc, work order, bảo trì định kỳ • Gia hạn: cửa sổ 30 ngày, +10%, OTP • Chuỗi quá hạn và khôi phục • Chấm dứt cần FM + BOM • Job chạy từ cron Worker, không tua thời gian; realtime	☐
Chức năng phải demo	• Sơ đồ kho, đổi trạng thái • Giao việc, nhân viên nhận • Gia hạn hợp đồng 1 tháng	☐
API cần biết	• /units/:id/status, /reservations/:id/allocate, /tasks/*, /work-orders/* • /contracts/:id/renew(/otp), /delinquency, /overlocks, /termination-requests/*	☐
Bảng database	• units, unit_status_history, unit_allocations, work_tasks, staff_shifts, work_orders • renewal_offers, contract_terms, delinquency_events, overlocks, termination_requests	☐
Trạng thái cần nhớ	• units: AVAILABLE ↔ RESERVED → OCCUPIED ↔ OVERLOCKED → PENDING_INSPECTION • delinquency: CURRENT → GRACE → LATE → LOCKED_OUT → FINAL_NOTICE → DEFAULT_PENDING → DEFAULTED	☐
Câu hỏi luyện trả lời	• 15 câu ở mục 6.4 • F5/F6 ở 7.3, Kiến trúc	☐
Lỗi có thể gặp khi demo	• Hợp đồng không trong cửa sổ gia hạn; OTP không đến	☐
Học thuộc	• Bảng mốc D+1/6/10/30/45, lời kết luận	☐

9.2 Checklist kiến thức chung (cả 4 người)

☐ Mục đích dự án và 3 điểm khác biệt

☐ Đối tượng sử dụng: 5 vai trò, phạm vi OWN/FACILITY/GLOBAL

☐ Kiến trúc: React – NestJS – MySQL; bản demo Cloudflare khác ở đâu

☐ Công nghệ chính (chỉ những gì có trong package.json)

☐ Database: public_id, DECIMAL, open_unit_key, không xóa dữ liệu tiền

☐ 7 Flow: mục đích – ai làm – trạng thái đầu/cuối (30 giây mỗi Flow)

☐ Business logic quan trọng: khả dụng, giữ chỗ, điều kiện cấp mã, quyết toán, chính sách có phiên bản, chuỗi quá hạn, SLA

☐ Quy trình kiểm thử và kết quả thật (16/16 unit, 15/15 F3, nfr03)

☐ Các lỗi đã phát hiện và đã sửa (mục 8.3)

☐ Giới hạn hệ thống (RISK-01, 07, 10)

9.3 Kế hoạch ôn tập 7 ngày

Ngày	Cả nhóm	Khang	Triển	Phúc	Trường
D−7	Đọc Ch.1–2; thống nhất phân công	Ch.3 F1, F7	Ch.3 F2, F3	Ch.3 F4; Ch.4	Ch.3 F5, F6
D−6	Mỗi người kể lại 7 Flow trong 4 phút	Học lời thoại 6.1	Học lời thoại 6.2	Học lời thoại 6.3	Học lời thoại 6.4
D−5	Chạy test case Chương 4, điền Actual/Status	TC-F1, TC-F7	TC-F2, TC-F3	TC-F4 + chạy pnpm test	TC-F5, TC-F6
D−4	Sửa lỗi phát hiện (nếu có), chạy lại	Postman F1	SQL F2/F3	Postman quyền (401/403/404)	SQL F5/F6
D−3	Tổng duyệt lần 1 có bấm giờ	Demo F1, F7	Demo F2, F3	Demo F4	Demo F5, F6
D−2	Hỏi chéo Ch.7 (mỗi người 25 câu)	Nhóm 7.1–7.3	Nhóm 7.4, 7.9	Nhóm 7.8, 7.10, 7.14	Nhóm 7.11, 7.13
D−1	Tổng duyệt lần 2; chuẩn bị dữ liệu dự phòng; quay video	Checklist 5.3	Đơn/hợp đồng dự phòng	Kiểm tra API	Video, slide mốc F6
D0	Đến sớm 30 phút, làm nóng hệ thống, đăng nhập sẵn	—	—	—	—

CHƯƠNG 10

Xử lý tình huống khi bảo vệ

> Mục tiêu: trung thực, bình tĩnh, có phương án. Không nói dối, không khẳng định chức năng chưa làm là đã xong.

Những câu không bao giờ nói

- "Phần này chạy được hết, không có lỗi."
- "Cái này bạn kia làm, em không biết."
- "Chắc là do máy thầy/cô."
- "Em làm 100% theo đặc tả." (khi chưa có bằng chứng)

Tình huống	Cách xử lý	Câu nói mẫu
Giảng viên hỏi mà mình không biết	Thừa nhận ngắn, nói phần mình biết, đề nghị kiểm tra hoặc nhờ thành viên phụ trách.	"Phần này em chưa nắm chắc. Em hiểu là … ; để chính xác, em xin phép mở mã nguồn kiểm tra / em xin mời bạn Triển bổ sung."
Yêu cầu demo một chức năng bất ngờ	Xác định Flow, mở đúng tài khoản/role, nói trước sẽ làm gì; nếu ngoài phạm vi thì nói rõ.	"Dạ, chức năng này thuộc Flow 5, em đăng nhập tài khoản quản lý cơ sở để thao tác."
Website không hoạt động	Thử Ctrl+F5, kiểm tra /v1/health, đổi mạng 4G; sau 1 phút chuyển sang video dự phòng.	"Hệ thống đang không phản hồi, em xin phép trình bày bằng video quay lúc chạy thử và quay lại demo khi kết nối ổn định."
API trả lỗi	Đọc thông báo lỗi to lên — nhiều lỗi là đúng nghiệp vụ (409, 403).	"Hệ thống từ chối vì hợp đồng chưa ký — đây chính là quy tắc em vừa trình bày."
Database không kết nối (bản Node)	Chuyển sang bản demo Cloudflare; giải thích khác biệt.	"Em chuyển sang bản triển khai trên Cloudflare, dùng chung mã nghiệp vụ."
Chức năng hoạt động sai	Ghi nhận, mô tả kết quả mong đợi, nêu hướng sửa; không đổ lỗi.	"Kết quả đúng phải là …; nhóm em ghi nhận lỗi này và sẽ sửa ở …"
Hỏi sâu vào code của thành viên khác	Trả lời mức tổng quan rồi mời người phụ trách.	"Ở mức tổng quan, phần này xử lý ở checkin.service.ts …; chi tiết em xin mời bạn Triển."
Một thành viên không trả lời được	Thành viên khác bổ sung tự nhiên, không ngắt lời giảng viên.	"Em xin phép bổ sung ý của bạn Khang …"
Giảng viên phát hiện lỗi logic	Cảm ơn, xác nhận lại tình huống, đánh giá ảnh hưởng, nêu cách sửa.	"Cảm ơn thầy/cô. Đúng là trong trường hợp … hệ thống sẽ …; cách sửa là thêm kiểm tra … ở service …"
Hỏi vì sao thiết kế nghiệp vụ như vậy	Trả lời theo: yêu cầu đề bài/đặc tả → rủi ro nếu làm khác → cách nhóm chọn.	"Theo đặc tả mục 6.6, chấm dứt hợp đồng có hậu quả pháp lý, nếu tự động thì … nên nhóm em chọn để FM xác nhận và BOM duyệt."
Bị hỏi về giới hạn bản demo (DB mô phỏng, không tua thời gian)	Chủ động nêu, kèm bằng chứng thay thế (test).	"Bản Cloudflare không tua được thời gian; kết quả các mốc quá hạn em trình bày bằng unit test và e2e trên bản đầy đủ."
Hết thời gian giữa chừng	Cắt phần demo, giữ phần kết quả và điểm nhấn.	"Em xin tóm tắt kết quả: …"

PHỤ LỤC

Tra cứu nhanh

> Tài khoản, địa chỉ, lệnh, thuật ngữ và tình trạng tài liệu.

A.1 Tài khoản, địa chỉ, lệnh

Mục	Giá trị
Giao diện	https://vaultra-844.pages.dev
API	https://vaultra-gateway.zerngulon.workers.dev/v1 (health: /v1/health)
Khách	customer@vaultra.test
Nhân viên Cầu Giấy	staff.hn@vaultra.test
Quản lý cơ sở Cầu Giấy	fm.hn@vaultra.test
Quản lý vận hành	bom@vaultra.test
Quản trị	admin@vaultra.test
Mật khẩu	Mật khẩu demo chung của nhóm (không in trong tài liệu)
Thẻ VNPay sandbox	NCB 9704198526191432198 · NGUYEN VAN A · 07/15 · OTP 123456
Unit test	pnpm --filter @vaultra/api test
E2E (cần MySQL + API)	pnpm --filter @vaultra/api test:e2e
Seed dữ liệu demo	pnpm seed:demo
Chạy bản đầy đủ	start.bat hoặc infra/docker-compose.yml
Deploy	wrangler deploy --config apps/gateway/wrangler.toml · wrangler pages deploy apps/web/dist --project-name vaultra

A.2 Thuật ngữ

Thuật ngữ	Giải thích
Self-storage	Kho tự quản: khách thuê một ô kho và tự ra vào bằng mã/khóa.
Quote	Báo giá có hạn 15 phút, khóa giá khi đặt.
Allocation	Việc gán một kho cụ thể cho đơn/hợp đồng.
Credential	Phương tiện truy cập: PIN, thẻ, khóa.
Overlock	Khóa chặn vật lý khi khách nợ quá hạn.
Holdover	Hết hạn không gia hạn nhưng vẫn dùng tiếp theo tháng.
IPN	Instant Payment Notification: VNPay gọi máy chủ báo kết quả.
Idempotency	Gửi lại cùng yêu cầu không tạo kết quả thứ hai.

Thuật ngữ	Giải thích
Outbox	Bảng sự kiện ghi cùng giao dịch, gửi đi sau.
SLA	Cam kết thời gian phản hồi/giải quyết.
RBAC	Phân quyền theo vai trò.
ULID	Mã định danh 26 ký tự, khó đoán, có thứ tự thời gian.

A.3 Báo cáo tình trạng tài liệu

- **Đã kiểm chứng khi soạn (08/11/2026)**: unit test 16/16, audit unit 5/5; cron job trên bản deploy 16/16 job đạt (sau khi sửa lỗi số liệu ngày); e2e trên Worker cục bộ (f3 15/15, nfr03 5/5, f1 10/12, f2 17/24); realtime, mở/đóng kho, đối soát, gửi email trên bản deploy.
- **Chưa xác minh**: e2e f4-f7-lifecycle, rbac, vnpay, audit-findings trên bản hiện tại; các câu SQL chưa chạy trên MySQL; toàn bộ test case Chương 4 ở trạng thái Not Tested.
- **Tồn tại**: các rủi ro RISK-01 đến RISK-15 ở Chương 8.